Group Project Milestone 3:

Integration and Communication — Deployment, Final Report and Presentation

Predicting Delivery Performance and Satisfaction Risk
on the Olist Brazilian E-Commerce Marketplace

A date the customer can rely on, and help when it slips:
two prediction moments and a deployed application

Team 10

Pan Pinyou	A0329105L
Li Yudan	A0330335R
Zhu Gengyu	A0333773Y

National University of Singapore

IT5006 Fundamentals of Data Analytics

AY2026/2027 Semester 1 · Final report, November 2026

Revised after the presentation of 5 November 2026

Application: `https://team10-it5006-olist.streamlit.app`

Repository:
`https://github.com/gengyuzccc-cell/Team10_IT5006_Ecommerce_Analytics_AY2627Sem1`

Contents

Executive Summary **1**

1 Problem Definition and Stakeholder Context **2**

2 Literature Review **4**

3 Dataset and Preprocessing **6**

4 Exploratory Analysis Highlights **7**

5 Feature Engineering **9**

6 Modelling Methodology **10**

7 Results and Evaluation **11**

8 Model Deployment and Application **15**

9 Business Recommendations and Impact **16**

10 Limitations and Conclusion **19**

A Reproducing this report **26**

B Data model, cleaning decisions and dictionary **27**

C Success criteria registered in advance **31**

D Hyperparameter search spaces and selected values **31**

E Re-checking a Phase 1 claim about least squares **33**

F Full result tables **33**

G The cross-validation protocol, and every candidate on every window **34**

H Lateness anatomy: where the late deliveries came from **35**

I Responses to the Phase 1, Phase 2 and presentation feedback **37**

J Training, validation and test side by side **39**

K Feature catalogue **40**

L The pilot, per contact and as designed on slide 11 **42**

M When is the review written? **43**

N Contributions **46**

O Figures **47**

P Where the numbers on the slides come from **61**

Q The customer's view: the tables behind Sections 4 and 9 **70**

AI Tool Declaration **76**

Executive Summary

Every order on Olist — a Brazilian marketplace that places small sellers on large retail platforms and manages fulfilment for them — carries a promise to a customer: the delivery date shown at checkout. Its public dataset (99,441 orders in nine relational tables, 2016–2018) shows what keeping that promise is worth to the customer: 9.2% of customers whose parcel arrives on time rate the order one or two stars, against 54.0% of those whose parcel is late. Box 1 follows one of them, from checkout to a one-star review written 5.8 days before the parcel arrived. The project asks two questions at two moments of the customer's order, each for the team that can act on the answer. **Problem 1, at checkout** (operations): how long will this order take, and will it miss the date the customer is shown? **Problem 2, at delivery** (customer experience): will this customer rate the order one or two stars, while there is still time to help?

Three disciplines run through the work. Leakage is defined per prediction moment and enforced by a guard — one forbidden column lifts a late-order classifier's ROC-AUC from 0.589 to 1.000. The split is chronological, and the test window (orders placed 30 June to 30 August 2018) was scored once, for models already chosen. Three model families climb from their simplest variant, each rung compared with the one below.

The lead-time regression is off by **3.90 days** on average, 41% less than a constant fitted on the past and 60% less than the platform's own date; compared with that date it gives a late-order flag at 2.9 times the no-skill floor, where the best classifier trained on the label reaches 1.2 times. The low-review model reaches ROC-AUC 0.749 and PR-AUC 0.414, **3.75 times its floor**; at its own moment — each day's delivered parcels whose customer has not yet rated them — its top ten find 26 future 1–2 star reviewers in 100, 3.2 times chance.

The recommendations follow the customer's order and are counted first in customers (Table 9). *At checkout*, show the forecast plus an eight-day margin chosen on the validation months (the forecast alone is exceeded by 31.9% of deliveries): on the held-out window the date is broken for 25 in 1,000 delivered orders instead of 74, for a quoted wait 1.1 days longer. *When the promised day ends without a delivery* (69.5 in 1,000 orders placed; 55% of those who rated the order only afterwards gave it 1–2 stars), tell the customer that day — an apology, the status, a new date — and ask every customer for the review once the order is complete. *At delivery*, work the daily list from the top with a replacement or refund to offer, since customers whose parcel arrived complain about wrong or missing items, quality and damage. Expediting freight on the checkout flag is not recommended. Money comes second, and on assumed costs, since Olist publishes none: at R$5 a contact and R$40 a low rating, the promised-date message pays if it turns around about one unhappy customer in four, the delivery-day call one in two — an effect no historical data can show, which a randomised pilot of five to seven weeks would measure (Section 9.4).

Everything is deployed and reproducible. A Streamlit application at `https://team10-it5006-olist.streamlit.app` opens on one customer's order, serves the three models, replays any held-out day blind and then with its outcomes, and labels each figure observed, assumed, derived or planned. The repository regenerates every number in this document, 89 validation checks tie models, tables and text together, and the feedback on Phases 1 and 2 and on the presentation of 5 November 2026 — including where each number on the slides comes from — is answered in Appendices I and P.

1 Problem Definition and Stakeholder Context

1.1 The customer's promise, and two moments to keep it

Box 1. One customer's order order `e1320973`; `report/results/customer_story.csv`

16 July 2018. A customer in São Gonçalo (RJ) buys a R$64.99 health-and-beauty item (R$27.74 freight) from a seller 2,267 km away in São Luís (MA), and is shown a delivery date of 7 August, 21 days away. The deployed models expect 18.0 days and put the risk of missing the date at 18%, 23rd of the 238 delivered orders placed that day. **20 July.** The seller's shipping deadline passes; the carrier receives the parcel on 1 August. **7 August.** The promised date passes; the tables record no message to the customer. **9 August.** The satisfaction survey is sent. **11 August.** One star, titled *Bom* ("good"), with the comment *Não veio a embalagem certa* ("the right packaging did not come"; ambiguous). **17 August.** The parcel arrives 10.5 days late; the review is 5.8 days old. Of the 244 reviewed orders placed on 16 July, this one tops the review-risk ranking (0.85).

Under the recommendations of Section 9.2, checkout would have shown 12 August (also missed, by five days); a message would have reached the customer as 7 August ended, more than three days before the review, and a call at delivery 5.8 days after it. Eight of that day's top ten (5.7 on an average day) rated their order 1–2 stars — four waiting for a late parcel, four whose order never came — and all eight were without a parcel, and had not yet written, when their promised day ended. Whether a message would have helped them is what the pilot of Section 9.4 measures.

Of every 1,000 orders placed in the test window, 905 arrived by the promised date, 72 after it and 23 never arrived; 110 customers rated their order 1–2 stars, 71 of them with a parcel that came on time. Among late parcels later rated 1–2 stars, Box 1 is typical: for 71% the survey went out exactly two days after the promised date, and 77% were rated before they arrived. The project turns this into two predictions at two moments of the customer's order (Table 1), each owned by the team that can act on it.

Table 1: The two problems. They share one feature pipeline, one model-family budget and one dataset; what separates them is *when* in the customer's order the model runs.

	Problem 1 — delivery performance	**Problem 2 — satisfaction risk**
Question	How long will this order take, and will it miss the date the customer is shown?	Will the customer rate this delivered order 1–2 stars?
Customer need	a date they can rely on	help while a problem can still be put right
Moment	`at_checkout` — the order has just been placed	`at_delivery` — the parcel has arrived; the customer has not yet rated it
Who acts	Operations and logistics	Customer experience
Task	**1A regression**: lead time in days, on MAE. **1B classification**: delivered after the promised date, on PR-AUC against its no-skill floor	**Classification**: `review_score` ≤ 2, on PR-AUC against its floor and balanced accuracy

The brief permits two problems or one framed twice; we take two, because they are halves of one decision chain: operations acts before the parcel ships, customer experience after it arrives, while the customer can still be helped, and Problem 1's output is, in substance, Problem 2's most important input. Problem 1 is the brief's *dual framing*: one underlying quantity, two outputs, so the late-order flag *is* the regression's answer compared against the promised date; five classifiers trained directly on the binary label run alongside as a control arm (Section 7.2).

1.2 Where in the customer's order the platform can act

A prediction is only worth producing if someone changes a decision because of it — the lesson of the prescriptive delivery framework reviewed in Section 2 [23]. Table 2 follows the customer's order: what the customer needs at each moment, what the platform does, who does it and the evidence. The date applies to every order and is judged on the regression's accuracy; the delivery-day calls are rationed by capacity and judged on the top of a ranked list (Section 7.4).

Table 2: The customer's order and where the platform can act. Costs are the assumptions of Section 9.3; Table 9 counts the customers each step reaches.

Moment	Customer need	What the platform does	Who acts	Evidence; cost
Checkout	a date to rely on	quotes the forecast plus 8 days, chosen on validation	Operations	missed for 2.5% of delivered orders, not 7.4%; quoted wait +1.1 d
Promised day ends, no parcel	to hear first	that day: apology, status, new date, or a refund; asks for the review once the order is complete	Customer care	calendar, no model; 55% of those who rate later give 1–2 stars; R$5
Delivery	a fix if the parcel disappoints	contacts the top of each morning's ranking of deliveries not yet rated	Customer experience	26 in 100 rate 1–2 stars (3.2×); R$5
Review	to rate what happened	asks every customer; never filters a review, delays its publication or rewards it	Platform	—

Not recommended: expediting freight on the checkout flag (R$12): its precision peaks at 0.27, under the 0.30 it would need (Section 7.4).

1.3 Targets, and what the review score measures

All three targets come from the provided tables alone. *Lead time* is the time from purchase (`order_purchase_timestamp`) to arrival (`order_delivered_customer_date`), in days, defined for the 97.0% of orders that reached the customer. *Late* is delivery strictly after `order_estimated_delivery_date`, which carries no time component, so a parcel delivered during the promised day counts as late (Appendix Q). *Low review* is `review_score` ≤ 2, defined for the 99.2% carrying a review. Undefined rows are `NaN` and excluded, never imputed to a negative label: the event has not happened rather than gone missing, and a silent customer is not a satisfied one.

The target of Problem 2 is a platform rating, not the *perceived quality* of the survey literature, and the distinction — which the Phase 1 evaluation asked for — matters for what the model claims. Perceived service quality is measured by instruments such as E-S-QUAL [35] and e-LSQ [39]: self-reported and retrospective. A star rating is a behavioural trace left by the customers who choose to write one, and its J-shaped distribution — 57.8% five stars here, with a second mode at one star — is the signature of acquisition and under-reporting biases [20, 21]; a rating also moves with the product, its price and the seller's communication [10]. Problem 2 therefore predicts the *rating*, the customer's own recorded verdict, which later buyers read, and claims nothing about latent satisfaction; the consequence, that part of the residual is unobserved product and seller effects, is carried into Section 10.

1.4 Success criteria, registered in advance

Expected ranges for all six headline metrics were written into `config.EXPECTED_BANDS` before any model was tuned (Appendix C). They are the models' targets and a leak alarm — leakage shows as an implausible improvement — and all six landed inside, none above; the customer measures are in Table 9. The ceiling was stated as plainly: the largest swings in lateness are shocks to carrier capacity and to the platform's own promising (Section 4.2), so a late-delivery classifier reporting 95% accuracy here would be the majority-class rule or a leak.

2 Literature Review

This review was revised after the Phase 1 evaluation, which asked for four things the first version lacked: a distinction between survey-based perceived quality and platform ratings, applied rather than foundational citations for the model choices with a justification within the tree family, literature support for the features, and an evaluation aligned with a team's limited capacity. Each is addressed in the subsection it belongs to.

2.1 Fulfilment, delivery promises and satisfaction

The e-commerce service-quality literature is direct about which lever matters. Parasuraman et al. [35] decompose electronic service quality into efficiency, *fulfilment*, system availability and privacy, and find fulfilment — the accuracy of delivery promises and the condition in which goods arrive — among the strongest contributors to perceived quality; Rao et al. [39] sharpen this into logistics terms across 260 online retailers. Both measure a survey construct. What a marketplace observes is a rating, and ratings are not surveys: they are written by a self-selected minority, J-shaped by acquisition and under-reporting biases [20, 21], and moved by product, price and seller behaviour as much as by service [10]. Transactional evidence bridges the two: across more than 15 million marketplace orders, Deshpande and Pendem [11] trace a chain — logistics performance drives the rating a customer leaves, and that rating moves seller sales. That chain makes the rating more than a diagnostic: it is the customer's own verdict, later customers act on it, and it is the construct this project predicts.

The promise itself is a second decision variable. Cui et al. [9] show that committing to a faster delivery date raises sales but increases returns and depresses retention, so an estimated delivery date is a business decision, not a forecast, and should not be expected to behave like one. Prediction pays only when it feeds a decision: Kandula et al. [23] route machine-learned delivery outcomes into a scheduling optimiser and report cost savings of up to 10.2%, the pattern Table 2 follows.

2.2 Model families for tabular transactional data

Regularised linear and logistic models are the natural baseline for structured data of this size: cheap, interpretable and, as Section 7 shows, revealing about distribution shift. Within the tree family the literature knows two ways to improve on one tree: a random forest bags decorrelated trees to cut variance [5]; boosting fits trees to residuals with shrinkage to cut bias [15]. Across the large empirical comparisons — ten algorithms on eleven problems [6], 179 classifiers on 121 datasets [13], 45 tabular benchmarks against deep architectures [18, 44] and 41 classifiers on business credit data [29] — boosted and bagged tree ensembles are the consistent top tier, boosting usually ahead by a small margin. In

e-commerce and logistics the same family is the working choice: quantile regression forests for delivery times at JD.com [41], tree models for late supplier deliveries [1], and random forests or boosted trees in the published studies on this very dataset [14, 17, 49], none of which controls leakage by prediction moment — the gap Section 3 closes.

The boosted learner used here is `HistGradientBoosting`: LightGBM's histogram-binned design [26] implemented inside `scikit-learn` [42], so it stays within the brief's toolset rather than adding XGBoost [8]; it fits an order of magnitude faster than a comparable forest; and although it can route missing values natively, every family here shares one median imputation, so the rungs are compared like for like. Heterogeneous ensembles follow Wolpert [50]. The literature supplies the ranking; the ladder of Section 6 supplies what it cannot — whether, on this data, each rung beats the one below it.

2.3 Feature engineering on a relational order model

Deriving features from nine tables raises a problem a flat file does not: three of them are one-to-many against orders, so a naive join multiplies rows and reweights every aggregate towards multi-item orders, and each child relation must be aggregated to the analysis grain before joining. The feature groups then follow the literature. Origin–destination structure and distance are the backbone of published delivery-time models, from JD.com's arrival forecasts [41] to last-mile travel-time predictors [30]; the quoted promise encodes what the platform's routing knew [9]; order size, value and freight are the order-level controls through which Deshpande and Pendem [11] identify the logistics–rating link, and demand peaks stress fulfilment. The past performance of the parties to a transaction is the standard predictor in supply-chain delay models [1], and an as-of aggregate of a key's earlier outcomes is the time-respecting form of target encoding [31], which must be fitted inside the cross-validation loop. The general vocabulary of counts, ratios and calendar encodings follows Kuhn and Johnson [28] and Heaton [19].

Leakage is the dominant risk. Kaufman et al. [25] formalise it as information about the target unavailable at prediction time and identify the two forms that threaten this dataset, post-outcome features and improper partitioning; Kapoor and Narayanan [24] show how routinely both survive peer review, across 294 affected papers in 17 fields. Ours are their *illegitimate feature* and *no clean separation* categories, and the framing makes leakage relative to a *prediction moment*: delivery outcomes are forbidden when forecasting delivery and admissible when predicting a review not yet written. On partitioning, Bergmeir and Benítez [2] show that standard k-fold cross-validation is invalid for temporally ordered data.

2.4 Evaluation under imbalance, drift and capacity

Both classification targets are heavily imbalanced. Saito and Rehmsmeier [40] show that precision–recall curves are more informative than ROC curves under strong imbalance, because ROC-AUC is insensitive to a large true-negative mass; PR-AUC is reported here against its no-skill floor, with raw accuracy as a diagnostic beside the majority-class rule. The obvious correction is one the project tests rather than applies: van den Goorbergh et al. [47] find that under-sampling, over-sampling and SMOTE [7] do not improve discrimination and badly miscalibrate the resulting probabilities, and recommend adjusting the decision threshold on uncorrected data instead (Section 7.3 reports the test). Calibrated probabilities are then a requirement, because a cost-sensitive decision multiplies them by a cost [12], and calibration is reported, not assumed [33, 37].

Regression errors are judged against a naïve benchmark [22], decisive here because delivery performance improves dramatically across the window: under such a level shift R^2 against a baseline fitted on the past turns negative even for a useful model, whereas mean absolute error stays interpretable; this is concept drift [16], whose recommendation that non-stationary models be refitted with respect to time shapes the deployment plan. A ranking metric summarises the whole list, but a team contacts only the K riskiest orders it can reach in a day, so the quantity that describes its day is the precision and recall of that top-K — the targeting view of Provost and Fawcett [38], and the top-decile lift that profit-driven evaluation uses in place of accuracy [29, 48]. Whether an intervention changes the outcome is a causal question observational data cannot answer; the standard instrument is a randomised controlled experiment [27], which Section 9.4 designs. The deployment follows the practice literature: a model card stating what is served and where it fails [32], and an explicit account of the version and data dependencies that quietly break production systems [43]. All modelling uses `scikit-learn` pipelines [36], keeping preprocessing inside each fold.

2.5 The gap addressed

The service-quality literature measures fulfilment by survey while the marketplace measures a rating; the tabular literature converges on tree ensembles without saying *when* the model would run; and the methodological literature shows that on imbalanced, temporally ordered data the split, the metric and the imbalance handling decide whether a number means anything. The contribution here is narrow: make the prediction moment a first-class design decision, measure its cost, evaluate at the capacity the stakeholder has, and count the result in customers before reais.

3 Dataset and Preprocessing

3.1 Nine tables, one row per order

The dataset [34] comprises nine related CSV tables, roughly 1.55 million rows; `orders` is the spine, at 99,441 rows, one per order, and the other eight attach to it directly or through `order_items` (Figure 1; Table 13 in Appendix B is the full data dictionary). Three tables are **one-to-many** against orders, and joining them naively is the first trap the data sets: a direct orders–items–payments merge yields 118,434 rows rather than 99,441, a 19% inflation that silently reweights every mean towards large baskets. Each child table is therefore aggregated to one row per order — counts, sums and the attributes of the most expensive item — and the grain is asserted after the join. All data come from these nine tables, the brief's only data source. Outside information enters in two places only, each labelled where it is used: two news reports date the May 2018 strike, and the costs of Section 9.3 are assumptions, not data.

The second trap is identity. `customer_id` is regenerated for every order, so it is an order key, not a person: 99,441 distinct values map to only **96,096** `customer_unique_id` values. Grouping on the wrong one reports zero repeat buyers; the right one finds 2,997 people (3.1%) who ordered again. `customer_unique_id` is carried throughout.

Missingness is **structural rather than random**: 97.0% of orders reach status *delivered*, and the 2.98% with no delivery timestamp are orders whose delivery has not happened, so their delivery targets are undefined and excluded rather than imputed. Elsewhere the data is remarkably complete: 99.2% of orders carry a review and product attributes are

missing for under 2% of products. The remaining issues — reused `review_id` values and orders with several reviews (keyed on `order_id`, earliest kept), free-text comments, written with the rating and so never features (Section 4.4 reads them descriptively), a million geolocation rows collapsed to a median coordinate per postcode prefix, products without a category — are recorded with their evidence as `CLEANING_DECISIONS` in `src/data_load.py` and tabulated in Table 11 (Appendix B). Numeric imputation (median) happens inside each model pipeline, never on the frame.

3.2 Analysis window and the three-way split

The raw file spans September 2016 to October 2018, but the first four months are a pilot contributing 329 orders against a 2018 run rate above 6,000 a month, and the final weeks are truncated mid-cycle (Figure 2), so the analysis is restricted to **orders placed from 1 January 2017 to 30 August 2018**, leaving **99,091** orders.

Each period of the split has exactly one job. **Train** (1 January 2017 to 29 April 2018, 73,008 orders) fits every model, with expanding-window cross-validation inside it tuning hyperparameters. **Validation** (30 April to 29 June 2018, 13,156) chooses between tuned models, sets every decision threshold and fits the margin-to-risk calibration. **Test** (30 June to 30 August 2018, 62 purchase days, 12,927) is scored once, at the end, for models already chosen, and is the held-out window the deployed application replays (Section 8). The split is chronological rather than stratified-random because the target drifts (Section 4) and a random split would train on the future; the positive rate of each period is reported instead — late orders 8.8%, 5.0% and 7.4% of the training, validation and test periods, low reviews 15.8%, 11.4% and 11.0%. A purely temporal cut can still put one person's orders on each side, so the **227** straddling buyers are removed from a clean test subset used in Section 7.3.

3.3 Leakage: one rule per prediction moment

Leakage is defined per *prediction moment*, because what is known depends on when the model runs. *At checkout* nothing about fulfilment or the customer's opinion exists, so delivery dates and everything derived from them, the review and its timestamps, `order_approved_at`, `shipping_limit_date` and `order_status` are forbidden. *At delivery* only the review and its timestamps remain forbidden: the delivery outcome is now history, and the brief lists "delivery performance" among that problem's features. `assert_no_leakage(features, regime)` enforces the set before any model is fitted, is tested in both directions and raises on an unknown regime.

`order_estimated_delivery_date` is admissible at checkout: the customer sees that promise when ordering, so it precedes the outcome and encodes what Olist's routing knew — a useful feature and (Section 7.1) a poor prediction. Getting the regime wrong is measurable: one forbidden column, the realised delivery time, lifts the late-delivery model's ROC-AUC from 0.589 to **1.000** (Figure 9) — not a result but the target restated.

4 Exploratory Analysis Highlights

Four findings from the exploratory phase shaped everything that followed; the figures are in Appendix O and every number is regenerated by `src/report_stats.py`.

4.1 Growth, seasonality and drift

Monthly order volume rises roughly **eight-fold** across the window, from about 800 orders in January 2017 to 6,000–7,500 a month through 2018, peaking at **Black Friday, November 2017** with 7,544 orders, 1.5 times the median month (Figure 3a; the weekly rhythm and the lead-time distribution are in Figure 4).

The most consequential temporal finding is not seasonality but **drift**. Mean lead time falls from 16.9 days in February 2018 to 7.7 in August 2018 — Olist materially improved fulfilment over twenty months (Figure 3b) — and across the chronological split the mean falls by **5.34 days** between training and test. Under a level shift this size, R^2 against a baseline fitted on the past is negative even for a useful model, so mean absolute error leads the regression results [22], and periodic refitting is a deployment requirement rather than an optimisation [16].

4.2 Lateness: volatile, with identifiable shocks

Lateness behaves differently: it is **volatile rather than trending**, swinging from 1.4% (June 2018) to 21.4% (March 2018). The Phase 1 report attributed the spikes to Black Friday and to the May 2018 truckers' strike; the evaluators pointed out that the figure did not support the second claim, and `src/lateness_anatomy.py` re-examined the series at three resolutions (Appendix H, Figure 23). Black Friday 2017 is a genuine spike (14.3%). The largest peaks, February and March 2018 (16.0% and 21.4%), are a network-wide **carrier-side slowdown**: transit from carrier handoff to doorstep lengthened from 8.7 to 13.2 days while sellers' handling time did not move, and it coincided with the platform *tightening* its promised window from 26.3 days in January to 22.0 in March. The truckers' strike of 21 May to 1 June 2018 [4, 45] is real but invisible at monthly resolution: orders caught in transit were 16.9% late against a 5.0% average over the preceding five weeks, and the platform then lengthened its promise from 19.5 to 36.8 days, which is why June 2018 is the quietest month in the series. None of these shocks is an attribute of an order, so nothing available at checkout can anticipate them — the ceiling Section 1.4 states in advance.

4.3 Geography, products and payment

The marketplace is geographically **lopsided**, and that asymmetry drives the delivery problem: **59.7%** of sellers sit in São Paulo state against **42.0%** of customers, only 35.7% of orders stay within the customer's own state, and the median customer–seller distance is **434 km** (Figure 5). Distance is the strongest checkout-time correlate of lead time ($r = 0.39$), ahead of the platform's own estimate ($r = 0.38$). The full correlation matrix (Figure 8) adds three facts the models inherit: promise and distance correlate 0.52, so the promise is largely a distance rule; freight tracks weight (0.64) rather than price; and review score correlates -0.33 with lead time, -0.12 with the number of items and essentially zero with price. The item count matters more than that suggests: among parcels that arrived on time, orders from two or more sellers draw 1–2 stars 47.1% of the time against 7.5% for a single item, and 84% of those unhappy customers who commented say an item was wrong, missing or never came (Appendix Q). Demand is long-tailed across 74 categories; the typical order is small (median item value R$86.90) but freight is not a rounding error (median R$17.17, 22% of item value), and 51.4% of orders are paid in instalments (Figure 6).

4.4 Reviews, and the relationship that dominates

Review scores are severely imbalanced — **77.1%** are four or five stars — exactly the imbalance the brief warns about; the interesting structure is what produces the minority. Late parcels are rated 1–2 stars **54.0%** of the time against **9.2%** on time, **5.9** times as often, monotone in how late the parcel is (Figure 7). This is the clearest relationship in the dataset, and it reproduces at order level the logistics–rating link Deshpande and Pendem [11] establish across 15 million orders. It also has a methodological consequence: a model predicting the review *at checkout* cannot see the variable that matters most.

What unhappy customers wrote. Three in four 1–2 star reviews carry a comment. Read with a keyword dictionary checked by an AI assistant on a held-out sample (80–96% precision by topic; Appendix Q), they describe two failures. Customers whose parcel was late or never came write that they do not have it (60% and 58% of comments), and those whose order never came write about refunds (25%). Customers whose parcel came on time write about what was in it: a wrong or missing item (34%), quality or a product unlike its description (17%), damage or a defect (15%). Comments are written with the rating, so they are read here and never used as a feature.

4.5 From candidates to the two problems

Three candidates were probed on the chronological split with untuned pipelines (Figure 10; Table 12 in Appendix B). *Delivery performance framed twice* became Problem 1: the regression reached MAE 3.89 days against 6.65 for a constant, while the direct late classifier was weak (PR-AUC 0.121 against a 0.074 floor). *Low review at delivery* became Problem 2 (PR-AUC 0.402 against 0.110); the same target at checkout reached only 0.212, which made the prediction moment a design decision. *Repeat purchase within 30 days* was rejected: a 1.8% base rate, balanced accuracy 0.517 and right-censoring at the natural 90-day horizon.

5 Feature Engineering

5.1 The 36 features, and why each is there

Phase 1's 25 features carry forward unchanged: 6 calendar (month, weekday, hour, weekend, business hours, the Black Friday window), 11 basket-and-money (`log_price`, `freight_ratio`, `log_weight`, volume, item and seller counts, instalments), 4 geographic (`log_distance`, `same_state`, customer state, seller region), 2 for the promise and 2 catalogue. The design carries no exact duplicates, and `purchase_year` is absent because the split is chronological, so a linear term on it would extrapolate into an unseen year. Phase 2 added **eight track-record features**, taking the checkout set to 33 and the at-delivery set to 36: a seller's mean lead time, late rate, low-review rate and order count; a route's mean lead time; a category's low-review rate; and the platform's own lead time and late rate. The at-delivery regime additionally sees three columns describing how delivery actually went.

Each group has a reason in the literature of Section 2 and a screen in the data. Geography and the promise are the backbone of delivery-time models [9, 30, 41]; basket, money and calendar are the order-level controls of the logistics–rating studies [11]; track records are the standard predictor in supply-chain delay models [1] in the time-respecting form of target encoding [31]. The empirical screen was the correlation matrix of Figure 8: lead time correlates 0.39 with distance, 0.38 with the promised window, 0.17 with freight

and 0.07 with weight; promise and distance correlate 0.52 with each other, so both are kept and the trees resolve the overlap. Table 24 in Appendix K lists all 36 features with definition, rationale and the moment each becomes knowable.

5.2 Track records, and the trap inside them

A seller's history is the strongest thing a marketplace knows about an order at checkout, and the easiest place here to leak: *past* must mean the earlier order had **closed** before this one was placed, not merely that it was placed earlier — otherwise every order averages in outcomes that had not yet happened, Kaufman et al.'s *illegitimate feature* [25], invisible in the metrics because it simply looks like a better model. Every aggregate is therefore an as-of join against the moment the earlier outcome became observable, with a strict boundary. Below five prior orders a fallback chain takes over (seller → route → seller state → platform), `hist_seller_n` carries how many orders the estimate rests on, and the platform features use a **28-day trailing window**, because lead time halves across the window (their range, 5.2 to 17.1 days, measures the drift). The validation suite rebuilds `hist_seller_lead_days` from first principles for 200 random orders and requires exact agreement, and checks that the platform tracker lags rather than anticipates (Appendix A).

6 Modelling Methodology

6.1 Model family budget and pipelines

The brief allows two to three families and asks for depth rather than breadth. We use three, reused across both task types, each arranged as a *complexity ladder* whose rung 0 is the family's simplest variant — the only way "the added complexity was worth it" becomes a finding rather than an assumption. **A — Linear** runs `LinearRegression` / `LogisticRegression` at rung 0 and `Ridge` / ℓ_2-penalised logistic at rung 1. **B — Tree-based** runs `DecisionTree` → `RandomForest` → `HistGradientBoosting`, the three steps Section 2 argued from the literature. **C — Ensemble** is soft `Voting` and `Stacking` assembled from the winners of A and B; its members are complete pipelines rather than bare estimators, because the two families need different encodings and one shared preprocessor would force a single choice on both. The full specification, the search spaces and the selected values of every tuned model are in Appendix D.

Every model is a `scikit-learn` `Pipeline`: median imputation, one-hot encoding of the four categoricals with rare levels folded into one bucket, then — for the linear family only — standardisation, since trees are invariant to monotone rescaling and a shared penalty is not [36]. The linear family also drops one level per categorical: Phase 1 called least squares "rank-deficient, so its coefficients explode", and re-running it four ways (Appendix E) shows the predictions were always fine and only the coefficients were not; with full rank restored, Ridge and OLS agree to three decimals, so regularisation buys numerical stability here, not accuracy.

6.2 Cross-validation, and how a model gets chosen

Delivery performance drifts hard — mean lead time falls **5.34 days** between training and test — so shuffled k-fold lets a model learn from the future [2]. Hyperparameters are tuned by `TimeSeriesSplit` with five expanding windows inside the training period

and `RandomizedSearchCV` [3] over 30 draws per model (six for the costly random forests), scoring MAE or average precision.

Selecting between tuned models is a separate question, and both obvious answers fail here. Averaging cross-validated scores fails because the folds differ enormously in difficulty — lead time runs 11.5–16.5 days across the folds against 8.3 in the test window — so the *ordering* does not survive: cross-validated MAE ranks Ridge above the boosted trees, while validation and test rank them the other way round (Figure 11b). The validation period alone fails too: its two months pull in opposite directions, 8.2% late in May 2018 and **1.4%** in June. Each candidate is therefore scored on all six out-of-sample windows, **ranked within each**, and the winner is the best mean rank over the **three most recent** — ranks because only ranks are comparable across windows of different difficulty, three because they cover roughly February to June 2018 and neither odd month decides alone. All six windows are reported for every candidate in Appendix G; a shuffled, customer-disjoint `StratifiedGroupKFold` is computed only as a contrast (Figure 19). The selected models are *not* refitted on train plus validation before the test window is scored, so what is reported is exactly what is deployed.

6.3 Imbalance, metrics aligned to the action, and reproducibility

Both targets are imbalanced — 7.4% late and 11.0% low review in the test period — so accuracy alone is meaningless and every results row carries `majority_accuracy` beside `accuracy`, with PR-AUC reported against its no-skill floor [40]. Following van den Goorbergh et al. [47] we fit on the observed prior and move the *decision threshold* instead, tuned on the validation period by maximising F1, and Section 7.3 tests the alternative rather than assuming the result; no resampling is used anywhere [7]. Calibrated probabilities are a requirement rather than a nicety, because Section 9.3 multiplies them by a cost [12]; they are reported, not assumed [33].

A second alignment follows from Table 2. PR-AUC summarises the whole ranking and is the right criterion for *choosing* between models, but neither team acts on the whole ranking: customer experience contacts the K riskiest customers its agents can reach in a day, and a warning, where one is shown, goes to the riskiest orders. Section 7.4 therefore reports Precision@K and Recall@K for both flags [38, 48], with K as a share of the day's volume so that the same table holds on a quiet Tuesday and on Black Friday, and Section 9.3 chooses the operating point as a capacity rather than as a threshold.

`SEED = 42` throughout. `src/train_models.py` fits everything and writes the models, metadata and result tables, `src/customer_view.py` the customer counts; `src/phase3_stats.py` distils those into `report/stats_phase3.txt`, and no number enters this report that it cannot regenerate; every figure is drawn from the same tables. Artifacts record the `scikit-learn` version that wrote them and the loader checks it; `tests/test_leakage.py` runs **89** validation checks, including one that ties the deployed artifacts to the metadata and to this text (Appendix A).

7 Results and Evaluation

Everything below is the untouched test window — orders placed 30 June to 30 August 2018 — scored once, for models already chosen. Cross-validated and validation columns sit beside it because the disagreement between them is part of the result; Appendix J adds the in-sample training fit of the deployed models, and Section 7.6 carries the error

analysis by month and region. Figures are in Appendix O.

7.1 Problem 1A — lead time

Table 3: Lead-time regression: two reference points that are not models, then each family's complexity ladder. CV MAE is the mean over the five expanding folds with its fold-to-fold standard deviation. **Bold** is the selected and deployed model. Figure 11.

Model	Family	CV MAE ± sd	Val MAE	Test MAE	RMSE	R^2
Mean of the training period	baseline	—	—	6.645	7.572	−0.990
Olist's own delivery promise	baseline	—	—	9.788	12.155	−4.130
LinearRegression *(rung 0)*	A linear	5.242 ± 1.00	4.965	4.181	5.639	−0.104
Ridge *(rung 1)*	A linear	5.201 ± 0.95	5.164	4.309	5.775	−0.158
DecisionTree *(rung 0)*	B tree	5.539 ± 0.91	5.927	6.116	8.664	−1.606
RandomForest *(rung 1)*	B tree	5.457 ± 0.90	5.513	4.956	6.169	−0.321
HistGradientBoosting *(rung 2)*	B tree	5.495 ± 0.81	4.802	4.141	5.517	−0.057
VotingRegressor (A+B)	C ensemble	5.315 ± 0.87	**4.784**	**3.898**	**5.342**	+0.009
StackingRegressor (A+B)	C ensemble	5.443 ± 0.84	4.792	4.121	5.430	−0.024

The selected model puts mean absolute error at **3.90 days**: **41.3% below** a constant fitted on the training period and **60.2% below the platform's own promised date**. That second comparison needs care. A promised date is a commercial decision, padded because a broken promise costs returns and retention [9] — an excellent feature and a poor forecast — and the forecast is not a promise either: on its own it is exceeded by 31.9% of delivered test orders, which is why Section 9.2 adds a margin.

Three things matter more than the winner. **Regularisation does not pay:** Ridge is *worse* than plain least squares on validation (5.164 against 4.965) and test (4.309 against 4.181). **Inside family B complexity does pay**, monotonically on validation: 5.927 → 5.513 → 4.802. And **only the ensemble has a positive** R^2; the others' negative values are the signature of a level shift rather than of useless models, and the reason MAE leads [22]. The blend beats both members (3.898 against 4.181 and 4.141) because a tree cannot predict outside the range of its training targets, test-window lead times are shorter than anything common in training, and a linear model extrapolates the level (Figure 12); the 1:2 weights are fixed, not tuned.

7.2 Problem 1B — late against the promise

Table 4: The late-order flag, best of each route; the full seven-candidate table is in `report/results/clf_p1b.csv`. No-skill PR-AUC floor 0.074. Figure 14.

Route	Model	ROC-AUC	PR-AUC	lift	bal. acc	prec.	recall	F1
direct	best of five (HistGB)	0.584	0.092	1.24×	0.524	0.088	0.279	0.134
derived	predicted > promised, unfitted	0.569	0.109	1.47×	0.569	0.274	0.175	0.214
derived	**regression margin, calibrated**	**0.782**	**0.214**	**2.89×**	**0.705**	0.137	**0.826**	**0.235**

The deliverable is the derived route, and that follows from the framing fixed in Section 1 rather than from the numbers: Problem 1 is the brief's dual framing, so the flag *is* the regression's own answer compared against the promised date. The five direct classifiers answer a different question — does the flag need a model of its own? It does not. The derived route reaches **2.89**× the no-skill floor where the best direct classifier reaches 1.24×, and it costs no additional model: the margin $\hat{y}$ − promised is mapped to a probability by

a two-parameter logistic [37] fitted on validation. Even the zero-parameter rule — flag whenever the model expects a miss — beats every fitted classifier on PR-AUC.

The selection protocol got this one wrong. Applied to these seven candidates the rolling-origin rule of Section 6.2 ranks a direct classifier first: on the in-training windows the candidates sit within noise of one another, while on the forward window the derived route leads by a factor of 2.3. Those windows are dominated by the February–March 2018 slowdown and the strike (Section 4.2); a classifier can fit a regime, the derived route encodes a mechanism (Appendix G). We report the disagreement rather than credit the rule with this choice.

7.3 Problem 2 — low-review risk

Table 5: Low-review classification at delivery. CV is the mean PR-AUC over the five expanding folds with its fold-to-fold standard deviation (Figure 16; every fold in Table 18). Accuracy is reported in the text; the majority-class rule scores 0.890, flagging no one. No-skill PR-AUC floor 0.110. **Bold** is the selected and deployed model. Figure 15.

Model	Fam.	CV ± sd	Val ROC	PR-AUC	lift	bal. acc	prec.	recall	F1	Brier
LogisticRegression *(r0)*	A	0.465 ± .095	0.320	0.659	0.291	2.63×	0.636	0.328	0.365	0.345	0.090
LogisticRegression (ℓ_2) *(r1)*	A	0.466 ± .095	0.320	0.659	0.291	2.63×	0.636	0.317	0.371	0.342	0.090
DecisionTree *(r0)*	B	0.543 ± .092	0.389	0.722	0.396	3.59×	0.677	0.562	0.391	0.461	0.080
RandomForest *(r1)*	B	0.568 ± .089	0.403	0.744	0.405	3.67×	0.680	0.576	0.396	0.470	0.078
HistGradientBoosting *(r2)*	B	0.568 ± .088	0.407	0.749	0.415	3.76×	0.680	0.579	0.396	0.471	0.078
VotingClassifier (A+B)	C	0.561 ± .084	0.404	0.747	0.407	3.69×	0.683	0.477	0.422	0.448	0.079
StackingClassifier (A+B)	C	0.568 ± .087	**0.409**	0.749	0.414	3.75×	0.681	0.575	0.398	0.470	0.078

The selected model reaches **ROC-AUC 0.749**, **balanced accuracy 0.681** and **PR-AUC 0.414 — 3.75× the no-skill floor**. Its raw accuracy is 0.901 against a majority-class rule scoring 0.890, which is exactly why accuracy is printed but never led with. The tree family beats the linear one by 1.43× on PR-AUC — but a *single depth-limited decision tree* already reaches 0.396 of the eventual 0.414, so 96% of the achievable signal comes from one interpretable tree. And the model does not memorise buyers: on the 12,617 test orders from customers never seen in training it scores PR-AUC **0.415** against 0.414 overall. On the 12,571 delivered parcels alone it scores ROC-AUC 0.711 and PR-AUC 0.333, 3.5 times their 9.6% floor; the 284 reviewed orders that never arrived, 73% of them rated 1–2 stars, raise the overall figures (Appendix M).

What the fold-to-fold variation says. The Phase 2 evaluation asked for the spread behind each cross-validated number; Figure 16 and Table 18 show every candidate on every window. The spread, a standard deviation of about 0.09, is almost entirely a property of the *windows*: the mean over candidates runs from 0.376 on the first fold to 0.630 on the fourth, while within a window the candidates' sd has a median of 0.045. So the forest, the boosted model and the stack (CV means of 0.568 apiece) cannot be separated by cross-validation, the case for the rank-based rule of Section 6.2; the linear family is behind the trees on every window, so that conclusion is robust to the spread.

Class weighting: the ablation Phase 1 promised. On the best tree model, weighting the loss leaves ranking unchanged (ROC-AUC 0.749 to 0.747, PR-AUC 0.415 to 0.411) while the Brier score degrades by 71% (0.0778 to 0.1331) and mean predicted risk inflates

from 0.119 to **0.337** against an actual 0.110 (Figure 17). We therefore fit on the observed prior and move the threshold: tuned on validation by maximising F1, it lands at 0.19 rather than 0.50 and raises test F1 from 0.414 to **0.470**; calibration holds where the volume is (a gap under 0.013 in the bins holding 11,892 of the 12,855 orders).

7.4 Precision@K and Recall@K at a fixed capacity

Table 6: What a ranked list delivers at a fixed capacity, on the test window. K is the share of reviewed (late flag: delivered) orders acted on; the daily rows group them by purchase day, as on the slides, while the list a team would work groups delivered, not-yet-rated parcels by delivery day (text). Break-even precision is $c_{\text{action}}/c_{\text{miss}}$ (Section 9.3), if every action works. Figure 22.

	Review risk, at delivery			**Late flag, at checkout: derived / direct**		
Capacity K	**precision**	**recall**	**lift**	**precision**	**recall**	**lift**
top 1%	0.705	0.064	6.4×	0.270 / 0.111	0.036 / 0.015	3.6 / 1.5×
top 5%	0.678	0.307	6.1×	0.274 / 0.104	0.185 / 0.071	3.7 / 1.4×
top 10%	0.475	0.431	4.3×	0.258 / 0.093	0.348 / 0.126	3.5 / 1.3×
top 20%	0.300	0.544	2.7×	0.211 / 0.089	0.568 / 0.241	2.8 / 1.2×
top 30%	0.236	0.641	2.1×	0.175 / 0.086	0.707 / 0.349	2.4 / 1.2×
10 a day (4.8%)	0.572	0.248	5.2×	—	—	—
20 a day (9.5%)	0.440	0.378	4.0×	—	—	—
break-even precision	0.125 (R\$5 : R\$40)			0.300 (R\$12 : R\$40)		

Table 6 reads the predictions of Tables 4 and 5 as a team with limited capacity would. **Capacity, not the threshold, is the binding constraint for the review flag.** At a capacity of 5% of reviewed orders — about ten a day on this volume of 207 a day — two listed customers in three went on to rate the order 1–2 stars, 6.1 times random, covering 30.7% of such ratings; precision stays above 0.125, the break-even only if every contact worked, at every capacity (Figure 22a). **Not everyone on that list can still be reached at delivery.** At ten a day its 0.572 precision splits three ways: per 100 listed customers, 26.4 rated an order that never arrived, 22.0 had already written while waiting for a late parcel, and 8.8 could still be reached when the parcel came (Appendix M). Built as a team would actually work it — each morning, the previous day's deliveries whose customer has not yet rated the order — the daily top ten reaches 26 future 1–2 star reviewers in 100, 3.2 times the 8.1% base rate of that pool; late and undelivered orders are reached on their promised date instead (Section 9.2). **The derived late flag dominates the direct classifier at every capacity**, by a factor of 1.4 to 2.8 on precision, **and never clears the break-even for an expensive action**: its precision peaks at 0.274 and falls as K grows, under the 0.300 a R\$12 expedite would need.

7.5 Does the third family earn its place, and what is deployed

Family C is selected on both problems, but whether it earns that differs. On **lead time** it earns its place: the Voting blend improves on the better of its members by 0.24 days of MAE (5.9%) and is the only model in Table 3 with a positive R^2, for the extrapolation reason above. On **review risk** it does not: Stacking wins the rank vote by a nose (mean rank 1.67 against 2.00, ahead on three of the six windows), then matches the boosted trees to three decimals on ROC-AUC and comes in 0.001 *below* them on PR-AUC, although the ensemble was *flattered*: stacking's inner folds must partition the training set and so

cannot be an expanding window (Appendix D).

What is deployed. The Phase 2 report named the deployed review model inconsistently; the artifacts settle it. The served models, read back from the pickles (Table 7), are the **VotingRegressor** for lead time, the derived route with its two-parameter calibration for the late flag (not the rule's pick; Section 7.2), and the **StackingClassifier** for review risk, whose meta-learner weights the tree member 13.8 times the linear one. The single boosted model would lose nothing measurable, but switching to it after seeing test scores would be selection on test; a validation check ties artifact, metadata and this text to the same estimator.

7.6 Error analysis: where the models fail

Across geography the review model is stable: ROC-AUC runs 0.744 to 0.778 across the five regions, with lift over each region's floor between 3.6× and 5.0× (Figure 21b). The regression is not: MAE runs from 3.42 days in the Southeast to **6.84 days in the North**, and the bias is positive everywhere, +1.15 days in the Southeast to +4.77 in the North, in every test month (Figures 21a and 13): a model fitted on a slower past over-predicts once fulfilment improves, the strongest argument for refitting on a rolling basis. Table 23 (Appendix J) separates that drift from over-fitting: the blend's in-sample MAE (4.72) sits level with validation (4.78) and above test (3.90), and the stacked review model shows a tree ensemble's modest in-sample optimism (ROC-AUC 0.836 against 0.762 and 0.749 on the periods it never saw).

8 Model Deployment and Application

The proof of concept is a Streamlit application deployed on Streamlit Community Cloud at `https://team10-it5006-olist.streamlit.app`, built from the `deployment/` folder of the repository. It serves all three deployed models, and because the Olist extract ends in 2018 it does what the brief anticipates: it predicts on held-out historical records that stand in for new orders, and it can replay the test window a day at a time.

8.1 Architecture

Table 7: The deployed artifacts, as pickled. Thresholds and the margin-to-risk mapping were fitted on the validation period; nothing was refitted for deployment, so the served models are exactly the ones Section 7 evaluates.

Problem	Estimator served	Artifact	Decision rule
1A lead time	`VotingRegressor`: LinearRegression + HistGradientBoosting, weights 1:2	381 KB	quote the estimate plus 8 days, rounded up to a calendar day
1B late flag	logistic map of (predicted − promised days) to a probability	1 KB	flag at $p_{\text{late}} \geq 0.11$
2 low review	`StackingClassifier`: ℓ_2 LogisticRegression + HistGradientBoosting, logistic meta-learner	321 KB	flag at $p_{\text{low}} \geq 0.19$; rank for the top-K

Nothing in the application reimplements the training-time feature engineering: a form submission or an uploaded row is pushed through the same `src/features.py` builders the models were fitted on, so no feature is computed one way in training and another in production [43]. A seller's track record, which a form cannot supply, comes from

`models/history_lookup.csv` at each level of the training code's fallback chain. The artifacts total under 1 MB, record the `scikit-learn` version that wrote them (pinned in `requirements.txt`; the loader warns if it drifts) and are loaded once per server process.

8.2 What the application does

The application has nine tabs (Figures 25 to 28) and opens on the customer. **Customer journey** follows any reviewed test order — by default the order of Box 1 — from checkout to review: what the customer went through, what each deployed model said at that moment, which contact could still have reached them, and their own words. Five tabs are the exploratory analysis of Section 4, four with filters. **Predict** (Figure 26) scores an order entered as it would look at checkout — the expected lead time with its typical error, the date the customer would be quoted (forecast plus the margin), the risk of missing the promise and the conditional low-review risk — and turns an uploaded CSV of up to 5,000 orders into a **triage list** with the top-K marked. **Replay a day** (Figure 27) ranks the held-out orders of any test day blind, by purchase or delivery day, then reveals what each customer went through; for 16 July 2018 at ten contacts, eight of the ten went on to rate 1–2 stars, four still waiting for a late parcel and four whose order never came, so a call at delivery could have reached none of them before they wrote and a message on the promised date all eight. **Model card** states what is deployed as read from the artifacts (Table 7), the protocol, the selected hyperparameters, the fold-to-fold variation, where the models should not be trusted [32], and a calculator for both pilot arms with the costs as inputs; every figure on it is labelled observed, assumed, derived or planned. The application refits nothing: every figure is read from the result tables or computed from the saved test predictions, and Appendix O gives a five-step guide to using it.

8.3 Robustness, reachability and reproducibility

Invalid input — a promised date before the purchase, a non-positive price, more sellers than items, a malformed, empty or oversized upload, a missing column, an unknown category or state, a missing artifact or a version mismatch — produces a specific message rather than a traceback, and the data loader stops the app if the row count of any source table changes. There are no API endpoints and no credentials: the batch CSV contract in `deployment/README.md` is the programmatic surface.

The Phase 1 dashboard went down with the suspended GitHub account it was deployed from. The application now deploys from the team's current repository; the README records what keeps it reachable (Community Cloud's one-click wake-up after twelve idle hours; Python 3.12 in the app's settings), and a fresh clone reproduces every table, figure and number here (Appendix A).

9 Business Recommendations and Impact

9.1 What the models rely on

Importance is measured by permutation on the held-out window rather than by impurity on the training set, which inflates high-cardinality features [46] (Figure 20). **Problem 2 has one dominant driver, and it is actionable.** How late the parcel was carries 0.206 of PR-AUC on its own, how long it took a further 0.074, the size of the order 0.045; the top two account for **76.9%** of total positive importance, which is why a single

depth-limited tree captured 96% of the achievable signal — good news for customers, because the platform already measures that driver and can act on it. **Problem 1A has no single point of failure**: no feature costs more than 0.034 days of MAE, because permutation understates correlated predictors such as the promise, the route histories and the distance.

9.2 Recommendations along the customer's order

1. At checkout: a date the customer can rely on (operations). Quote the forecast plus a margin, not the forecast: it estimates the typical delivery, and on its own it is exceeded by 31.9% of deliveries. The margin is the smallest, in tenths of a day, whose broken-promise rate on the validation months is no worse than the platform's own, with the date rounded up to a calendar day: eight days. On the held-out window the date is then missed for 2.5% of delivered orders against 7.4%, for an average quoted wait of 18.6 days against 17.5; a 6.9-day margin, set without outcomes to match the platform's average wait, misses 3.2%. The padding moves to where the risk is — Southeast customers, let down 8.6% of the time today, are let down 2.4% — though dates still missed are missed by more (6.8 days against 4.4; Appendix Q). With that date no blanket warning is needed; if the platform keeps its own, a warning at 11% risk would reach 45% of buyers, six in seven of them on time, so it belongs on the riskiest tenth, of whom one in four is late.

2. When the promised day ends without a delivery: hear from the platform first (customer care; no model). The calendar finds these customers without false alarms: 69.5 in 1,000 orders placed in the test window, 23 of them never delivered. That day each should get an apology, the parcel's status and a new date, or a refund where the order will not come. Today, as in Box 1, the survey reaches them first, typically two days after the promised date, and 77% of late-parcel customers who rated 1–2 stars did so before the parcel arrived. The survey should follow once the order is complete, as it does for parcels on time. Only the timing changes: every customer is still asked, and no review is filtered, delayed in publication or rewarded.

3. At delivery: help for customers whose parcel came but disappointed (customer experience). Across 2017–2018, of the customers whose parcel arrived and who rated it 1–2 stars, two in three had it on time and wrote a median 2.1 days after it came, about what was in it (Section 4.4). Each morning the team ranks the previous day's deliveries not yet rated and contacts the top of the list, with a replacement, refund or return to offer: at ten a day, 26 in 100 go on to rate 1–2 stars, against 8 at random. For orders from several sellers, check that every item arrived before the survey.

4. Over time: dates and lists that stay accurate (platform). Refit both models monthly, re-choose the margin at each refit, and track broken promises by region beside the gap between predicted and realised lead time, the signature of drift (Section 7.6); a week in which transit lengthens while the promise does not is the February–March 2018 pattern again (Appendix H).

9.3 From a probability to a decision: what is observed, what is assumed

Olist publishes no costs, so every reais figure below rests on *assumed* ones (`config.COSTS`): R$5 for a contact, and R$40 for a 1–2 star rating a contact turns around — a stand-in for the damage low ratings do to seller sales on another marketplace [11], not a measured cost. Nor can repeat purchases price it: 98% of first-time buyers order nothing within 180 days

of delivery (Appendix Q). What is *observed* is how many customers a contact reaches and the share who go on to rate 1–2 stars, its precision; the share a contact turns around is unknown until the pilot measures it. A contact pays when precision times that share exceeds $5/40 = 0.125$ [12], so Table 8 *derives* the share each contact must turn around.

Table 8: The decision layer. Reach (per 1,000 reviewed test orders) and precision are observed on the held-out window; the share a contact must turn around is derived from the two assumed costs as 0.125/precision; a value is R$40 for each customer turned around less R$5 for each contact, per 1,000 reviewed orders. Appendix P sources every figure on the slides.

Action	**reach** *observed*	**precision** *observed*	**must turn around** *derived*	**verdict**
Promised-date message (no model)	62.5	0.547	22.8% (1 in 4.4)	pilot
Delivery-day call, ten a day	42.5	0.260	48.1% (1 in 2.1)	pilot
Expedite on the late flag (R$12 : R$40)	—	≤ 0.27	needs precision 0.30	do not

If every contact turned its customer around, the promised-date message would be worth R$1,056 per 1,000 reviewed orders and the delivery-day call R$229; if one in four did, R$30 and −R$102 (Appendix Q varies the costs and the effect). Slide 11's R$775–1,466 values the review-risk list over all reviewed orders at a threshold, 0.11, read off the test window (Figure 18), with every contact assumed to work: R$1,466 counts every listed customer who went on to rate 1–2 stars, R$775 leaves out those who rated a late parcel before it arrived. On the slide's ten-a-day list, ranked by purchase day, the 35-in-100 share of the rest is worth R$433, and three fifths of the threshold list's gross value comes from customers whose order never came or who rated a late parcel before it arrived — the promised-date message's customers (Appendix P). **The late flag does not pay for an expensive action**: expediting at an assumed R$12 needs precision 0.300, a conservative bar (Appendix P), and the flag peaks at **0.274** — informative (ROC-AUC 0.782), but only for near-free actions: a better date, an early word.

9.4 A pilot for each contact: whether it helps the customer

Every reais figure above rests on an effect the data cannot show, because the tables record no contact other than the survey: whether a contact turns a customer's experience around. A randomised experiment measures it, as the Phase 2 evaluation asked [27], one arm per moment, each sized on the customers that moment can still reach (Table 25). *Arm B, the promised-date message:* every customer not yet rated when the promised day ends without a delivery is randomised to the message, with the survey once the order is complete, or to current practice; a third arm, message but survey unchanged, separates the two. About 13 a day rated the order only afterwards, 54.7% of them 1–2 stars: a one-in-four reduction at 80% power needs 209 per arm, five weeks for two arms or seven for three. *Arm A, the delivery-day call:* of each day's top ten delivered, not-yet-rated parcels, five are contacted. The control rate is 0.260, so the call pays only if it turns around about one in two; detecting that needs 159 per arm, about five weeks at ten listed a day. *Outcomes, customer first:* the final 1–2 star rate per randomised order (not the pre-arrival rate, which holding the survey removes by construction), the share who review, complaints and refund requests, and complaints about the contact; then reais per contact. Slide 11's design (191 per arm, six weeks) randomises the purchase-day list, on which a call can change at most 8.8 points, below the 12.5-point break-even (Appendix L, Table 26, Figure 24).

9.5 Impact: customer outcomes first, reais second

Table 9: Customer outcomes on the held-out window (`src/customer_view.py`). Each group names its population; no row assumes a cost or an effect of contact.

Customers	**count**	**rate**
At checkout — delivered orders (12,630)		
date broken: platform's date (mean 17.5 d) / forecast + 8 d (18.6 d)	936 / 318	7.4 / 2.5%
flagged at the 11% threshold: parcel then late / on time	773 / 4,874	1 : 6.3
When the promised day ends — orders placed (12,927); rate per 1,000		
no parcel when the promised day ended	898	69.5
. . . not yet rated then, and went on to rate 1–2 stars	440	34.0
At delivery — delivered orders not yet rated when the parcel came (12,254)		
ten a day: listed / went on to rate 1–2 stars (8% at random)	546 / 142	26%
All moments — customers who rated 1–2 stars (1,418)		
reachable when the promised day ended / only at delivery / neither	440 / 945 / 33	31 / 67 / 2%

Of the 1,418 customers who rated 1–2 stars, two in three could still be reached at delivery and almost all the rest when their promised day ended; the promised-date message reaches all 440 it can, the ten-a-day delivery-day list 142 of the 987 it could. The reais of Section 9.3 follow only if the pilot finds the effect. What it would take is modest: the three artifacts and a daily scoring job, which the application already runs on a held-out day; a calendar rule; a monthly refit; and the pilot.

10 Limitations and Conclusion

The models are fitted on a moving target, and the largest driver of lateness is not observable. Mean lead time falls 5.34 days between training and test, so a model fitted on the slower past is off-level on the faster present — the positive bias in every region of Section 7.6 — and needs refitting, with its calibration, on a rolling basis rather than once [16]. The late rate swings between 1.4% and 21.4% month to month with a Black Friday surge, a carrier-side slowdown under a tightened promise and a national strike (Section 4.2): shocks to carrier capacity or to the platform's own promising, none an attribute of an order. This, not the choice of algorithm, is the binding constraint on Problem 1B, which is why the ceiling was stated in advance.

The benefit of a contact is assumed, not measured. Section 9.3 values a contact at the low rating it turns around, and the tables record no contact with customers other than the survey, so whether reaching a customer earlier changes the rating — or whether an unsolicited contact lowers it — is unobserved. Part of it can be checked — whether the customer had already written — so each contact is counted only on the customers it can still reach (Appendix M). The counts of Table 9 stand on their own; the reais do not, and read as the value *if* the pilot of Section 9.4 finds the effect. Until it runs, the recommendation is to pilot, not to scale.

Risks to the customers the recommendations serve. A margin lengthens the average quoted wait by 1.1 days and the dates it still misses are missed by more, so how long to quote stays a commercial decision. A warning shown to almost half of all buyers would stop meaning anything, which is why it is confined to the riskiest tenth. Three

in four customers on the delivery-day list did not go on to rate the order 1–2 stars, so a contact must be an offer of help that is easy to decline. Asking for the review once the order is complete changes when a customer is asked, never whether, and the pilot measures its effect separately from the message's. And the forecast is least accurate where deliveries are longest (MAE 6.84 days in the North against 3.42 in the Southeast) while the margin is national, so broken promises should be tracked by region.

What the data does not contain, and what rests on assumption. The target of Problem 2 is a platform rating, with the self-selection Section 1 describes: a model of the rating is not a model of satisfaction, and the 1–2 star label cannot see the quietly dissatisfied customer who never reviews. The service-quality literature identifies product expectation gaps, seller communication and returns handling among the drivers of dissatisfaction [35, 39]; no order-level field records them, so the Problem 2 residual is unobserved cause rather than noise, which the comments, written with the rating, describe but cannot predict (Section 4.4). The outputs are triage scores, not entitlements: they must not penalise a seller, price a customer or drive an automated decision without review, and since the review-risk model largely restates lateness, a seller penalised by it is often penalised for a carrier's failure. Everything here is Brazil, 2017–2018, on one marketplace.

Conclusion

Two problems, at two moments of the customer's order, from one dataset and one feature pipeline, carried through to a deployed application. The regression reaches MAE 3.90 days, 60% below the platform's own promise; quoted with a margin chosen on validation, its date would have been broken for 2.5% of delivered orders instead of 7.4% (3.2% at the platform's own average wait); and thresholded against the promise it makes a better late-order flag (2.89× the floor) than any classifier trained on the label (1.24×), so the brief's dual framing is the better design. The low-review model reaches 3.75× its floor, and at its own moment its daily top ten reach 26 future 1–2 star reviewers in 100. The timing of the review decided more than the choice of model: most customers let down by a late or missing parcel had, like the customer of Box 1, rated the order before it came, so they can be reached on the promised date without a model, and the model's list serves the customers whose parcel arrived but disappointed.

Methodologically, cross-validation inside a drifting training period reorders the candidates, so selection belongs on several recent windows judged by rank, and weighting the loss for imbalance leaves ranking unchanged while inflating predicted risk threefold, so the threshold is the lever. The most useful results are negative or conditional: an informative flag that does not justify an expensive action, and contacts whose benefit to customers only a five-to-seven-week pilot can measure.

References

[1] George Baryannis, Samir Dani, and Grigoris Antoniou. Predicting supply chain risks using machine learning: The trade-off between performance and interpretability. *Future Generation Computer Systems*, 101:993–1004, 2019. doi: 10.1016/j.future.2019.07.059.

[2] Christoph Bergmeir and José M. Benítez. On the use of cross-validation for time series predictor evaluation. *Information Sciences*, 191:192–213, 2012. doi: 10.1016/j.ins.2011.12.028.

[3] James Bergstra and Yoshua Bengio. Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13(10):281–305, 2012.

[4] Bloomberg News. Brazil truckers strike for fourth day despite fuel price cut. Bloomberg, 24 May. `https://www.bloomberg.com/news/articles/2018-05-24/brazil-truckers-strike-for-fourth-day-despite-fuel-price-cut`, 2018. Accessed 9 October 2026.

[5] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001. doi: 10.1023/A:1010933404324.

[6] Rich Caruana and Alexandru Niculescu-Mizil. An empirical comparison of supervised learning algorithms. In *Proceedings of the 23rd International Conference on Machine Learning (ICML '06)*, pages 161–168. ACM, 2006. doi: 10.1145/1143844.1143865.

[7] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357, 2002. doi: 10.1613/jair.953.

[8] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016. doi: 10.1145/2939672.2939785.

[9] Ruomeng Cui, Zhikun Lu, Tianshu Sun, and Joseph M. Golden. Sooner or later? promising delivery speed in online retail. *Manufacturing & Service Operations Management*, 26(1):233–251, 2024. doi: 10.1287/msom.2021.0174.

[10] Chrysanthos Dellarocas. The digitization of word of mouth: Promise and challenges of online feedback mechanisms. *Management Science*, 49(10):1407–1424, 2003. doi: 10.1287/mnsc.49.10.1407.17308.

[11] Vinayak Deshpande and Pradeep K. Pendem. Logistics performance, ratings, and its impact on customer purchasing behavior and sales in E-Commerce platforms. *Manufacturing & Service Operations Management*, 25(3):827–845, 2023. doi: 10.1287/msom.2021.1045.

[12] Charles Elkan. The foundations of cost-sensitive learning. In *Proceedings of the Seventeenth International Joint Conference on Artificial Intelligence (IJCAI-01)*, pages 973–978, Seattle, Washington, 2001.

[13] Manuel Fernández-Delgado, Eva Cernadas, Senén Barro, and Dinani Amorim. Do we need hundreds of classifiers to solve real world classification problems? *Journal of Machine Learning Research*, 15:3133–3181, 2014.

[14] Maha Fouad, Sherif Barakat, and Amira Rezk. Effective E-commerce based on predicting the level of consumer satisfaction. In *Green Sustainability: Towards Innovative Digital Transformation*, Lecture Notes in Networks and Systems, pages 261–278. Springer Nature Singapore, 2023. doi: 10.1007/978-981-99-4764-5_17.

[15] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232, 2001. doi: 10.1214/aos/1013203451.

[16] João Gama, Indrė Žliobaitė, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. A survey on concept drift adaptation. *ACM Computing Surveys*, 46(4), 2014. doi: 10.1145/2523813.

[17] Akula Venkata Satya Sai Gopinadh, S. V. S. N. Sarma, and Gudipudi Radhesyam. Customer satisfaction prediction in online goods delivery through interpretable predictive models and sentiment analysis. In *Proceedings of the 7th International Conference on Communications and Cyber Physical Engineering*, Lecture Notes in Electrical Engineering, pages 1015–1025. Springer Nature Singapore, 2025. doi: 10.1007/978-981-95-0269-1_114.

[18] Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. Why do tree-based models still outperform deep learning on typical tabular data? In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022) Datasets and Benchmarks Track*, volume 35, pages 507–520, 2022.

[19] Jeff Heaton. An empirical analysis of feature engineering for predictive modeling. In *SoutheastCon 2016*, pages 1–6. IEEE, 2016. doi: 10.1109/SECON.2016.7506650.

[20] Nan Hu, Jie Zhang, and Paul A. Pavlou. Overcoming the J-shaped distribution of product reviews. *Communications of the ACM*, 52(10):144–147, 2009. doi: 10.1145/1562764.1562800.

[21] Nan Hu, Paul A. Pavlou, and Jie Zhang. On self-selection biases in online product reviews. *MIS Quarterly*, 41(2):449–471, 2017. doi: 10.25300/MISQ/2017/41.2.06.

[22] Rob J. Hyndman and Anne B. Koehler. Another look at measures of forecast accuracy. *International Journal of Forecasting*, 22(4):679–688, 2006. doi: 10.1016/j.ijforecast.2006.03.001.

[23] Shanthan Kandula, Srikumar Krishnamoorthy, and Debjit Roy. A prescriptive analytics framework for efficient E-commerce order delivery. *Decision Support Systems*, 147:113584, 2021. doi: 10.1016/j.dss.2021.113584.

[24] Sayash Kapoor and Arvind Narayanan. Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 4(9):100804, 2023. doi: 10.1016/j.patter.2023.100804.

[25] Shachar Kaufman, Saharon Rosset, Claudia Perlich, and Ori Stitelman. Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4), 2012. doi: 10.1145/2382577.2382579.

[26] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, pages 3146–3154, 2017.

[27] Ron Kohavi, Roger Longbotham, Dan Sommerfield, and Randal M. Henne. Controlled experiments on the web: Survey and practical guide. *Data Mining and Knowledge Discovery*, 18(1):140–181, 2009. doi: 10.1007/s10618-008-0114-1.

[28] Max Kuhn and Kjell Johnson. *Feature Engineering and Selection: A Practical Approach for Predictive Models*. Chapman and Hall/CRC, Boca Raton, 2019. doi: 10.1201/9781315108230.

[29] Stefan Lessmann, Bart Baesens, Hsin-Vonn Seow, and Lyn C. Thomas. Benchmarking state-of-the-art classification algorithms for credit scoring: An update of research. *European Journal of Operational Research*, 247(1):124–136, 2015. doi: 10.1016/j.ejor.2015.05.030.

[30] Sheng Liu, Long He, and Zuo-Jun Max Shen. On-time last-mile delivery: Order assignment with travel-time predictors. *Management Science*, 67(7):4095–4119, 2021. doi: 10.1287/mnsc.2020.3741.

[31] Daniele Micci-Barreca. A preprocessing scheme for high-cardinality categorical attributes in classification and prediction problems. *ACM SIGKDD Explorations Newsletter*, 3(1):27–32, 2001. doi: 10.1145/507533.507538.

[32] Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT* '19)*, pages 220–229, New York, NY, USA, 2019. ACM. doi: 10.1145/3287560.3287596.

[33] Alexandru Niculescu-Mizil and Rich Caruana. Predicting good probabilities with supervised learning. In *Proceedings of the 22nd International Conference on Machine Learning (ICML '05)*, pages 625–632. ACM, 2005. doi: 10.1145/1102351.1102430.

[34] Olist and Sionek, André. Brazilian E-Commerce public dataset by Olist. Kaggle, 2018. Distributed for IT5006 via Canvas; nine relational CSV tables covering 99,441 orders placed between 2016 and 2018.

[35] A. Parasuraman, Valarie A. Zeithaml, and Arvind Malhotra. E-S-QUAL: A multiple-item scale for assessing electronic service quality. *Journal of Service Research*, 7(3): 213–233, 2005. doi: 10.1177/1094670504271156.

[36] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[37] John C. Platt. Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In Alexander J. Smola, Peter L. Bartlett, Bernhard Schölkopf, and Dale Schuurmans, editors, *Advances in Large Margin Classifiers*, pages 61–74. MIT Press, Cambridge, Massachusetts, 1999.

[38] Foster Provost and Tom Fawcett. *Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking*. O'Reilly Media, Sebastopol, California, 2013.

[39] Shashank Rao, Thomas J. Goldsby, Stanley E. Griffis, and Deepak Iyengar. Electronic logistics service quality (e-LSQ): Its impact on the customer's purchase satisfaction and retention. *Journal of Business Logistics*, 32(2):167–179, 2011. doi: 10.1111/j.2158-1592.2011.01014.x.

[40] Takaya Saito and Marc Rehmsmeier. The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, 10(3):e0118432, 2015. doi: 10.1371/journal.pone.0118432.

[41] Nooshin Salari, Sheng Liu, and Zuo-Jun Max Shen. Real-time delivery time forecasting and promising in online retailing: When will your package arrive? *Manufacturing & Service Operations Management*, 24(3):1421–1436, 2022. doi: 10.1287/msom.2022.1081.

[42] scikit-learn developers. HistGradientBoostingClassifier — scikit-learn 1.5.2 documentation. `https://scikit-learn.org/1.5/modules/generated/sklearn.ensemble.HistGradientBoostingClassifier.html`, 2024. Histogram-based gradient boosting "inspired by LightGBM", with native support for missing values. Accessed 9 October 2026.

[43] D. Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-François Crespo, and Dan Dennison. Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems 28 (NIPS 2015)*, pages 2503–2511, 2015.

[44] Ravid Shwartz-Ziv and Amitai Armon. Tabular data: Deep learning is not all you need. *Information Fusion*, 81:84–90, 2022. doi: 10.1016/j.inffus.2021.11.011.

[45] Victor Hugo Silva. Correios suspendem entregas do Sedex por conta de paralisação de caminhoneiros. Tecnoblog, 23 May. `https://tecnoblog.net/noticias/correios-suspendem-entregas-sedex/`, 2018. In Portuguese. Correios suspended its time-definite Sedex services and added five business days to standard Sedex and PAC lead times during the strike. Accessed 9 October 2026.

[46] Carolin Strobl, Anne-Laure Boulesteix, Achim Zeileis, and Torsten Hothorn. Bias in random forest variable importance measures: Illustrations, sources and a solution. *BMC Bioinformatics*, 8:25, 2007. doi: 10.1186/1471-2105-8-25.

[47] Ruben van den Goorbergh, Maarten van Smeden, Dirk Timmerman, and Ben Van Calster. The harm of class imbalance corrections for risk prediction models: Illustration and simulation using logistic regression. *Journal of the American Medical Informatics Association*, 29(9):1525–1534, 2022. doi: 10.1093/jamia/ocac093.

[48] Wouter Verbeke, Karel Dejaeger, David Martens, Joon Hur, and Bart Baesens. New insights into churn prediction in the telecommunication sector: A profit driven data mining approach. *European Journal of Operational Research*, 218(1):211–229, 2012. doi: 10.1016/j.ejor.2011.09.031.

[49] P. Wangkiat and C. Polprasert. Machine learning approach to predict E-commerce customer satisfaction score. In *2023 8th International Conference on Business and Industrial Research (ICBIR)*, pages 1176–1181. IEEE, 2023. doi: 10.1109/ICBIR57571.2023.10147542.

[50] David H. Wolpert. Stacked generalization. *Neural Networks*, 5(2):241–259, 1992. doi: 10.1016/S0893-6080(05)80023-1.

A Reproducing this report

Table 10: Everything in this document is generated. The order below is the order to run it in. The fitting step dominates; the rest takes a few minutes.

Command	What it produces
`pip install -r requirements-dev.txt`	The full analysis environment. `requirements.txt` alone is the deployment runtime
`python tests/test_leakage.py`	89 validation checks: join grain, the history construction against a brute-force recomputation, the leakage guard in both directions, split disjointness, Phase 1 backward compatibility, the capacity and lateness tables against recomputation, and — added in this phase — that the pickled models, the metadata, the hyperparameter table, the README and this report name the same deployed estimators, and that the review-timing value recomputes from the per-order predictions; since the presentation, that the customer-view tables and the slide sources agree with each other and with the earlier tables, and that the slides are byte-identical to the deck locked on 4 November
`python src/train_models.py`	Fits every model, scores the test window once, writes `models/` and every result table into `report/results/`
`python src/capacity_metrics.py`	Precision@K and Recall@K from the saved per-order test predictions (Section 7.4); nothing is refitted
`python src/lateness_anatomy.py`	The weekly, monthly and strike-cohort lateness tables of Appendix H
`python src/fit_diagnostics.py`	The three deployed models scored on the training, validation and test periods (Appendix J)
`python src/selection_tables.py`	Every candidate's score on every out-of-sample window with its fold-to-fold spread (Appendix G), and the selected hyperparameters of every searched model and of the deployed ensembles' members, read from the pickles (Appendix D)
`python src/pilot_design.py`	The sizing of the pilot as designed on slide 11 (Appendix L)
`python src/review_timing.py`	When each review was written against the delivery, the share of the list not yet written, the value restated on that share, and the per-order flags the application's replay shows (Appendix M)
`python src/customer_view.py`	The customer's view (Appendix Q, Section 9): the promised date under each rule, what happened to every 1,000 orders, complaint topics, baskets, repeat purchase, every contact list with its value, the per-contact pilot sizes (Table 25), the order of Box 1 and the per-order journeys the application shows. The deployed lead-time model is re-scored, not refitted, and its scores are asserted equal to the saved predictions
`python src/slide_provenance.py`	Every number on the presentation slides with its kind and source; the 88 that can be checked are recomputed from these tables (Appendix P), and it stops if any no longer matches its slide

Table 10, continued

Command	What it produces
`python src/figures_phase2.py`	Draws every modelling figure as vector PDF into `report/figures/` from those tables; the exploratory figures are drawn by notebooks 01–04
`python src/phase3_stats.py`	Distils every number quoted in this report into `report/stats_phase3.txt` (it runs `phase2_stats.py` first, so the Phase 2 contract is a subset). Diff it against the manuscript
`python src/report_stats.py`	Reproduces every exploratory number of Section 4 into `report/stats.txt`, unchanged since Phase 1 — the check that the later feature additions are backward compatible
`streamlit run deployment/app.py`	The application of Section 8; `dashboard/app.py` is the Community Cloud entry point and runs the same file

Table 10 is the run order. `SEED = 42` is set in every estimator, every randomised search and every subsample. Both requirements files pin exact versions that install on every Python up to 3.12; the reported numbers were produced on Python 3.10.9 with those pins, and the deployed application runs on Python 3.12. Model artifacts record the `scikit-learn` version that wrote them and `modeling.load_model()` checks it on the way back in. The ten notebooks (01–04 exploratory, 05–09 modelling, 10 the additions of this phase) are committed with their outputs.

B Data model, cleaning decisions and dictionary

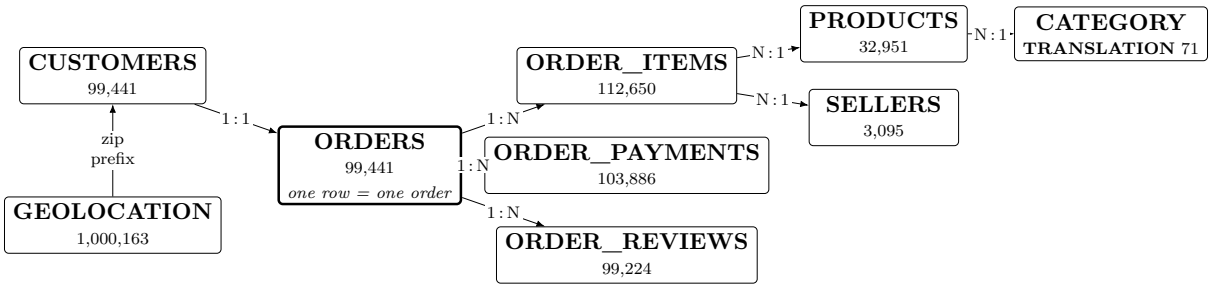


Figure 1: The nine source tables. `ORDERS` is the analysis grain. The three one-to-many children (items, payments and reviews) are each aggregated to one row per order *before* joining; merging them directly inflates the table by 19% and biases every average.

Table 11: Data quality audit and cleaning decisions, implemented as `CLEANING_DECISIONS` in `src/data_load.py`.

Issue	Evidence	Decision
One-to-many joins	112,650 items and 103,886 payments against 99,441 orders; a naive merge yields 118,434 rows	Aggregate each child table to one row per order, then join; assert the grain
Customer key	99,441 `customer_id` values but 96,096 `customer_unique_id`; 2,997 people ordered more than once	Carry `customer_unique_id` and group on it for repeat behaviour and for splitting
Undelivered orders	2.98% of orders have no delivery timestamp; status is *delivered* for 97.0%	Delivery targets computed only for delivered orders; the rest are `NaN`, never imputed
Review keys	`review_id` is not unique (789 ids reused); 547 orders carry 2–3 reviews and 768 carry none	Key on `order_id`, keep the earliest review; no review means `NaN`
Review text	`review_comment_title` present for 11.7% of reviews, `review_comment_message` for 41.3%; 75.4% of the analysis window's 1–2 star reviews carry one or the other	Not a feature: a comment is written with the rating, after the prediction moment. Read descriptively (Appendix Q)
Geolocation grain	1,000,163 rows for 19,015 zip prefixes; 42 points lie outside Brazil and are dropped (19,010 prefixes kept)	Collapse to a median coordinate per prefix; use it for great-circle distance
Missing product attributes	610 products (1.9%) have no category	Label `unknown` rather than dropping orders; impute numerics in-pipeline
Coverage ramp	329 orders before 2017; final weeks truncated mid-cycle	Restrict to orders placed 1 January 2017 to 30 August 2018 (99,091)
Portuguese categories	71 category names	Translate to English for all output; keep the original key

Table 12: Candidate problems against the brief's scoping checklist (Section 4.5), as scored in Phase 1 with untuned probes.

Criterion	**C1 delivery (checkout)**	**C2 low review (delivery)**	**C3 repeat (checkout)**
Derivable target	Yes, from order timestamps	Yes, `review_score` ≤ 2	Yes, next order by person
No leakage	Yes, checkout regime	Yes, at-delivery regime	Yes, checkout regime
Sufficient signal	Regression yes (-41% MAE); classification weak ($1.6\times$)	Yes, PR-AUC 0.40 vs 0.11 ($3.6\times$)	No, $2.0\times$ at a 1.7% base rate
Manageable imbalance	8.1% late; PR-AUC and thresholds, no resampling	14.6% low; same handling	1.8%, too extreme
Clear stakeholder	Operations	Customer experience	CRM
Verdict	**Problem 1**	**Problem 2**	Rejected

Table 13: Data dictionary for the nine source tables. *Null* is measured, not declared: Y means the column actually contains missing values in this extract, with the observed rate given.

Table	Column	Type	Null	Comment
orders	order_id	text	N	Primary key. The analysis grain
	customer_id	text	N	**Per-order** key, not a person
	order_status	text	N	8 values; *delivered* for 97.0%
	order_purchase_timestamp	datetime	N	The checkout prediction moment
	order_approved_at	datetime	Y 0.16%	Post-checkout; excluded as a feature
	order_delivered_carrier_date	datetime	Y 1.79%	Post-outcome; splits lead time into seller and carrier share
	order_delivered_customer_date	datetime	Y 2.98%	Post-outcome; defines both delivery targets
	order_estimated_delivery_date	date	N	Promised at checkout; **admissible**. Date only, 459 distinct values
order_items	order_id	text	N	**1:N** — 112,650 rows for 98,666 orders
	order_item_id	int	N	Sequence within the order, 1–21
	product_id, seller_id	text	N	Foreign keys
	shipping_limit_date	datetime	N	Post-checkout; excluded
	price, freight_value	float	N	Per item; summed to order level
order_payments	order_id	text	N	**1:N**. One order has no payment row
	payment_sequential	int	N	1–29
	payment_type	text	N	5 values; credit card 75.4%
	payment_installments	int	N	0–24; > 1 for 51.4% of orders
	payment_value	float	N	Summed to order level
order_reviews	review_id	text	N	**Not unique** — 789 ids reused across orders
	order_id	text	N	**1:N**; 547 orders carry 2–3 reviews
	review_score	int	N	1–5; target of Problem 2
	review_comment_title	text	Y 88.3%	Written with the rating; not a feature
	review_comment_message	text	Y 58.7%	Written with the rating; not a feature. Read descriptively (Appendix Q)
	review_creation_date	date	N	Post-outcome for the review target
	review_answer_timestamp	datetime	N	Post-outcome
customers	customer_id	text	N	Joins 1:1 to orders
	customer_unique_id	text	N	**The actual person**; 96,096 of them. Grouping key for splits
	customer_zip_code_prefix	int	N	Joins to geolocation
	customer_city, customer_state	text	N	4,119 cities, 27 states
sellers	seller_id	text	N	3,095 sellers
	seller_zip_code_prefix	int	N	Joins to geolocation
	seller_city, seller_state	text	N	59.7% of sellers are in SP
products	product_id	text	N	32,951 products
	product_category_name	text	Y 1.85%	Portuguese; 73 values. Nulls labelled *unknown*
	product_weight_g	float	Y 0.01%	Summed to order level
	product_length/height/ width_cm	float	Y 0.01%	Combined into item volume
	product_photos_qty	float	Y 1.85%	Listing-quality proxy
geolocation	geolocation_zip_code_prefix	int	N	19,015 prefixes over 1,000,163 rows
	geolocation_lat, _lng	float	N	Collapsed to a median per prefix; out-of-Brazil points dropped
category_ translation	product_category_name	text	N	Join key
	product_category_name_english	text	N	71 translations
Derived in src/ — not present in any source file:				
derived	delivery_days	float	Y	Purchase to arrival; target of 1A
derived	is_late	bool	Y	Arrival after the promised date; target of 1B
derived	distance_km	float	Y 1.2%	Great-circle customer–seller distance
derived	estimated_days	float	N	Promised window in days
derived	lead_category	text	N	Category of the most expensive item
derived	freight_ratio	float	Y	Freight as a share of item value
derived	hist_*	float	Y	The eight as-of track records of Section 5.2

C Success criteria registered in advance

Written into `config.EXPECTED_BANDS` before any model was tuned. `evaluate.check_bands()` compares the headline results against them at the end of every training run and writes its verdicts to `report/results/bands.csv`. The alarm is direction-aware: for an error metric the suspicious end of the interval is the low one, for a score the high one, and confusing the two would point the check at the shortfalls and wave the leaks through.

Problem	Metric	Registered band	Where the range came from	Result
1A	MAE (days)	3.3 – 4.3	Phase 1 reported 3.89 untuned	3.898
1B	balanced accuracy	0.58 – 0.72	Phase 1 reported 0.58 for a direct classifier	0.705
1B	ROC-AUC	0.72 – 0.85	Untuned scoping probe reached 0.79 for the derived route	0.782
2	balanced accuracy	0.63 – 0.74	Phase 1 reported 0.69	0.681
2	ROC-AUC	0.70 – 0.79	Phase 1 reported 0.73	0.749
2	PR-AUC	0.34 – 0.48	Phase 1 reported 0.40 against a 0.11 floor	0.414

D Hyperparameter search spaces and selected values

Family	Rung	Classification	Regression
A — Linear	0	`LogisticRegression`	`LinearRegression`
	1	`LogisticRegression` (ℓ_2, tuned C)	`Ridge` (tuned α)
B — Tree-based	0	`DecisionTreeClassifier`	`DecisionTreeRegressor`
	1	`RandomForestClassifier`	`RandomForestRegressor`
	2	`HistGradientBoostingClassifier`	`HistGradientBoostingRegressor`
C — Ensemble	0	`VotingClassifier` (soft, A+B)	`VotingRegressor` (A+B)
	1	`StackingClassifier` (A+B)	`StackingRegressor` (A+B)

Every tuned model was searched with `RandomizedSearchCV` over the distributions of Table 14, scored by five-fold expanding-window cross-validation inside the training period: thirty draws per model, except the random forests, which get six because a forest costs one to two orders of magnitude more per fit and Phase 1 had already shown the forest and the boosted trees landing within 0.01 days of each other. Table 15 lists the values that won, which the Phase 2 report referred to but did not print; the deployed ensembles carry exactly their winning members' values, read back from the pickles, and the validation suite asserts it.

A compromise inside stacking, stated plainly. `StackingClassifier` and `StackingRegressor` build their meta-features with `cross_val_predict`, which requires the inner folds to *partition* the training set: every row must be predicted exactly once. An expanding window partitions nothing — the earliest rows are never in a test fold — so the closest valid approximation is unshuffled `KFold`, which at least keeps each fold contiguous in time. The meta-learner is consequently fitted on out-of-fold predictions partly made by models trained on later orders. The residual optimism is confined to the meta-learner's coefficients; the base models are refitted on the whole training period afterwards, and the

Table 14: Search spaces.

Model	Draws	Distributions
`LinearRegression` / `LogisticRegression`	—	No free parameters. Fitted and scored under the same protocol so the row is comparable
`Ridge`	30	$\alpha \sim \text{loguniform}(10^{-3}, 10^{3})$
`LogisticRegression` (ℓ_2)	30	$C \sim \text{loguniform}(10^{-3}, 10^{2})$
`DecisionTree`	30	`max_depth` $\sim$ randint(3, 20) [classifier: (3, 16)]; `min_samples_leaf` $\sim$ randint(5, 200) [classifier: (20, 400)]; `max_features` $\sim U(0.4, 1.0)$
`RandomForest`	6	150 trees; `max_depth` $\sim$ randint(8, 18) [classifier: (6, 16)]; `min_samples_leaf` $\sim$ randint(20, 140) [classifier: (30, 180)]; `max_features` $\sim U(0.15, 0.40)$
`HistGradientBoosting`	30	`max_iter` $\sim$ randint(120, 500); `learning_rate` $\sim$ loguniform$(0.02, 0.3)$; `max_leaf_nodes` $\sim$ randint(15, 64); `min_samples_leaf` $\sim$ randint(10, 120); `l2_regularization` $\sim$ loguniform$(10^{-4}, 10)$
`Voting`	—	Weights fixed at 1:2 in the tree's favour. A stated prior, not a tuned parameter: under drift the linear member is materially weaker and an equal blend would be worse than its better half
`Stacking`	—	Meta-learner `Ridge`($\alpha = 1$) / `LogisticRegression`($C = 1$); inner CV five contiguous folds — see the note below

Table 15: Selected hyperparameter values, from `report/results/hyperparameters.csv`. Members of the deployed ensembles are the rows marked †; the stacked review model's meta-learner has coefficients 0.405 (linear member) and 5.572 (tree member).

Problem	Model	Selected values
1A	LinearRegression †	no free parameters
1A	Ridge	$\alpha = 659.9$
1A	DecisionTree	depth 17, min leaf 158, max features 0.897
1A	RandomForest	150 trees, depth 14, min leaf 107, max features 0.165
1A	HistGradientBoosting †	433 iterations, learning rate 0.1065, 36 leaf nodes, min leaf 117, ℓ_2 9.142
1A	VotingRegressor †	weights 1:2 (linear : tree)
1B	LogisticRegression (ℓ_2)	$C = 0.004983$
1B	DecisionTree	depth 9, min leaf 290, max features 0.878
1B	RandomForest	150 trees, depth 13, min leaf 132, max features 0.300
1B	HistGradientBoosting	426 iterations, learning rate 0.07079, 21 leaf nodes, min leaf 30, ℓ_2 8.244
1B	derived route †	two-parameter logistic on the margin, fitted on validation; threshold 0.11
2	LogisticRegression (ℓ_2) †	$C = 8.431$
2	DecisionTree	depth 14, min leaf 211, max features 0.963
2	RandomForest	150 trees, depth 12, min leaf 44, max features 0.349
2	HistGradientBoosting †	208 iterations, learning rate 0.02073, 28 leaf nodes, min leaf 18, ℓ_2 0.01789
2	StackingClassifier †	meta-learner LogisticRegression($C = 1$); threshold 0.19

outer protocol scoring the finished ensemble is still the expanding window. It matters less than it otherwise would because of how the result lands: the ensemble is *flattered* by this arrangement and still does not separate from the boosted trees, which makes the conclusion in Section 7.5 safer rather than weaker.

E Re-checking a Phase 1 claim about least squares

Phase 1 reported that ordinary least squares was unusable on this design matrix — "rank-deficient, so its coefficients explode". Phase 2 re-ran it four ways. The observation reproduces exactly; the conclusion drawn from it needed correcting.

Table 16: Reproducible via `report/results/encoding_diagnostic.csv`. The deficiency is exactly four — one per categorical, because each one-hot block sums to one and is therefore collinear with the intercept.

Encoding	rank / cols	deficiency	max $\lvert\beta\rvert$	CV MAE	test MAE
OLS, full one-hot block	101 / 105	4	2.7×10^{13}	3.9×10^{11}	4.181
OLS, one level dropped	101 / 101	0	14.7	5.242	4.181
Ridge, full one-hot block	101 / 105	4	9.82	5.241	4.182
Ridge, one level dropped	101 / 101	0	12.5	5.240	4.183

Least squares still returns a minimum-norm solution and its *predictions* are fine — 4.181 days on the test window, indistinguishable from Ridge. What is not fine is the coefficient vector at 10^{13}, and the cancellation holding it together is fragile enough to fail on one of five expanding-window folds, which is why the cross-validated error is astronomical while the single-split test error looks healthy. Dropping one level per categorical restores full rank, so the linear family is encoded that way; the compromise is that a level unseen in training is scored as the reference level, which affects a handful of test orders with a payment type absent from training. Once the design is full rank, Ridge and OLS agree to three decimals: regularisation buys numerical stability here, not accuracy.

F Full result tables

The body reports the models that matter. The complete tables — every model, every metric, cross-validated, validation and test scores side by side — are written to `report/results/`:

File	Contents
`reg_p1a.csv`	Problem 1A: baselines, both ladders, ensembles; MAE, RMSE, R^2
`clf_p1b.csv`	Problem 1B: five direct classifiers and two derived routes
`clf_p2.csv`	Problem 2: both ladders and both ensembles, all metrics
`selection_*.csv`, `cv_folds.csv`	Every candidate on every out-of-sample window, its ranks, and the fold-to-fold spread (Appendix G)
`hyperparameters.csv`	Selected values of every searched model and of the deployed ensembles' members (Appendix D)
`encoding_diagnostic.csv`	The Phase 1 re-check of Table 16
`cv_protocol.csv`	Temporal against shuffled CV against the forward test
`weight_ablation.csv`	Class weighting: discrimination and calibration
`calibration_p2.csv`, `threshold_sweep_p2.csv`	Reliability and the operating-point sweep
`cost_p2.csv`, `cost_p1b.csv`	Expected net benefit by threshold
`perm_p1a.csv`, `perm_p2.csv`	Permutation importance on the test window
`segments_*.csv`	Performance by region, category and test month
`predictions_*.csv`	Per-order test predictions, from which the figures and the application's replay are drawn
`bands.csv`	Headline metrics against the bands registered in advance
`capacity_*.csv`	Precision@K and Recall@K, pooled and per day, for both flags
`lateness_*.csv`	The lateness anatomy of Appendix H
`train_val_test.csv`	The deployed models on the training, validation and test periods (Appendix J)
`pilot_design.csv`, `pilot_breakeven.csv`	The pilot as designed on slide 11 (Appendix L)
`review_timing*.csv`, `story_order.csv`	Review timing against delivery, the share of the list not yet written, the restated value, the on-time check and the example order (Appendix M)
`customer_*.csv`	The customer's view (Appendix Q): the promised date under each rule, by region and over a grid of margins; outcomes per 1,000 orders; the 1–2 star rate by days late; complaint topics, their patterns and precision, and the labelled sample behind it (`customer_topic_labels.csv`, a committed input rather than an output); baskets; repeat purchase; every contact list, its value, the value's decomposition and its sensitivity to the costs; the per-contact pilot sizes; the order of Box 1 and its timeline; the per-order journeys the application shows
`slide_provenance.csv`	Every number on the slides: kind, source or formula, recomputed value, whether it matches, and the reading this report stands by (Appendix P)

G The cross-validation protocol, and every candidate on every window

Table 17: One model, three protocols. Raw average precision disagrees; the same numbers divided by each period's no-skill floor do not. Figure 19.

Protocol	PR-AUC	fold spread	no-skill floor	lift
`TimeSeriesSplit`, expanding window	0.562	±0.091	0.158	3.55×
`StratifiedGroupKFold`, shuffled	0.580	±0.006	0.158	3.66×
Forward test window — the truth	0.414	—	0.110	3.75×

Both cross-validated estimates in Table 17 sit about 0.16 above the forward result, which looks like catastrophic optimism — and almost all of it is mechanical. Average

precision's no-skill baseline *is* the positive rate [40], and the low-review rate falls from 15.8% inside the training period to 11.0% in the test window; normalised by that floor the three agree to within 0.2 of a multiple. **Raw PR-AUC is not comparable across periods with different base rates; its lift over the floor is** — and quoting the raw number alone would have supported either a claim of severe overfitting or a claim of none. The shuffled protocol adds a second warning: its fold-to-fold spread is ±0.006 against the temporal protocol's ±0.091, an interval fifteen times narrower on an estimate no closer to the truth.

Table 18: Every Problem 2 candidate on every out-of-sample window (PR-AUC), from `report/results/cv_folds.csv`; the standard deviation is over the five expanding folds. "Rank" is the mean rank over the three most recent windows (folds 4, 5 and validation), the selection criterion of Section 6.2. Figure 16. The same table for Problems 1A and 1B is in the CSV.

Model	fold 1	fold 2	fold 3	fold 4	fold 5	val.	mean ± sd	test	rank
StackingClassifier (A+B) *deployed*	0.413	0.537	0.617	0.660	0.614	0.409	0.568 ± 0.087	0.414	1.67
HistGradientBoosting	0.411	0.539	0.618	0.660	0.613	0.407	0.568 ± 0.088	0.415	2.00
RandomForest	0.404	0.544	0.622	0.654	0.616	0.403	0.568 ± 0.089	0.405	2.67
VotingClassifier (A+B)	0.413	0.526	0.611	0.650	0.604	0.404	0.561 ± 0.084	0.407	3.67
DecisionTree	0.380	0.510	0.595	0.640	0.592	0.389	0.543 ± 0.092	0.396	5.00
LogisticRegression (ℓ_2)	0.306	0.416	0.534	0.574	0.497	0.320	0.466 ± 0.095	0.291	6.33
LogisticRegression	0.307	0.416	0.534	0.574	0.497	0.320	0.465 ± 0.095	0.291	6.67

Table 18 is what the Phase 2 evaluation asked for. Read down a column, the windows differ far more than the models: every candidate is at its worst on the first fold (early 2017, few orders, a 20% low-review rate) and at its best on the fourth. Read across a row, the forest, the boosted model and the stack are within 0.0004 of one another on the cross-validated mean against a fold-to-fold standard deviation of 0.09; the stacked ensemble is ahead of the single boosted model on three of the six windows and behind on three. Cross-validation cannot order them, which is why the rank rule was fixed before any test score was seen and why the deployed choice is reported as a tie resolved by that rule rather than as a win.

H Lateness anatomy: where the late deliveries came from

Phase 1 read the monthly late-rate series and attributed its spikes to Black Friday and to the May 2018 truckers' strike. The Phase 1 feedback pointed out that the figure does not support the second claim — the largest peaks are February and March 2018, and May is unremarkable — and asked for a source and a more careful link. `src/lateness_anatomy.py` is that analysis, at three resolutions, and Figure 23 draws it.

Weekly, not monthly. A ten-day event cannot be resolved by a monthly series. In Monday-to-Sunday purchase weeks the strike of 21 May to 1 June 2018 [4] — nationwide road blockades, during which Correios suspended its time-definite Sedex services and added five business days to standard parcels [45] — is plainly visible, but one week *earlier* than a naive reading expects: orders placed on 14–20 May were promised 19.5 days under normal conditions and were in transit when the roads closed, and 16.0% of them arrived late against a 5.0% average over the preceding five weeks. Of all orders in transit during the strike (4,228), 16.9% were late, against 0.4% for the May orders delivered before it (Table 19). What the monthly series records instead is the platform's reaction: order

volume in the strike week halved, the promised window for orders placed that week rose to 25.3 days and in the following week to 36.8, and the late rate of those orders was 5.5% and 0.4%. That is why June 2018, at 1.4%, is the quietest month in the data.

February–March 2018 is a different mechanism. Panel (b) of Figure 23 splits lead time at the carrier handoff. Outside the peak, sellers hand parcels to the carrier in about 3.2 days and the carrier takes 8.7 more; in February–March 2018 the seller's share was unchanged at 3.4 days while the carrier's rose to 13.2, in every customer region. At the same time the promised window was *tightened*, from 26.3 days in January to 22.0 in March — within the São Paulo to São Paulo route alone, from 20.1 to 15.2 days — so the slack that normally absorbs a slow fortnight was removed exactly when it was needed. The late rate peaked at 29.0% in the week of 26 February, the promise was lengthened again from the week of 9 April, and by that week the late rate was back under 5%. The data do not identify the external cause of the carrier slowdown and we do not assert one; what they do establish is that it was carrier-side, network-wide and compounded by a promising decision, which is the careful form of the claim Phase 1 made loosely.

Table 19: The cohorts around the strike, from `report/results/lateness_strike.csv`. Carrier days run from the carrier handoff to the doorstep; the promise is the window quoted at checkout.

Cohort	orders	late	carrier days	promise, days
Bought 1–20 May, delivered before the strike	2,749	0.4%	4.8	19.5
In transit during the strike	4,228	16.9%	13.1	25.5
Bought during the strike, 21 May–1 June	1,502	3.5%	9.2	29.4
Bought 2–30 June	5,912	1.4%	6.6	27.5

I Responses to the Phase 1, Phase 2 and presentation feedback

Three rounds of feedback were treated as specifications. Tables 20 to 22 list each point and the section, table or figure of this report that acts on it.

Table 20: Phase 1 feedback (on the literature review, the exploratory analysis and the dashboard) and where this report acts on it.

Feedback	Response
Survey-based perceived quality and platform ratings were treated as one construct	Sections 2 and 1 separate them, cite the self-selection literature [10, 20, 21], and define the target as the platform rating; Section 10 carries the consequence
Model choices cited original algorithm papers rather than e-commerce applications; no comparison within the tree family; HistGradientBoosting not justified	Section 2 argues the rungs from the empirical comparisons and from e-commerce applications, including three studies on this dataset, and states why HistGradientBoosting; the ladder results of Section 7 are the within-family comparison
Features not supported by literature	Section 5 cites the support for each feature group and uses the correlation matrix as the empirical screen; Appendix K gives a rationale per feature
Problem 2 should be evaluated under limited capacity, with Precision@K and Recall@K	Section 7.4, Table 6 and Figure 22; the operating point in Section 9.3 is a capacity; the application returns a top-K triage list and replays any test day
Figure 3 attributed the late-rate peaks to the May 2018 strike; the strike needed a source	Section 4.2, Appendix H and Figure 23: the peaks are February–March 2018; the strike is cited [4, 45], shown at weekly resolution, and its effect traced through the promised window
Figure 5a omitted São Paulo; "roughly triples"; Figure 8 not referenced	Figure 5a redrawn (the notebook had taken the first eight states alphabetically); growth is about eight-fold (Section 4.1); the correlation matrix is discussed in Sections 4 and 5
Candidate 1 named a stakeholder without saying how the team would use the prediction	Table 2 and Section 9.2: at each moment of the customer's order, what the platform does, which team does it and the evidence behind it
Dashboard inaccessible	Redeployed from the team's current repository under a new URL; Section 8 explains how it stays reachable

Table 21: Phase 2 feedback (on the modelling report) and where this report acts on it.

Feedback	Response
Report CV scores with their fold-to-fold variation for the Problem 2 candidates	Table 5 now carries the mean ± standard deviation over the five folds for every candidate (and Table 3 for the regression); Table 18 and Figure 16 show every candidate on every window; Section 7.3 interprets the spread; the model card shows the same table
State the deployed Problem 2 model clearly and consistently; justify the choice	Section 7.5 and Table 7: the StackingClassifier, as pickled, with the justification; the README, the model card and this text are tied to the artifact by a validation check
Refer to Figure 3 in the text; the selected hyperparameter values were said to be in Appendix C but were not	Every figure in Appendix O is referred to from the body; the selected values are in Table 15, generated from the results tables and the deployed pickles
The expected benefit assumes that contacting a customer changes the rating; state it as a limitation and propose a pilot or A/B test	Section 10 states it; Section 9.3 gives the share of customers a contact must turn around to pay; Section 9.4 and Table 25 size a randomised pilot for each contact on the customers it can still reach, and Appendix L compares the design on slide 11; the model card carries a calculator for it. Appendix M adds the part of the assumption the data *can* check — whether the customer has already written — and restates the value on the share not yet written

Table 22: Feedback on the presentation of 5 November 2026 and where this revised report acts on it. The slides are submitted unchanged, as the brief requires; Appendix P gives the source of each of their numbers and corrects their wording where it was wrong.

Feedback	Response
The presentation was framed around the company — its teams, costs and reais — more than around the customer	The report now starts from the customer and keeps Olist's teams as the ones who act. Section 1 opens with one customer's order (Box 1); Table 2 gives, at each moment of the order, what the customer needs, what the platform does and who does it; Section 4.4 reports what unhappy customers wrote; Section 9.2 states each recommendation by what changes for the customer, and Table 9 counts the customers each step reaches before any reais; Section 10 states the risks to customers; and Appendix Q gives the tables behind them, including whether let-down customers came back. Money is kept, second, and its costs are labelled as assumptions (Section 9.3)
One case, a single customer's order followed from checkout to review, was singled out as a strength	The order now opens the report as Box 1 in Section 1, with every date read from the tables (`customer_story.csv`) and when each recommendation would have reached that customer; the paragraph after the box and Appendix M show from the test window that the case is typical rather than chosen for effect. The application now opens on the same order (its Customer journey tab, Figure 25)

Table 22, continued

Feedback	Response
(Teaching assistant) Where do the numbers on slide 11 come from?	They are of four kinds the slide did not separate: observed on the held-out window (57.2%, the 35 in 100); assumed, with no data source (R$5 a contact, R$40 a 1–2 star rating: Olist publishes no costs); derived by arithmetic on the two (R$433 to R$1,466, 44.7%, about 1 in 5, +R$0.72; R$1,466 is $1{,}000 \times (712 \times 40 - 1{,}927 \times 5)/12{,}855$, R$40 for each of the 712 listed customers who went on to rate 1–2 stars less R$5 for each of the 1,927 contacts); and planned, for a pilot not yet run (the one-in-four effect, 42.9%, 80% power, 191 per arm, six weeks). Appendix P sources every slide number (`src/slide_provenance.py` recomputes 88 of them): the values are per 1,000 *reviewed* orders; R$775 belongs to the threshold-0.11 list, not to the 35-in-100 share, which is worth R$433; 57.2% is a historical rate, not a control group; and the slide's statement about calls becomes "the tables record no contact other than the survey". Section 9.3 separates what is observed from what is assumed (Table 8), Section 9.4 re-sizes the pilot on the customers each contact can still reach (Table 25), and Appendix Q varies the costs and the effect
The deck described its own dates, test and findings in moral terms, implying that the platform's promised dates could not be trusted	The report drops that vocabulary. It describes the platform's date neutrally, as a commercial decision padded against the cost of a broken promise (Section 7.1; Cui et al. [9]), and replaces the slides' bare forecast with the forecast plus a margin chosen on validation, since the forecast on its own is exceeded by 31.9% of deliveries (Section 9.2). Appendix P maps each phrase of the deck to the measured quantity behind it, and restates the rule that every customer is still asked for a review and no review is filtered, delayed in publication or rewarded

J Training, validation and test side by side

The results tables carry cross-validated, validation and test columns. The in-sample fit on the training period is the missing fourth view, and it is what separates over-fitting from drift: a model that has memorised its training period collapses between training and validation, while a model facing a changed world drops between validation and test, or in step with the base rate. `src/fit_diagnostics.py` scores the three deployed artifacts on all three periods without refitting anything.

Three readings. **The lead-time blend does not memorise its training period**: its in-sample MAE, 4.72 days, is level with the validation period's 4.78 and *above* the test window's 3.90, and the fall in R^2 from 0.41 in-sample to 0.01 on test is the test period's small variance rather than a large error — the level shift of Section 10, not over-fitting. **The derived late flag is stable**: ROC-AUC 0.80, 0.79, 0.78 across the three periods, while its lift over the floor falls from 4.5× to 2.9× only because the floor itself moves with the late rate. **The stacking ensemble shows the expected, modest in-sample optimism of a tree ensemble**: ROC-AUC 0.84 on the period it was fitted to against

Table 23: The deployed models on the training, validation and test periods. Thresholds and the margin-to-risk mapping are the ones fitted on the validation period and shipped. Lift is PR-AUC divided by that period's own positive rate.

Model	Period	n	MAE	RMSE	R^2				
P1A lead time (Voting)	training	70,617	4.718	7.807	0.408				
	validation	12,957	4.784	6.592	0.222				
	test	12,630	3.898	5.342	0.009				
		n	**positives**	**ROC-AUC**	**PR-AUC**	**lift**	**bal. acc**	**F1**	**Brier**
P1B late flag (derived)	training	70,617	8.8%	0.803	0.396	4.48×	0.713	0.391	0.068
	validation	12,957	5.0%	0.787	0.166	3.34×	0.648	0.227	0.044
	test	12,630	7.4%	0.782	0.214	2.89×	0.705	0.235	0.066
P2 low review (Stacking)	training	72,380	15.8%	0.836	0.617	3.90×	0.754	0.601	0.089
	validation	13,094	11.4%	0.762	0.409	3.60×	0.678	0.436	0.083
	test	12,855	11.0%	0.749	0.414	3.75×	0.681	0.470	0.078

0.76 and 0.75 on the two it never saw, and PR-AUC 0.62 against 0.41 — but validation and test agree with each other (lift 3.6× and 3.75×, Brier improving from 0.083 to 0.078 as the base rate falls), which is the comparison that governs deployment.

K Feature catalogue

Every feature, with its definition, the reason it is in the model and the moment it becomes knowable. Literature pointers are to the works discussed in Section 2; empirical pointers are to Section 4. All 36 are built by `src/features.py` from the nine tables alone.

Table 24: The 36 features by group. "Checkout" features are admissible for both problems; "delivery" features only for Problem 2.

Feature	Definition	Why it is in the model	Known at
The promise (2)			
`estimated_days`	promised delivery date minus purchase time, days	the platform's own routing knowledge, shown to the customer at checkout; strongest single correlate of lead time after distance ($r = 0.38$) and a decision variable in its own right [9, 41]	checkout
`long_promise`	promised window above the pinned median of 23.2 days	flags the routes the platform itself expects to be slow; pinned so serving and training use the same cut	checkout
Basket and money (11)			
`n_items`, `n_sellers`	items and distinct sellers in the order	multi-item and multi-seller orders consolidate before shipping; order composition is the standard control in order-level logistics studies [11]	checkout
`is_multi_seller`	more than one seller	split shipments wait for the slowest seller	checkout
`log_price`	$\log(1 + \text{item total})$	money is right-skewed; the log stabilises the linear family; price relates to the rating, not to lead time (Figure 8)	checkout
`price_per_item`	item total divided by items	separates one expensive item from many cheap ones	checkout
`total_freight`	freight paid, R$	the platform's own pricing of distance and bulk	checkout
`freight_ratio`	freight divided by item total	the delivery cost as the customer experiences it, relative to the purchase	checkout

Table 24, continued

Feature	Definition	Why it is in the model	Known at
`log_weight`, `max_volume_cm3`	log(1 + total weight); bulkiest item's volume	handling and carrier constraints; physical attributes are standard delivery-time inputs [41]	checkout
`total_photos`	listing photos across the items	a listing-quality proxy for the expectation gap behind a low rating	checkout
`max_installments`	largest instalment count	payment behaviour and an affordability signal characteristic of this market	checkout
Calendar (6)			
`purchase_month`, `purchase_dow`, `purchase_hour`	month, weekday, hour of purchase	seasonality and the weekly and daily rhythm of fulfilment (Figure 4); the usual calendar encodings for transactional data [19, 28]	checkout
`is_weekend`, `is_business_hours`	weekend; 09:00–18:00	an order placed outside working hours waits for the next handling day	checkout
`is_nov_peak`	purchased on or after 20 November	the Black Friday surge stresses fulfilment: the late rate reached 14.3% in November 2017 (Section 4.2)	checkout
Geography (4)			
`log_distance`	log(1 + km) between postcode centroids	the strongest checkout-time correlate of lead time ($r = 0.39$); distance is the backbone of delivery-time models [30]	checkout
`same_state`	seller and customer in the same state	the cheapest route class; only 35.7% of orders	checkout
`customer_state`	destination state (27 levels)	regional logistics; rare levels pooled by the encoder	checkout
`seller_region`	origin region (5 levels)	origin logistics, collapsed from 27 states so the linear family is not swamped by sparse levels	checkout
Catalogue and payment (2)			
`lead_category`	category of the most expensive item (74 levels, rare ones pooled)	product type drives both handling and expectations; the category's effect is otherwise invisible to the model	checkout
`lead_payment_type`	payment method of the largest payment	a boleto is paid at a bank and clears days after a card, so handling starts later; the top permutation feature for lead time (Section 9)	checkout
Track records (8), each an as-of aggregate over orders that had closed before this one was placed			
`hist_seller_lead_days`, `hist_seller_late_rate`, `hist_seller_low_review_rate`	the seller's mean lead time, late rate and 1–2 star rate over at least five earlier orders	a party's past performance is the standard predictor in supply-chain delay models [1]; the time-respecting form of target encoding [31]	checkout
`hist_seller_n`	number of earlier orders the seller estimate rests on	cold start as information: the model can discount a seller it has barely seen	checkout
`hist_route_lead_days`	mean lead time on the seller-state to customer-state route	origin–destination history, the route-level analogue of the above [41]	checkout
`hist_category_low_review_rate`	the category's past 1–2 star rate	categories differ in how often they disappoint; encoded without seeing the future	checkout
`hist_platform_lead_days`, `hist_platform_late_rate`	platform-wide trailing 28-day mean lead time and late rate	the drift tracker: tells the model how fast the network is running right now [16]	checkout

Table 24, continued

Feature	Definition	Why it is in the model	Known at
Delivery outcome (3), Problem 2 only			
`delivery_days`, `delay_vs_estimate_days`, `is_late`	realised lead time; days late against the promise; late flag	the dominant driver of a 1–2 star review, 9.2% to 54.0% (Section 4.4; Deshpande and Pendem [11]); known once the parcel has arrived and forbidden before	delivery

L The pilot, per contact and as designed on slide 11

Section 9.4 sizes one randomised arm for each contact, on the customers that contact can still reach (Table 25, from `report/results/customer_pilot_arms.csv`). Every arm uses a two-sided two-proportion test at $\alpha = 0.05$ (`pilot_design.n_per_arm`); the duration is the number of weeks the eligible customers take to fill every arm. The control rate is the share of those customers who went on to rate the order 1–2 stars on the test window. It is a historical rate, not a measured control group: the tables record no contact other than the survey.

Table 25: Pilot size per contact, from `customer_pilot_arms.csv`. Arm B randomises every customer whose promised day ends without a parcel and who has not yet rated the order; Arm A, each day's ten riskiest delivered parcels whose customer has not yet rated the order, five of them contacted; the design on slide 11, shown for comparison, each day's top ten of the purchase-day list over all reviewed orders, five called. A day: customers randomised a day. Control: the share who went on to rate the order 1–2 stars. Reduction: the relative reduction in that share the arm is sized to detect (target), beside the one at which the contact breaks even under the assumed R$5 : R$40. n: customers per arm.

			reduction			80% power		90%
Arm	**a day**	**control**	**target**	**break-even**	**treated**	**n/arm**	**weeks**	**weeks**
B: the promised-date message, against current practice	13.0	0.547	25%	22.8%	0.410	209	5	7
B with a third arm: the message with the survey unchanged	13.0	0.547	25%	22.8%	0.410	209	7	10
A: the delivery-day call, five of each day's ten	10	0.260	48.1%	48.1%	0.135	159	5	7
A, sized for a one-in-four effect (reference)	10	0.260	25%	48.1%	0.195	652	19	25
Slide 11: the purchase-day top ten, five called	10	0.572	25%	21.9%	0.429	191	6	8

Arm B randomises every customer whose promised day ends without a parcel and who has not yet rated the order: 804 reviewed test orders over the 62 purchase days, about 13 a day, of whom 54.7% went on to rate the order 1–2 stars. These figures count the customers who went on to rate the order; a live pilot would also randomise the few overdue customers who never rate, which lowers the control rate slightly and lengthens the pilot. At the assumed costs the message pays if it turns around 22.8% of them, so the arm is sized for one in four; the optional third arm, the message with the survey unchanged, separates the message from the timing of the survey. Arm A lists each day's ten riskiest delivered parcels whose customer has not yet rated the order and contacts five. Its control rate of 0.260 means the call pays only if it turns around 48.1% of those

customers, so the arm is sized for that break-even effect: an effect large enough to pay shows in five weeks, while a one-in-four effect, which would not pay, needs 652 customers per arm and 19 weeks. Arm A's durations assume ten listed every day; on the test window the delivery-day list held 8.7 customers a day on average (ten on 50 of its 63 delivery days), which stretches the five weeks to six.

Table 26 is the design shown on slide 11: each day's top ten of the purchase-day list over all reviewed orders, five called and five held back. Its control rate of 0.572 mixes customers no delivery-day call can reach (26.4 in 100 never received the parcel and 22.0 had already rated a late one while waiting), so a call could change at most 8.8 points, below the 12.5-point break-even; Section 9.4 therefore sizes each arm on the customers it can reach.

Table 26: The design shown on slide 11 (the purchase-day list over all reviewed orders), by assumed effect, from `report/results/pilot_design.csv`. The control rate is the list's historical precision at that daily size on the test window, not a measured control group; n is orders per arm for a two-sided two-proportion test at $\alpha = 0.05$; the duration assumes K orders randomised a day (half contacted); net benefit per contact is under the assumed R\$5 : R\$40. Figure 24.

K/day	control	reduction	treated	n/arm (80%)	wks (80%)	wks (90%)	net/contact
10	0.572	15%	0.486	531	16	21	−R\$1.57
10	0.572	20%	0.457	299	9	12	−R\$0.43
10	0.572	**21.9% (break-even)**	0.447	250	8	—	R\$0.00
10	0.572	25%	0.429	191	6	8	R\$0.72
10	0.572	30%	0.400	133	4	6	R\$1.86
10	0.572	40%	0.343	74	3	3	R\$4.15
20	0.440	20%	0.352	484	7	10	−R\$1.48
20	0.440	**28.4% (break-even)**	0.315	235	4	—	R\$0.00
20	0.440	30%	0.308	210	3	5	R\$0.28
20	0.440	40%	0.264	115	2	3	R\$2.04

The slide's design randomises inside the top-K list, so both arms share the same predicted risk. Two of its properties carry over to the per-contact arms. The break-even effect grows with the size of the list — 21.9% at ten orders randomised a day, 28.4% at twenty, 42.1% at forty — because precision falls as the list is worked deeper, so a contact lower down has less to turn around; a pilot should therefore run at the size that would be deployed. And the pilot is short at the effect sizes that matter: six weeks detects a one-in-four reduction at ten a day, while an effect too small to pay for itself would take sixteen weeks to confirm, which is itself informative — if a pilot has not found an effect in two months, the contact should not be scaled.

M When is the review written?

Problem 2 is framed at delivery, before the review is written. Olist's review table carries two timestamps — `review_creation_date`, the day the satisfaction survey was sent, and `review_answer_timestamp`, the moment the customer answered — and comparing the second with the delivery timestamp tests that framing (`src/review_timing.py`). It holds for parcels that arrive on time and fails for most late ones. The timestamps show the survey going out around delivery or, for a parcel still on the road, after the promised date has passed (the rule itself is not recorded), so the customer of a late parcel is usually asked to review while still waiting (Table 27).

The 87.3% is the analysis window's figure. On the test window it is 77.4% of the 287 late parcels rated 1–2 stars (`customer_survey_timing.csv`). The order of Box 1 (Section 1) shows the pattern in one case, and it is typical: for 71.4% of those 287 customers the

Table 27: Review timing on the 95,561 delivered and reviewed orders of the analysis window, from `report/results/review_timing.csv`. A negative median means the review was answered before the parcel arrived.

	Late parcels	On-time parcels
Delivered and reviewed orders	7,659	87,902
1–2 star rate	54.0%	9.2%
Survey sent on or after the promised date	99.0%	2.1%
1–2 star reviews written before the parcel arrived	**87.3%**	0.5%
Share of all 1–2 star reviews	33.8%	66.2%
Median answer of a 1–2 star review, days after delivery	−4.5	2.1

survey went out exactly two days after the promised date, and none of their reviews was answered during the promised day itself, so a message when that day ends would have come before every one of them. Over the analysis window the same two figures are 68.5% of 4,136 and 99.6% (the rest were answered before the promised date).

Table 28: The review-risk list on the test window, its hits split into customers who had not yet rated the order when the parcel arrived (with every hit whose order never arrived, whenever they rated) and those who already had, from `report/results/review_timing_capacity.csv`. Precision is the share of listed customers who went on to rate the order 1–2 stars; the two columns after it sum to it. Late share: the share of the list whose parcel was late.

Capacity	precision	not yet written	already written	late share
top 1%	0.705	0.419	0.287	0.403
top 5%	0.678	0.362	0.316	0.521
top 10%	0.475	0.302	0.173	0.376
5 a day	0.631	0.353	0.278	0.434
10 a day	**0.572**	**0.352**	0.220	0.362
20 a day	0.440	0.288	0.152	0.298
contacting at random	0.110	0.093	0.017	—

Two consequences follow. **Not everyone on the list can still be helped at delivery.** At ten a day, 0.572 of the listed customers went on to rate the order 1–2 stars, but 0.220 had already rated it — almost always while waiting for a late parcel — and 0.352 had not, against 0.093 at random (Table 28). Counting only those not yet written, the value under the assumed R$5 : R$40 falls from R$1,466 to R$775 per 1,000 reviewed orders at the same threshold, 0.11, and at ten a day from R$853 to R$433; both figures still count every contact as turning its customer around, and both include orders that never arrived (below). Capacity changes how the optimum is used rather than where it is: a team that can work only 5% of reviewed orders still captures R$1,107 of those R$1,466 per 1,000 (Table 6; `capacity_p2.csv`), because the top of the list is where the precision is. **The action for a late parcel belongs on the promised date, not at delivery.** Customer care can reach the customer when the promised day ends without a parcel, with an apology, the parcel's status and a new date, and the survey can follow once the order is complete; every customer is still asked, and no review is filtered, delayed in publication or rewarded. That contact needs no model, since the calendar identifies the customers, and it is Arm B of the pilot (Section 9.4). The review-risk list keeps its value for the parcels that arrive on time, which produce two thirds of all 1–2 star reviews and are reviewed a median 2.1 days after delivery. On those parcels alone — delivered on time, 11,654 test orders at a 7.9% base rate — the deployed model still ranks usefully, if

less sharply: ROC-AUC 0.651, and 28.0% of its top 5% went on to rate the order 1–2 stars, 3.5 times their base rate; on all delivered parcels (12,571, base rate 9.6%) the top 5% reach 53.9%, 5.6 times the base rate (`review_timing_subsets.csv`).

Who each contact can still reach. Each customer who rated the order 1–2 stars falls into one of four classes by the contacts that would still have come before the rating: a message when the promised day ended without a parcel (the customer still waiting, and not yet rated then), a call at delivery (the parcel delivered and not yet rated), either, or neither. Of the 1,418 test-window customers who rated 1–2 stars, 945 could have been reached only by a call at delivery, 398 only by the promised-date message, 42 by either and 33 by neither (`customer_outcomes_test.csv`). Table 29 splits each list's hits the same way. Most of the slides' hits are customers no call at delivery could help. The list a team would actually work — each morning, the previous day's deliveries whose customer had not rated the order when it came — reaches 26 in 100, 3.2 times its 8.1% base rate, and the promised-date message reaches 55 in 100 with no model at all.

Table 29: Who each list reaches on the test window, from `customer_contact_lists.csv`. Hits are listed customers who went on to rate the order 1–2 stars, split by the contact that could still have come before the rating: only the promised-date message, only a call at delivery, either, or neither. A day: listed customers per day of the window. Random: the 1–2 star rate of the population the list is drawn from (the message contacts all of its population, so it has none). The first row is every customer of the test window who rated 1–2 stars (`customer_outcomes_test.csv`).

					hits reachable by			
List	**listed**	**a day**	**precision**	**random**	**message**	**call**	**either**	**neither**
All who rated 1–2 stars	—	—	—	—	398	945	42	33
Slides: ten a day, by purchase day	614	9.9	0.572	0.110	265	48	6	32
Slides: threshold 0.11	1,927	31.1	0.369	0.110	398	245	37	32
Deployed threshold 0.19	981	15.8	0.575	0.110	395	107	30	32
Delivery-day list, ten a day	546	8.7	0.260	0.081	0	118	24	0
Promised-date message (no model)	804	13.0	0.547	—	398	0	42	0

Orders that never arrived. The Problem 2 population is every reviewed test order, and 284 of the 12,855 (2.2%) were never delivered within the extract: 122 cancelled, 98 still *shipped*, 36 invoiced, 24 unavailable and 4 other. Their delivery fields are missing, which the model reads as a strong signal — 73.2% of their customers rated the order 1–2 stars — and they make up 37% of the daily top-ten picks. Of the 57 hits per 100 listed at ten a day, 26 are such orders, 22 are late parcels whose review was written before arrival, and 9 are delivered parcels whose customer had not yet written (`review_timing_composition.csv`). A delivery-day call cannot reach the first two groups. Of their 297 customers, 265 had not yet rated the order when their promised day ended (Table 29): they are the customers of the promised-date message of Section 9.2, which an order that misses its date triggers whether or not it ever arrives. Restricted to delivered parcels, the purchase-day top-ten list reaches a customer who went on to rate the order 1–2 stars 45 times per 100 listed (29 already written, 16 still reachable) against a 9.6% base rate, and on 16 July it finds six of its ten. Table 28 and the values above count the never-delivered hits as not yet written; they should be read with that in mind.

N Contributions

Member	Contribution
Pan Pinyou	Problem scoping and stakeholder framing; feature engineering for the track-record features; business interpretation, the cost analysis and the pilot design; report Sections 1, 5 and 9; presentation slides
Li Yudan	Literature review and its revision; modelling pipeline and the family ladders; hyperparameter search; imbalance, calibration and fold-to-fold analysis; report Sections 2, 6 and 7.3
Zhu Gengyu	Data pipeline and the leakage guard; exploratory analysis and the lateness anatomy; cross-validation protocol and the direct-against-derived comparison; dashboard, deployment, tests and reproducibility; report Sections 3, 4, 7.1, 8 and 10

All three members reviewed every section of the report as first submitted and verified the reported numbers against `report/stats_phase3.txt` and `report/stats.txt`; the revision's main tables (the customer counts, the contact lists and their values, the pilot arms, the order of Box 1 and the sources of the slide numbers) are tied to each other and to the earlier results by the assertions in the scripts that write them and by the test suite (Appendix A).

O Figures

Figures 2 to 10 are the exploratory analysis, drawn by notebooks 01–04; Figures 11 to 24 are drawn by `src/figures_phase2.py` from the same CSV tables the body's numbers come from, so a figure cannot disagree with a table; Figures 25 to 28 are the deployed application, and this guide explains how to use it.

Using the application.

1. Open `https://team10-it5006-olist.streamlit.app`. If it is asleep, press *Yes, get this app back up!* and wait about a minute.
2. **Customer journey** opens on the order of Box 1. Pick a customer from any test day's list, or type the first seven characters of an order id.
3. **Predict**: enter an order as it looks at checkout, or upload a CSV (template button; up to 5,000 rows), and read the expected lead time, the date the customer would be quoted, the late risk and the review risk.
4. **Replay a day**: choose 16 July 2018 and ten contacts, read the list blind, then switch on *Reveal what actually happened*.
5. **Model card**: what is deployed, where not to trust it, where each number comes from, and the pilot calculator with both costs as inputs. The CSV contract and the error messages are documented in `deployment/README.md`.

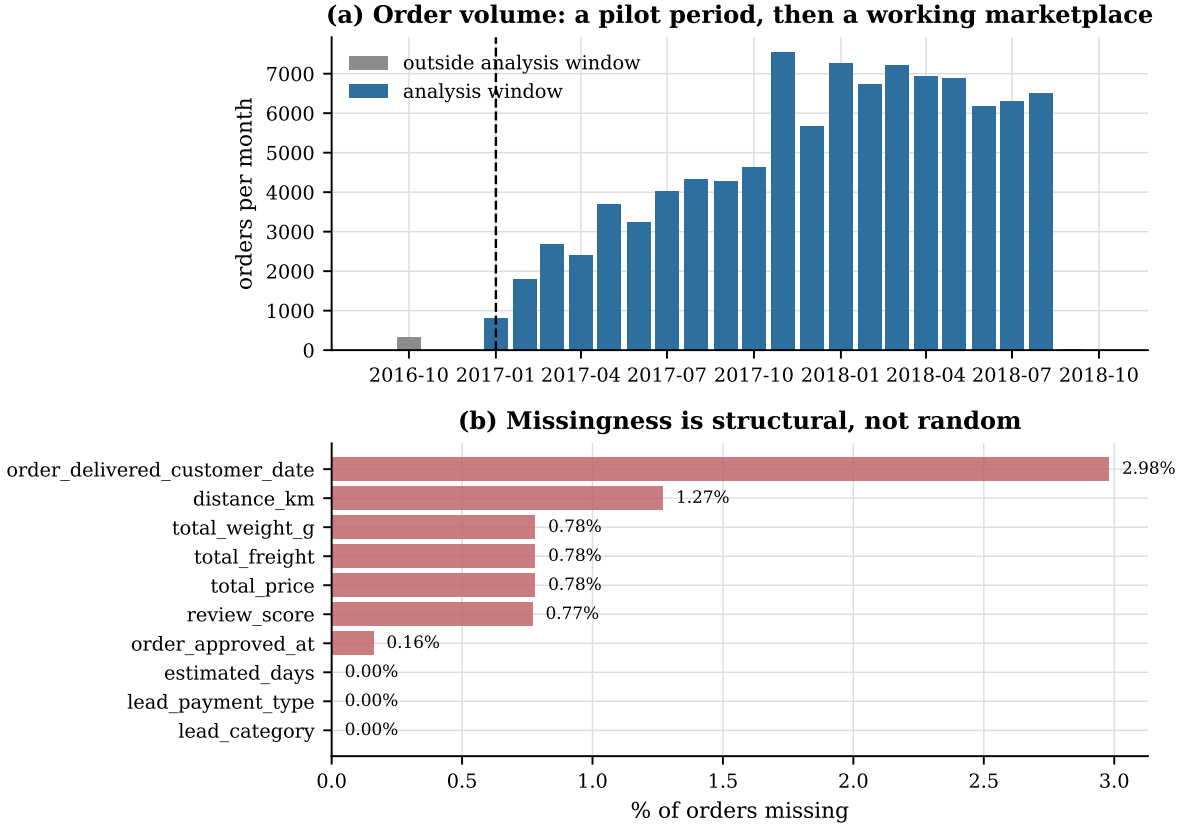


Figure 2: (a) Monthly order volume. The pilot period before 2017 contributes 329 orders in total and is excluded from the analysis window. (b) Missingness by column: structural rather than random, dominated by orders that never reached the customer.

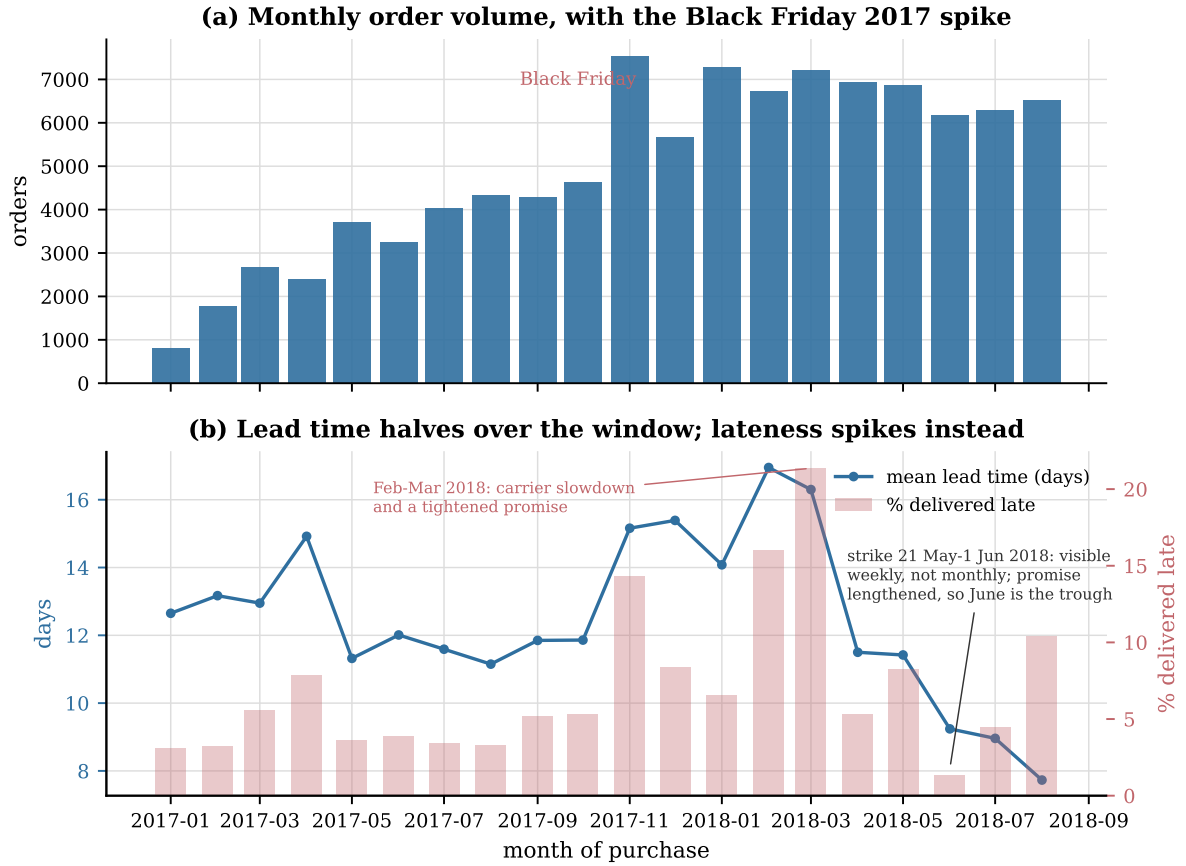


Figure 3: (a) Order volume with the Black Friday 2017 spike. (b) The central temporal finding: mean delivery lead time halves across the window (line, left axis) while the late rate is volatile rather than trending (bars, right axis). The spikes are Black Friday (November 2017) and February–March 2018, a carrier-side slowdown that coincided with a tightened promise. The truckers' strike of 21 May–1 June 2018 is not visible at monthly resolution (Section 4.2).

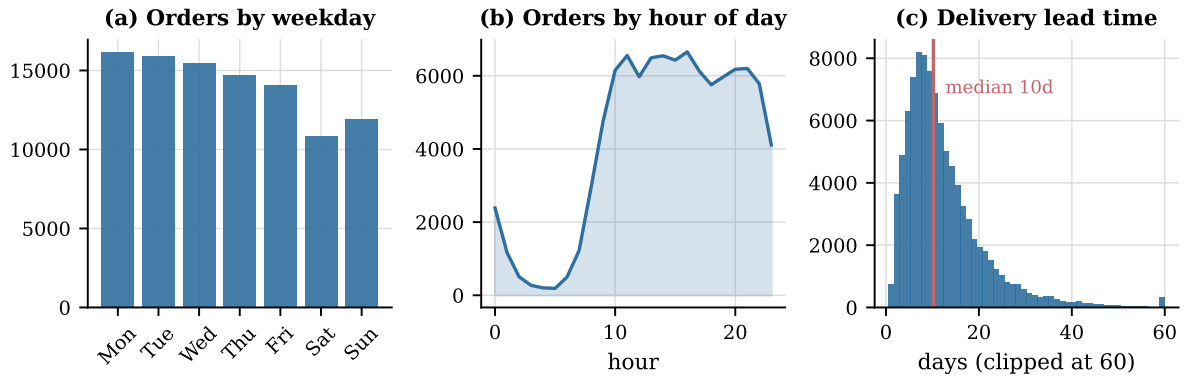


Figure 4: Purchase rhythm and the lead-time distribution. Ordering is weekday-heavy and peaks at 16:00; lead time is right-skewed with a median of 10.2 days and a long tail.

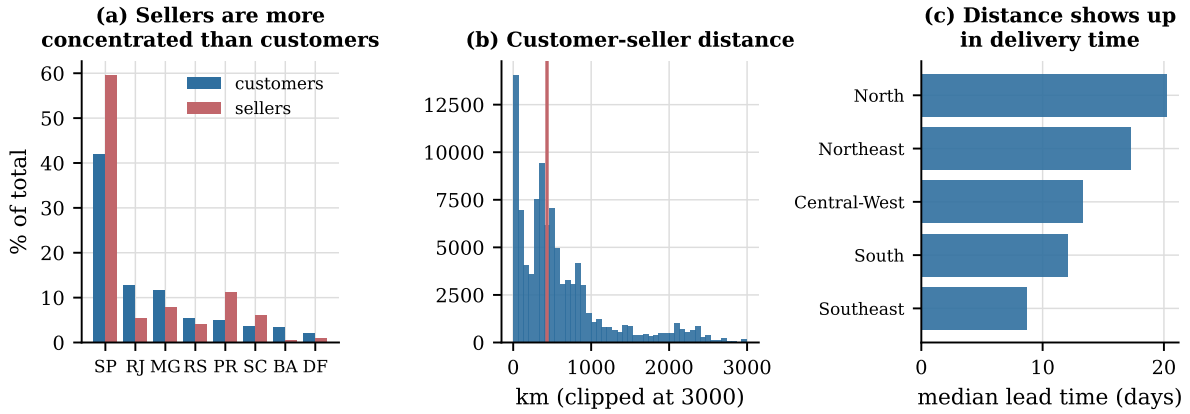


Figure 5: (a) Sellers are far more concentrated in São Paulo than customers are, so most orders must cross the country. (b) Customer–seller distance. (c) That distance shows up directly in regional delivery times.

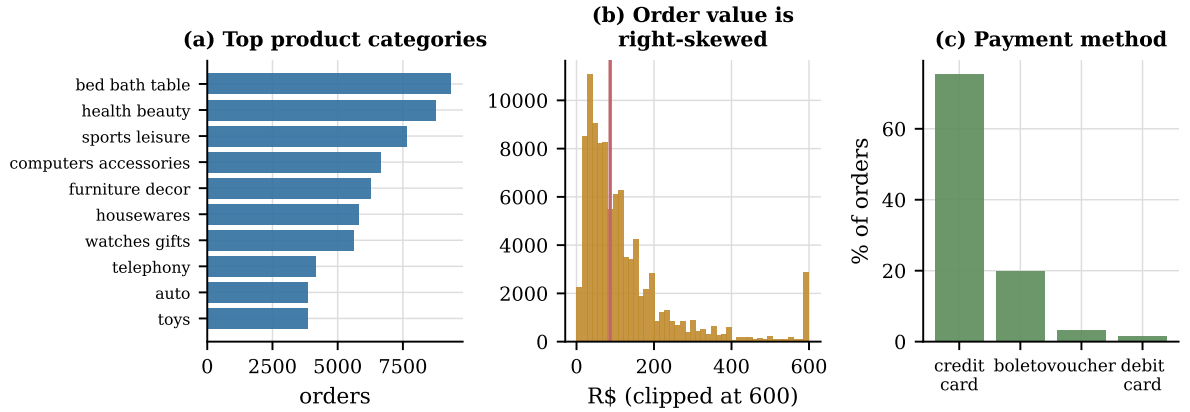


Figure 6: (a) Demand is long-tailed across 74 categories. (b) Order value is strongly right-skewed. (c) Credit card dominates, usually in instalments.

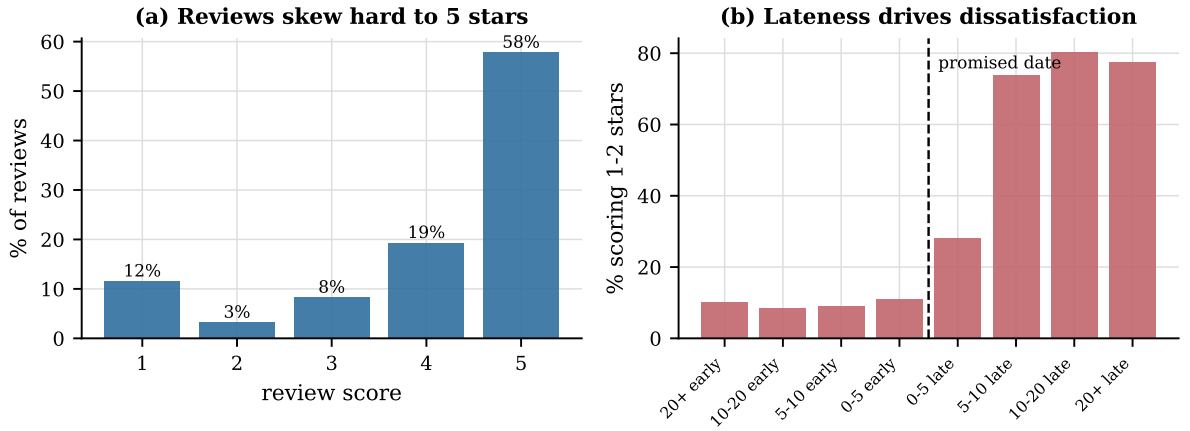


Figure 7: (a) Reviews are severely imbalanced towards five stars. (b) The strongest relationship in the dataset: the 1–2 star rate stays low however early the parcel arrives and climbs steeply once it is late, from 9.2% for on-time orders to 54.0% for late ones.

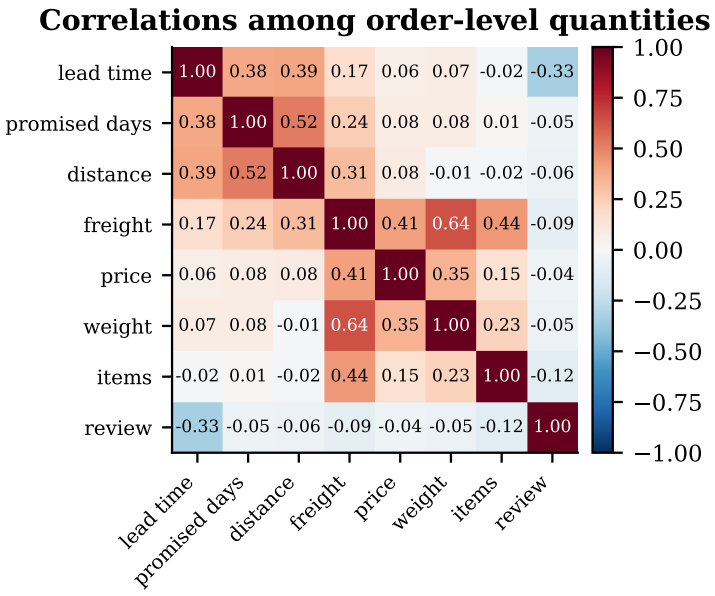


Figure 8: Correlations among order-level quantities. Distance and the platform's own promise are the strongest checkout-time correlates of lead time; the review score moves with lead time and not with price.

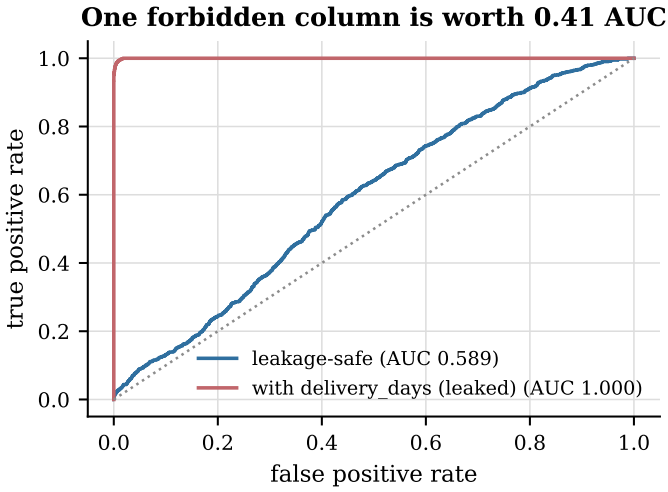


Figure 9: What one forbidden column is worth. Adding the realised delivery time to the late-delivery model takes ROC-AUC from 0.589 to 1.000 — the model is reading the answer, not predicting it.

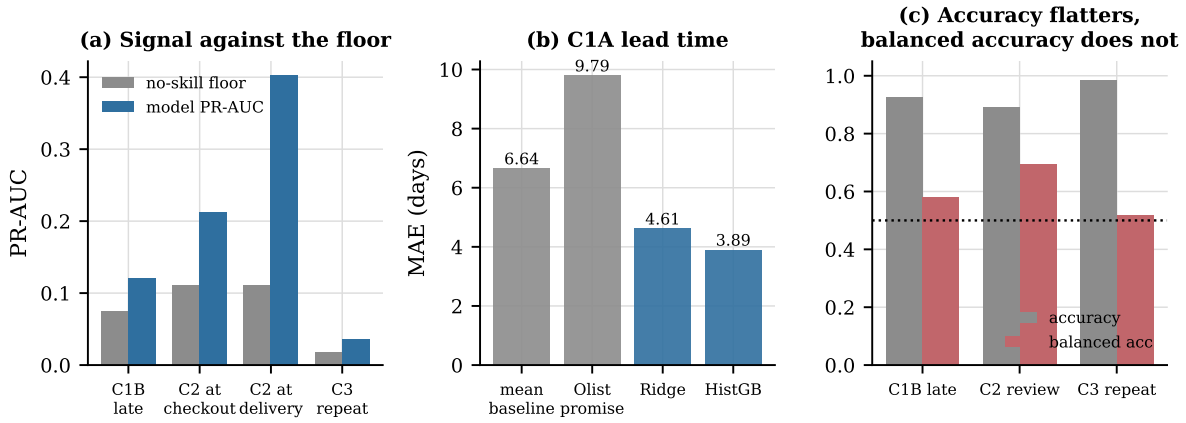


Figure 10: Candidate comparison from the scoping phase. (a) PR-AUC against the no-skill floor for each candidate and prediction moment. (b) Lead-time MAE: the platform's own promise is worse than a constant. (c) Raw accuracy flatters every imbalanced target; balanced accuracy does not.

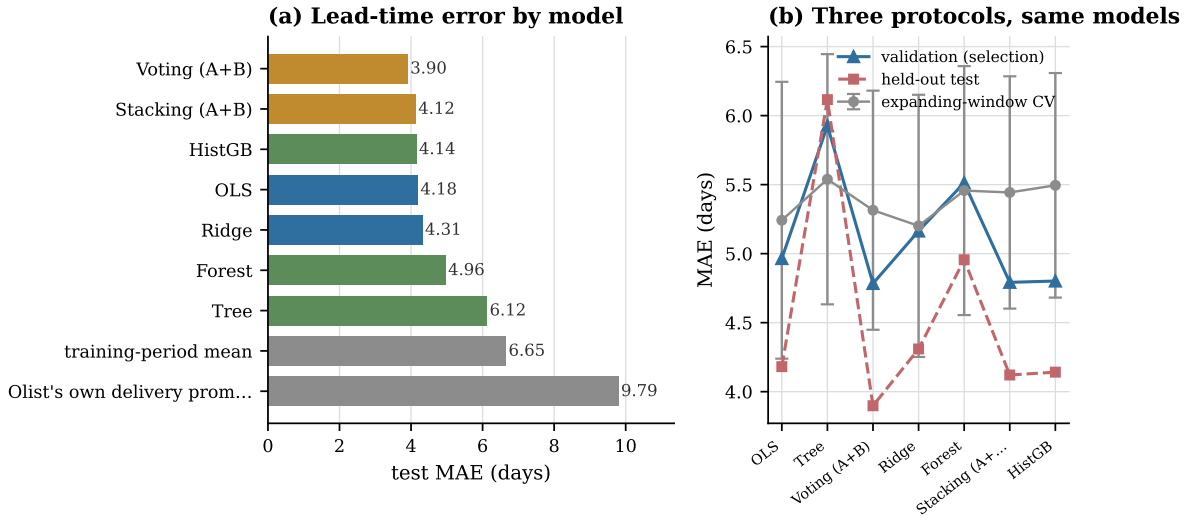


Figure 11: Lead-time error by model, and the same models under three protocols. Panel (b) is the argument of Section 6.2: expanding-window cross-validation does not order the candidates the way the validation period and the forward window — which agree with each other — do.

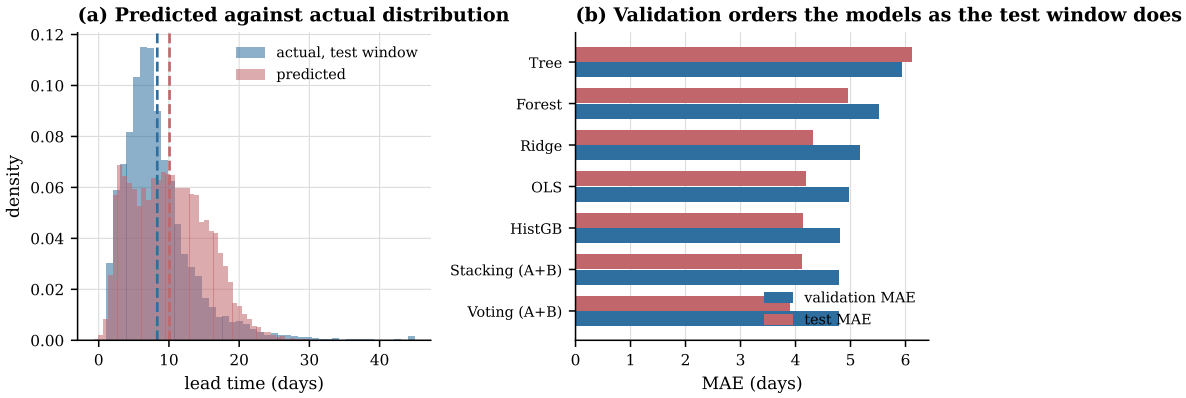


Figure 12: Why blending a linear model with a boosted one helps under drift. A tree cannot predict outside the range of the targets it was fitted on; lead times in the test window are systematically shorter than anything common in training. Panel (b) shows the validation period ordering the models as the test window does.

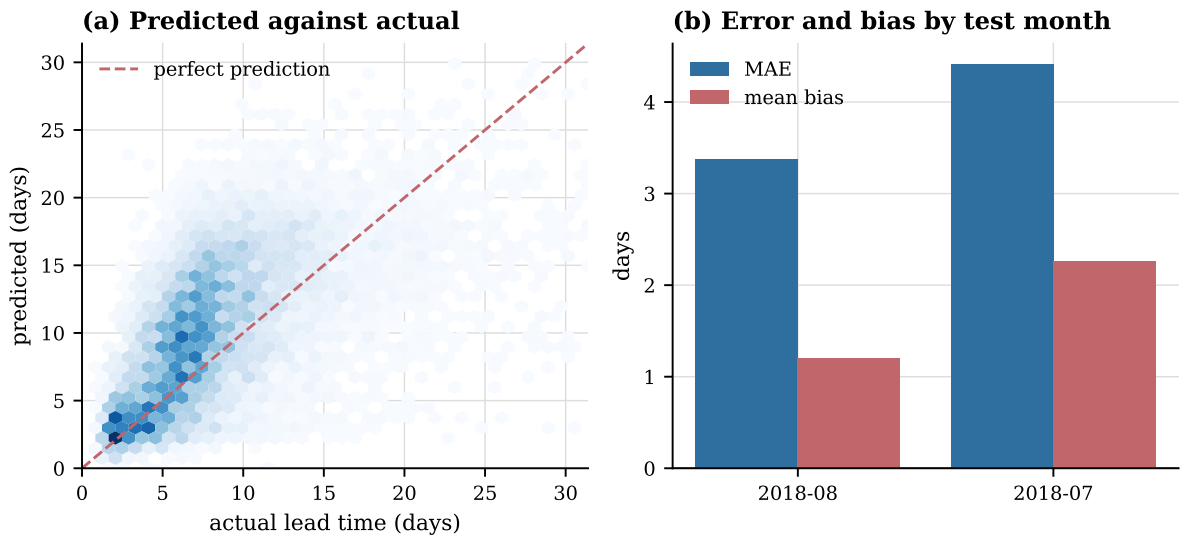


Figure 13: Where the regression is wrong. The bias in panel (b) is positive in both months plotted: a model fitted on a slower past over-predicts how long a parcel will take.

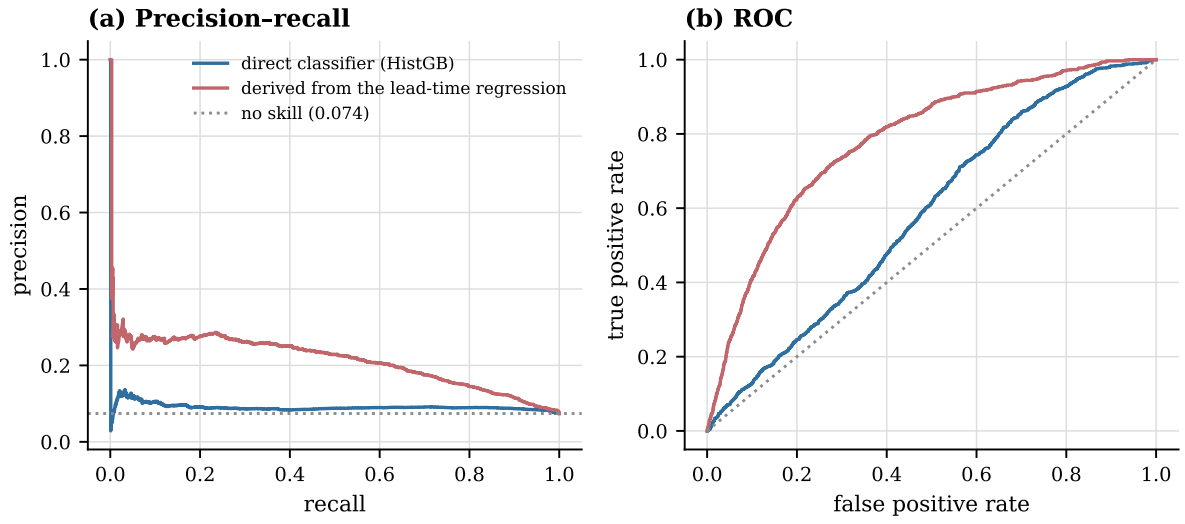


Figure 14: Problem 1B. The flag derived from the lead-time regression against the best purpose-built classifier, on the forward test window.

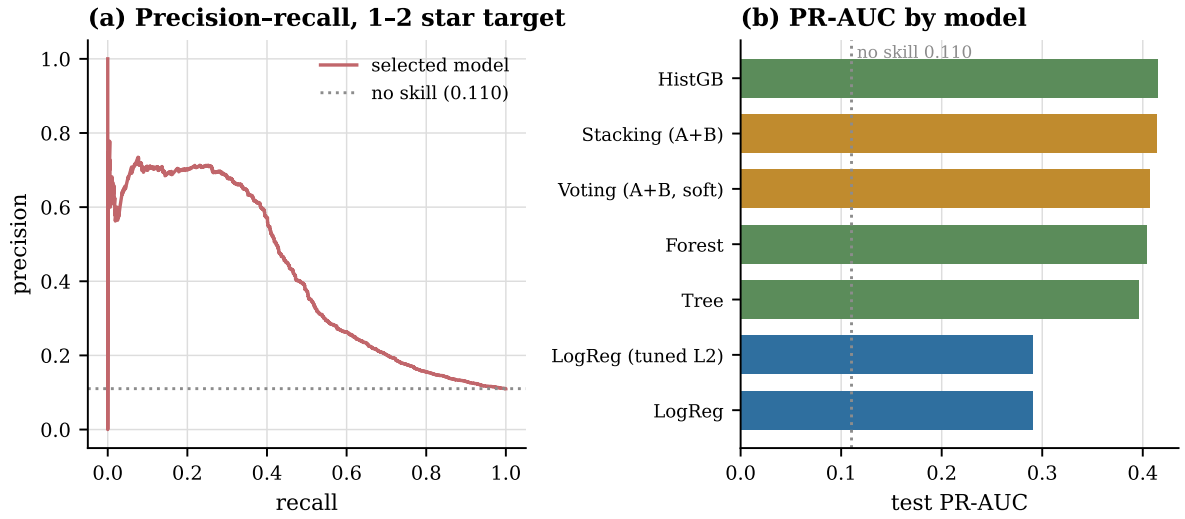


Figure 15: Problem 2 against its no-skill floor, and PR-AUC by model. Most of the achievable signal is already available from a single depth-limited tree.

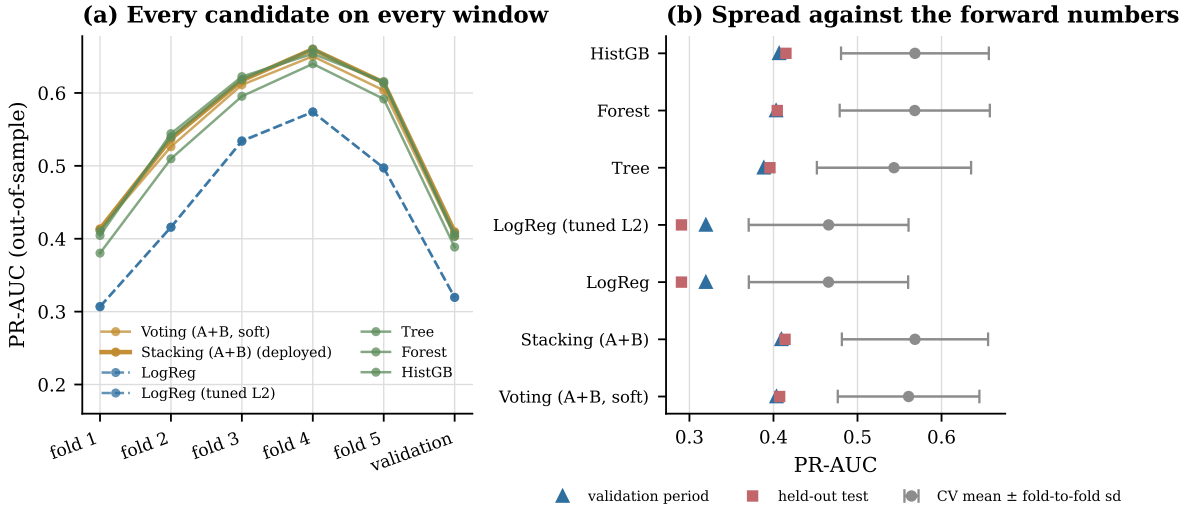


Figure 16: Fold-to-fold variation of every Problem 2 candidate (Table 18). (a) Each candidate's PR-AUC on the five expanding folds and the validation period: the windows move together, because their difficulty differs far more than the candidates do. (b) The cross-validated mean with its fold-to-fold standard deviation beside the validation and test scores: the spread is about 0.09 against a gap of under 0.001 between the best candidates.

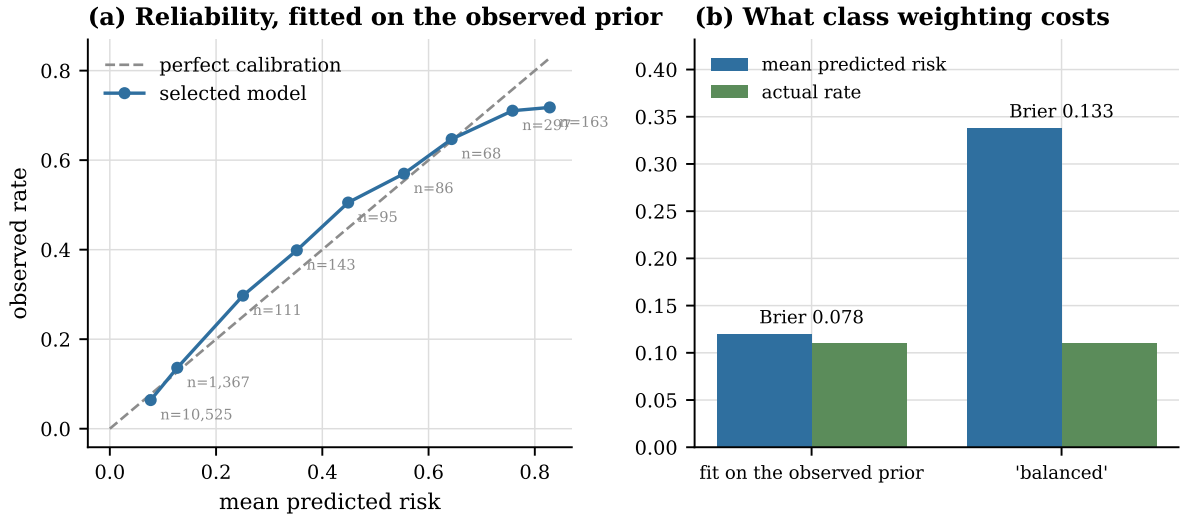


Figure 17: Calibration, and what class weighting costs. Weighting the loss leaves ranking unchanged and inflates mean predicted risk threefold against the actual rate.

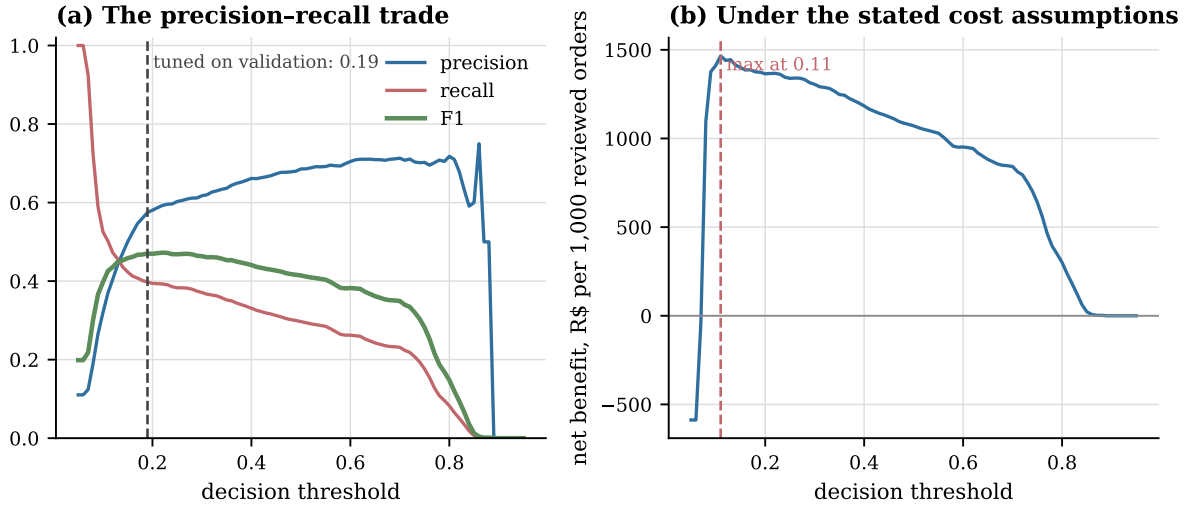


Figure 18: Choosing the operating point for the review model on the test window, and what it is worth per 1,000 reviewed test orders (12,855) under the assumed costs if every contact turned its customer around.

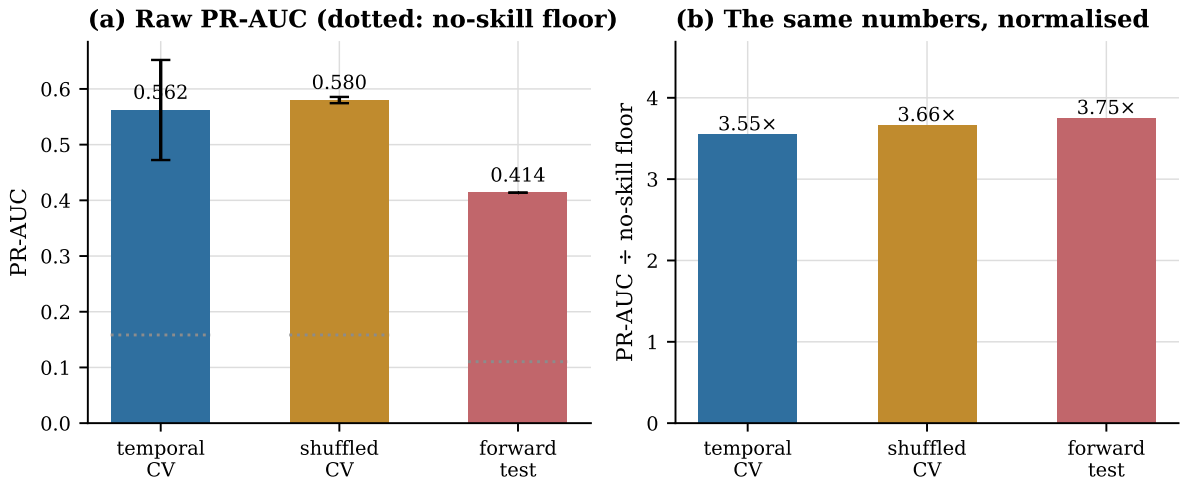


Figure 19: One model, three protocols. Raw average precision disagrees across periods; divided by each period's no-skill floor the three agree to within 0.2 of a multiple.

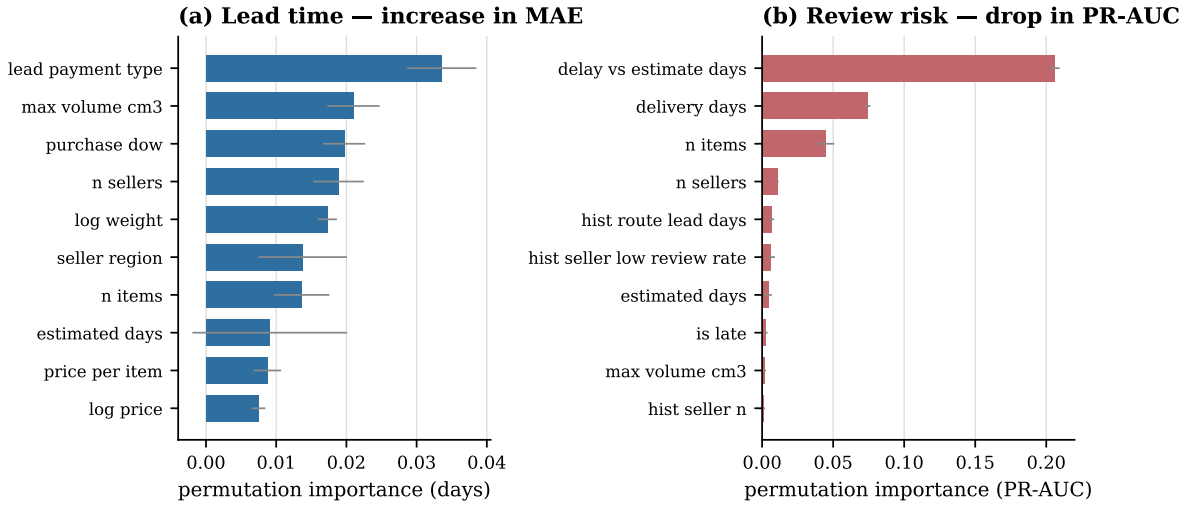


Figure 20: Permutation importance on the test window. The review model rests on two features; the lead-time model on none in particular, which is a property of correlated predictors as much as of the model.

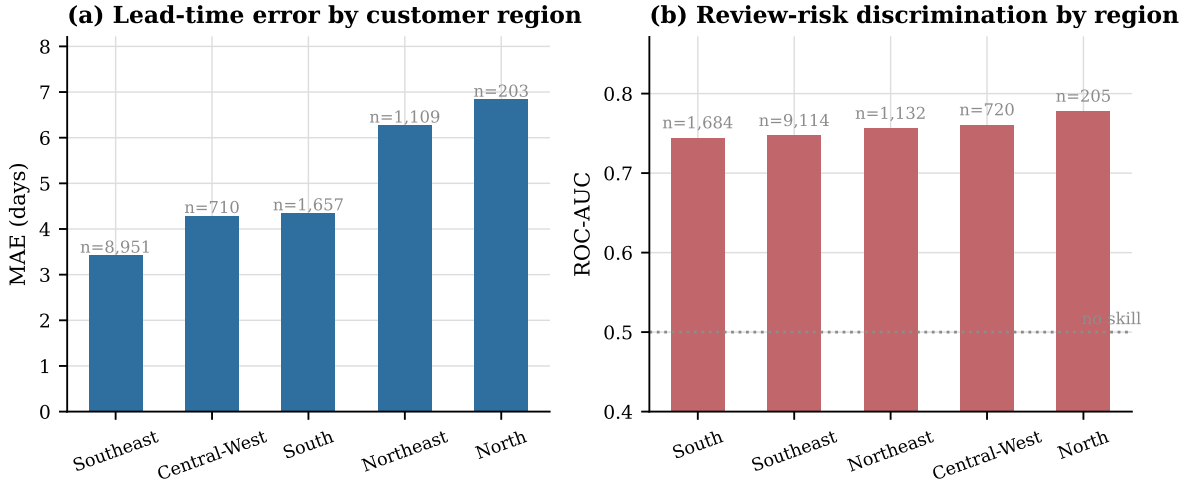


Figure 21: Performance by customer region. The review model is stable across all five; the regression is not, and degrades exactly where deliveries are longest.

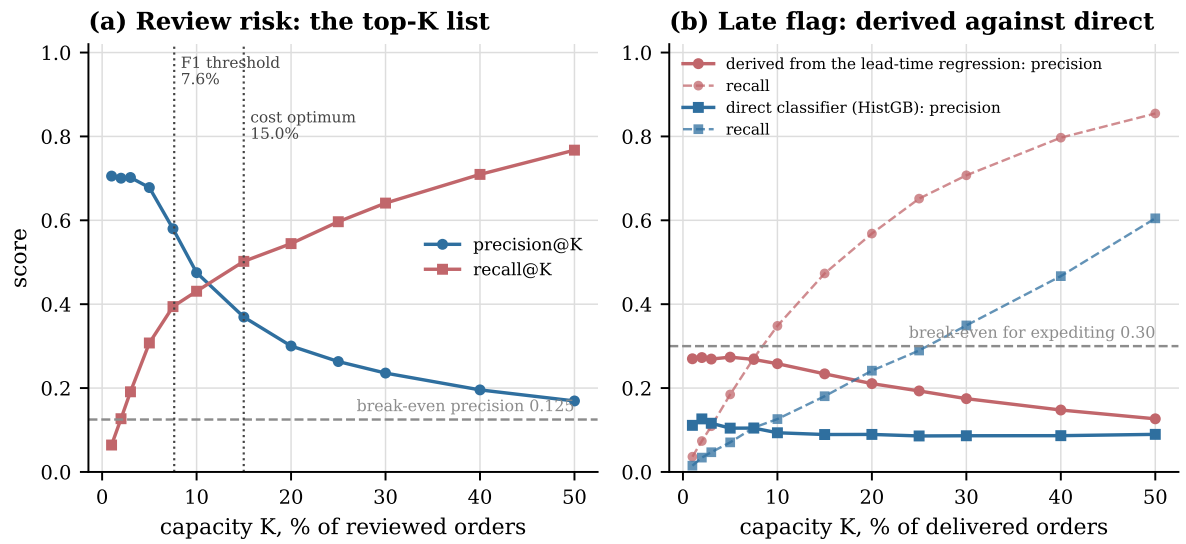


Figure 22: Precision and recall when only the top K can be acted on (Section 7.4). (a) The review-risk list, with the report's two threshold operating points placed on the capacity axis; precision stays above the 0.125 break-even, which assumes every contact works, at every capacity. (b) The late flag: the derived route beats the direct classifier at every K and never reaches the break-even an expensive expedite would need.

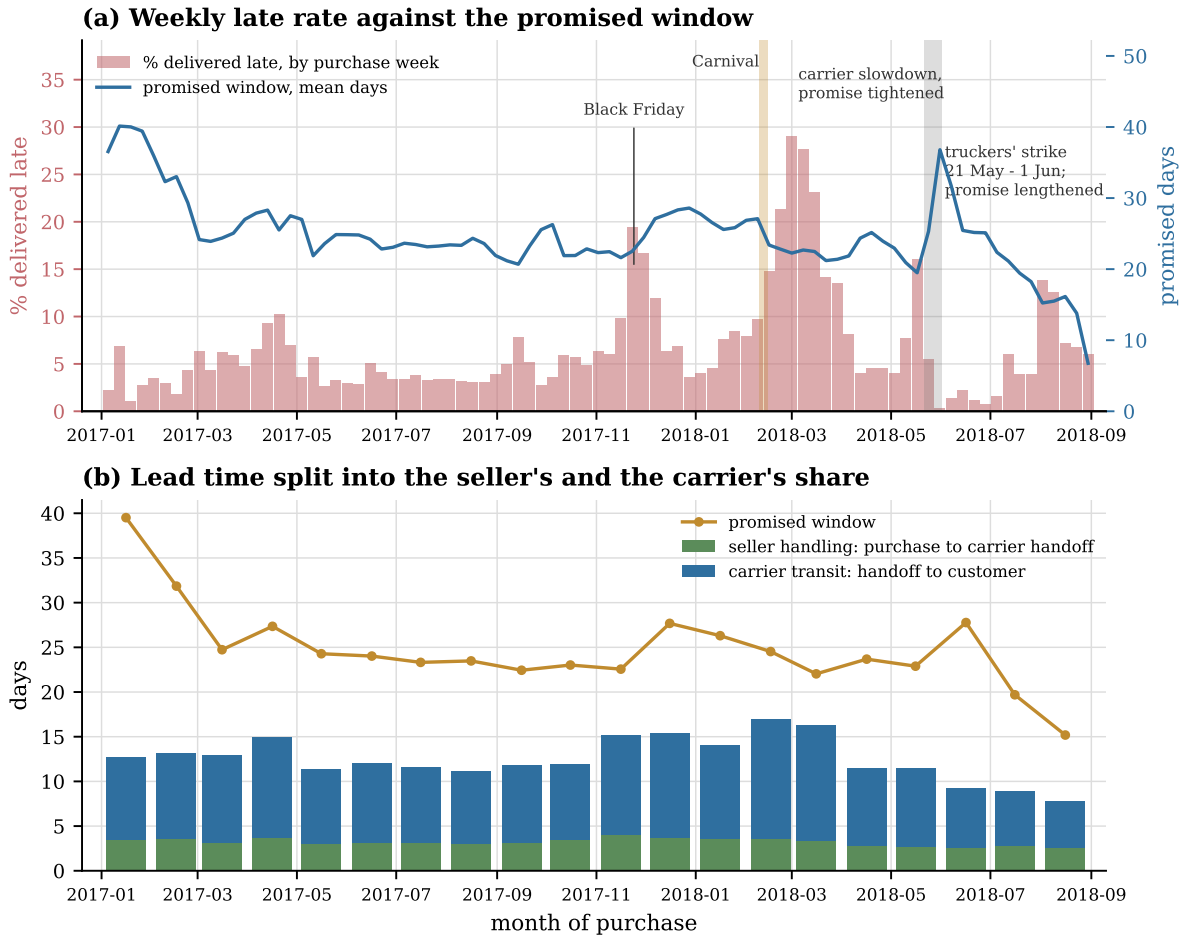


Figure 23: Where the late deliveries came from (Appendix H). (a) Weekly late rate against the promised window: the February–March 2018 peak coincides with a tightened promise, and the strike of 21 May–1 June 2018 (shaded) hits the week promised before it, after which the promise is lengthened and the late rate collapses. (b) Lead time split at the carrier handoff: the peak is carrier transit, not seller handling.

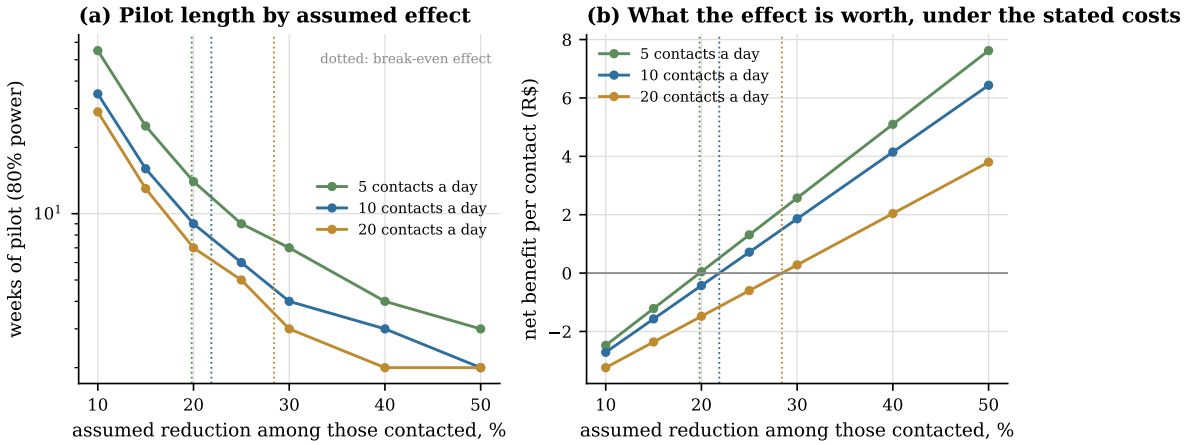


Figure 24: Sizing the pilot as designed on slide 11, on the purchase-day list over all reviewed orders (Appendix L; Section 9.4 sizes it per contact instead). (a) Weeks needed, at 80% power, to detect a given reduction in 1–2 star ratings among the customers contacted, for three daily list sizes (the legend's "contacts a day" are orders randomised a day, half of them contacted); the dotted lines mark each size's break-even effect. (b) The net benefit per contact that reduction would imply under the assumed costs.

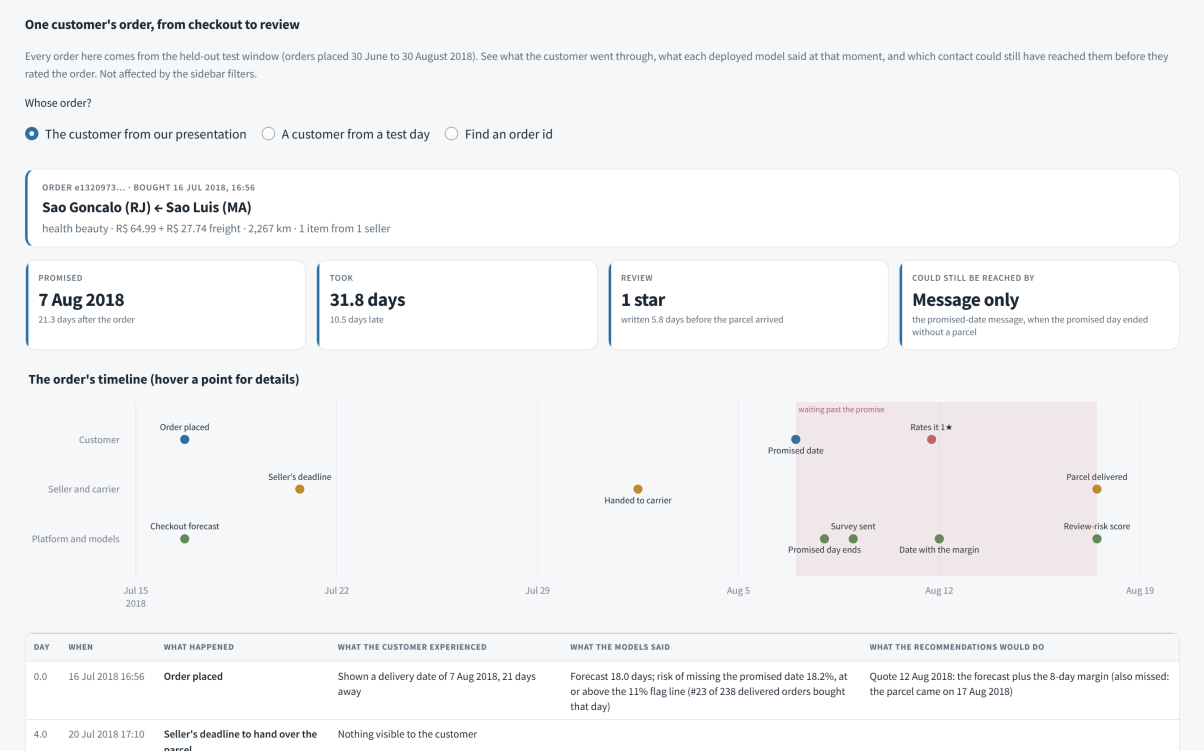


Figure 25: The deployed application, Customer journey: the order of Box 1 from checkout to review — the promise, the wait, the review and which contact could still have reached the customer, the timeline in three lanes, and for each moment what the customer experienced, what the models said and what the recommendations would have done. The customer's own words follow further down the tab.

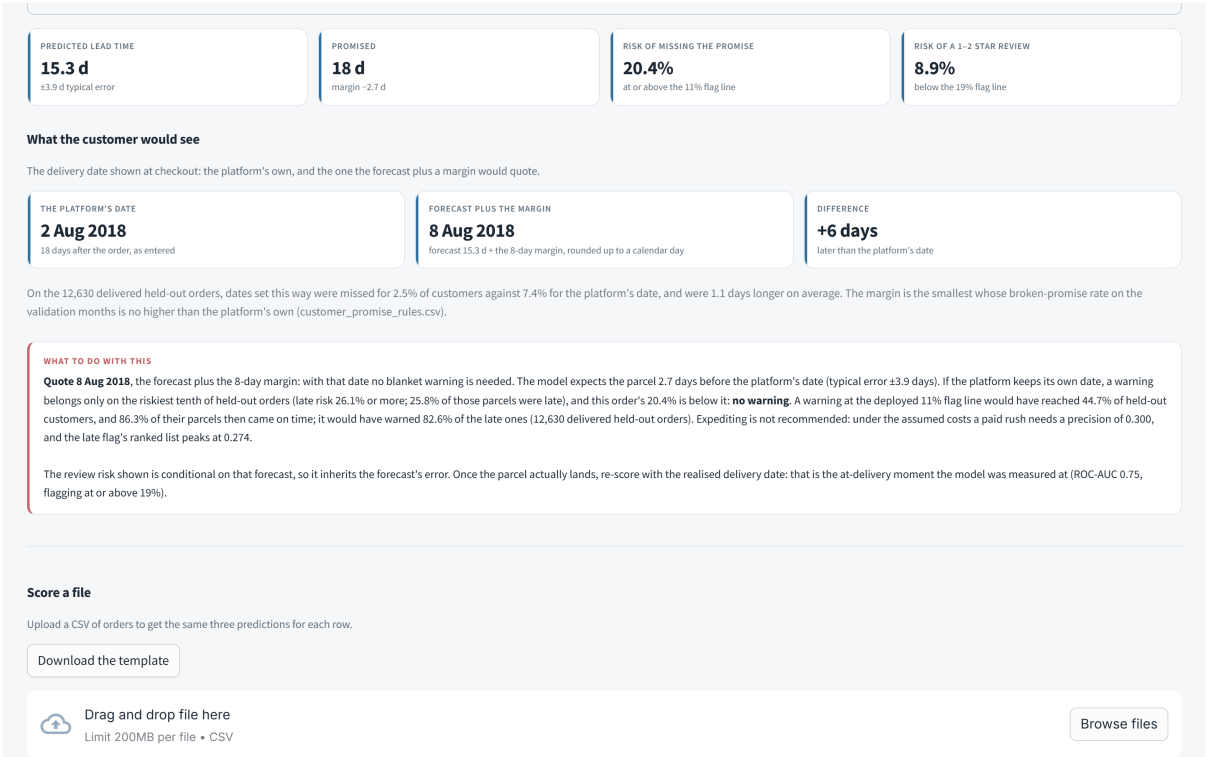


Figure 26: The deployed application, Predict tab, with one order scored at checkout. Below its four scores (lead time, margin to the promise, late risk against the 11% line, review risk against the 19% line) the tab shows what the customer would see — the platform's date, the date the forecast plus the eight-day margin would quote, and the difference — then what to do with the result, and the batch scorer with its template; the top-K triage list follows further down.

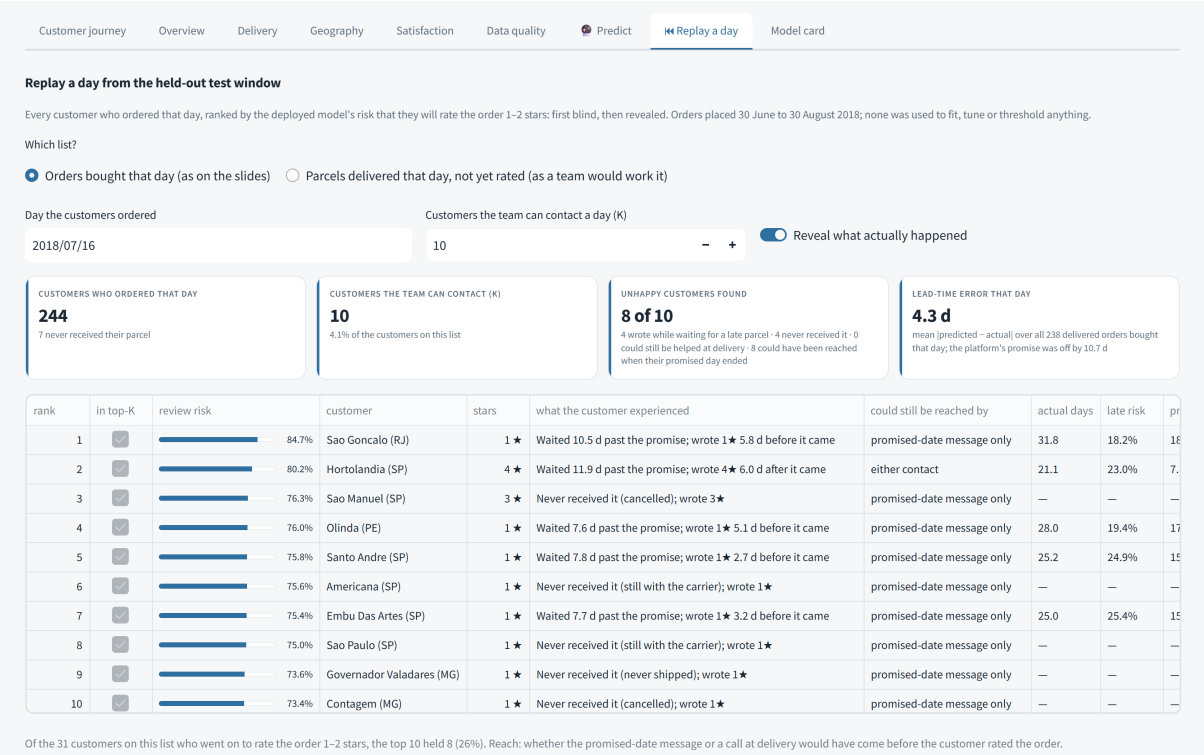


Figure 27: The deployed application, Replay a day, with the outcomes revealed: the customers who ordered on 16 July 2018 (the list grouped by purchase day, as on the slides), ranked by the deployed model's risk that they would rate the order 1–2 stars, with ten contacts. Eight of the ten did: four while still waiting for a late parcel and four whose order never came. None of the eight could still have been helped by a call at delivery, and all eight could have been reached when their promised day ended without a parcel (Appendix M). First on the list, at 84.7%, is the customer of Box 1.

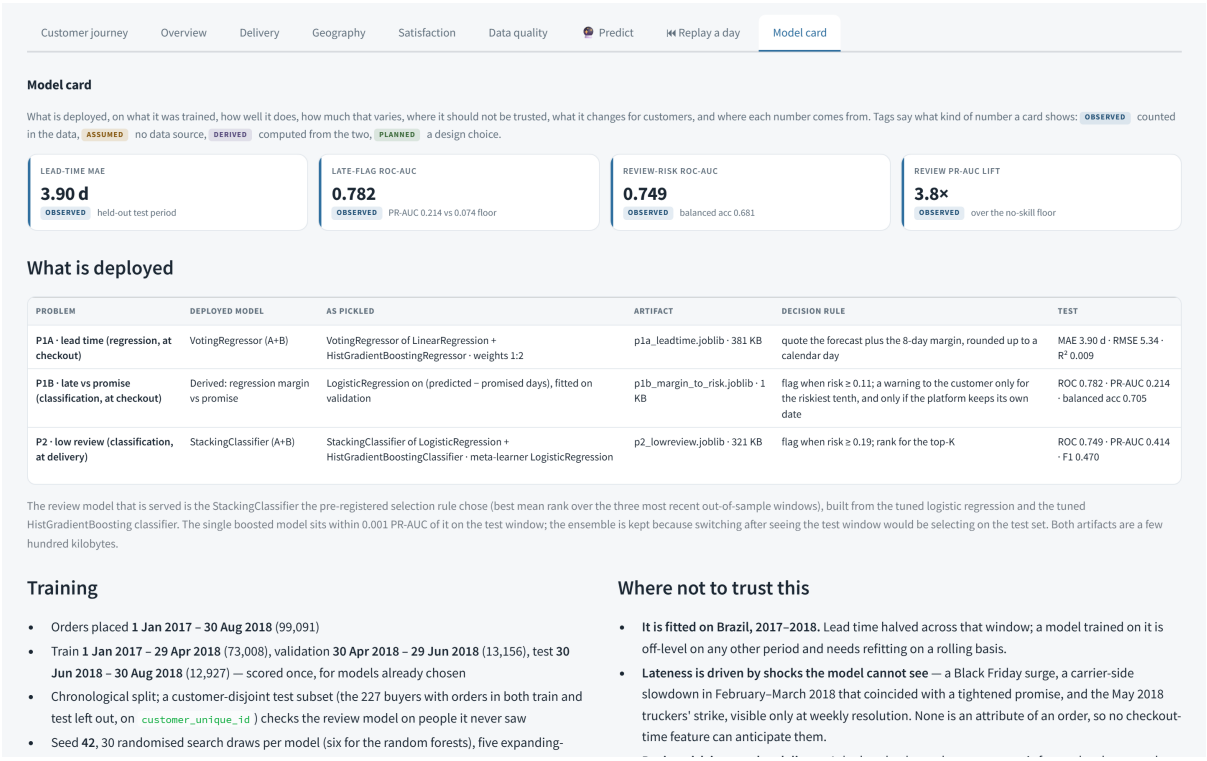


Figure 28: The deployed application, Model card: the headline scores, each tagged with the kind of number it is (observed, assumed, derived or planned), and the estimators as pickled. Further down, the tab gives the training protocol, where the models should not be trusted, the fold-to-fold table, what the recommendations change for customers, the pilot calculator with both costs as inputs, and the source of every number on the slides.

P Where the numbers on the slides come from

The slides were locked on 4 November 2026 and are submitted unchanged, as the brief requires. At the presentation we were asked where the numbers on the business-value slide (slide 11) come from; this appendix answers that for every number on every slide. Table 30 lists each one, slide 11 first, with its kind, its source or formula and, where the slide's wording is wrong or loose, the reading this report stands by. *Observed* numbers are counted in the Olist tables, on the test window (orders placed 30 June to 30 August 2018: 12,927 orders, 12,855 of them reviewed, 12,630 delivered) unless the row or its slide's heading names another window. *Assumed* numbers have no data source: Olist publishes no costs, so they are inputs we chose, and Appendix Q shows what changes if they are wrong. *Derived* numbers are arithmetic on the other two, with the formula given. *Planned* numbers are design choices: a pilot that has not been run, or the size of a recommended list. Three further kinds carry no number to check: a claim taken from the *literature*, a claim with *no data source* that the tables can neither support nor refute, and *wording* whose reading needs correcting. Of the 105 rows over the 13 slides, 88 carry a number that `src/slide_provenance.py` recomputes from the repository's outputs into `report/results/slide_provenance.csv`. All 88 match the slide to half a unit of its last digit, except three figures the slides rounded loosely (100,000 orders, 75% and the 28 of 27–28%), which match within the wider tolerance recorded in the CSV; the value after the arrow is the recomputed one. The script stops if any no longer matches, and a validation check fails if the CSV records a mismatch or a cost that is not labelled as assumed.

Table 30: Where the numbers on the slides come from: every number or claim, slide 11 first, with its kind, its source or formula (→ the value recomputed from it) and the reading this report stands by. Rows count the test window unless the slide's heading or a bracket names another. Generated by `src/slide_provenance.py` from `report/results/slide_provenance.csv`.

On the slide	Kind	Source or formula	Reading or correction
Slide 11: business value and the pilot			
"Worth R$775–R$1,466 / 1k orders"	derived	review_timing_value.csv, threshold 0.11: net_reachable_per_1k, net_all_per_1k → 775.2; 1,466.0	per 1,000 reviewed test orders, not orders; ceilings that count every contact as turning its customer around (at one in four: −R$368 and −R$196, customer_value.csv)
"R$5 CALL"	assumed	config.COSTS['outreach_cost'] → 5	no data source; varied R$2.5–10 in customer_value_sensitivity.csv
"R$40 BAD REVIEW"	assumed	config.COSTS['low_review_cost'] → 40	stands for a 1–2⋆ rating's effect on later buyers; the customer's own return cannot price it (repeat within 180 days: 1.50% after 1–2⋆, 1.90% after 4–5⋆, customer_repeat.csv); varied R$20–80
"R$775 conservative ceiling on the 35/100 share"	derived	1000 × (490 × 40 − 1,927 × 5) / 12,855: the threshold-0.11 list's 490 hits not written before arrival (282 delivered + 208 never delivered) → 775.2	belongs to the threshold-0.11 list (31 contacts a day), not to the 35-in-100 share, which is worth R$433

Table 30, continued

On the slide	Kind	Source or formula	Reading or correction
"R$433 at fixed 10 calls/day"	derived	1000 × (216 × 40 − 614 × 5) / 12,855 → 433.3	of the 35 in 100, 26.4 never received the parcel; only 8.8 could be reached at delivery (that call alone: −R$71)
"35/100 share"	observed	review_timing_capacity.csv, 10 a day: 216 of 614 = 162 never delivered + 54 delivered, not yet written → 35.2	includes the orders that never arrived
"R$1,466 full ceiling across all hits"	derived	cost_p2.csv, threshold 0.11: 1000 × (712 × 40 − 1,927 × 5) / 12,855 → 1,466.0	0.11 is the best of 91 thresholds read on the test window; the threshold fixed on validation, 0.19, gives R$1,373 (R$692 not written); 60.4% of the gross value is never-delivered orders or late parcels rated before arrival
"50 / 50 for 6 weeks (191 orders per arm)"	planned	pilot_design.csv (k 10, 25%): weeks = ceil(2 × 191 / 10 / 7); n_per_arm_80 → 6; 191	ten a day on every calendar day (8 weeks on weekdays); re-sized per contact in customer_pilot_arms.csv: message 209 per arm, 5 weeks; delivery-day call 159, 5 weeks
"5 RANDOMLY CALLED / DAY" … "5 HELD AS CONTROL / DAY"	planned	1:1 randomisation inside the daily top ten → 5; 5	as on the slide
"TODAY (NO CALL) / 57.2%"	observed	capacity_p2_daily.csv (k 10): 351 of 614 → 57.17	a historical rate, not a control group (no pilot ran): 26.4 points never received the parcel and 22.0 had rated a late parcel while waiting
"BREAK-EVEN (∼1 IN 5) / ≤ 44.7%"	derived	pilot_breakeven.csv: 0.572 / 0.125; 0.572 − 5/40 → 4.6; 44.67	21.9% (1 in 4.6); unrelated to the 44.7% of customers the 11% late warning reaches
"6-WK TARGET (1 IN 4) / 42.9% / 80% power"	planned	pilot_design.csv: 0.572 × (1 − 0.25); power 0.80, alpha 0.05 → 42.87; 80	one in four is a design choice, not an estimate
"+R$0.72/call"	derived	(0.572 − 0.429) × 40 − 5 → 0.717	only if the one-in-four effect and both costs hold; the ten-a-day list then nets R$34 per 1,000 reviewed orders
"Nobody in the 2016–2018 log"	no data source	none	the slide's statement about calls has no source: the tables record no contact other than the survey, so only a randomised pilot can measure what a contact changes
"PLUS ARM B (PROMISED-DATE OVERDUE ORDERS)"	planned	customer_pilot_arms.csv	13.0 customers a day rate the order only after the promised day ends without a parcel, 54.7% of them 1–2⋆: 209 per arm, 5 weeks (two arms) or 7 (three); outcome: the final rating
"LIVE VALIDATION" … "Historical data proves"	wording	none	nothing was validated live: the held-out days show; the pilot would measure

Table 30, continued

On the slide	Kind	Source or formula	Reading or correction
Slide 1: title			
"100,000 Brazilian e-commerce orders" … "99,441 orders (2016–2018)"	observed	orders table: rows, purchase years → 99,441; 99,441; 2016; 2018 [whole extract, 2016–18]	the analysis uses the 99,091 orders placed January 2017 to August 2018
"3. Delivery / Call top-10 risk list"	planned	customer_contact_lists.csv (delivery_day_k10) → 10	worked on the previous day's deliveries not yet rated: 26.0 in 100 rate 1–2⋆ (3.2× the 8.1% base)
Slide 2: one customer's order *(rows: order e1320973, unless bracketed)*			
"16 Jul 2018 / Day 0" … "7 Aug 2018 / Day 22" … "9 Aug 2018 / Day 24" … "11 Aug 2018 / Day 26" … "17 Aug 2018 / Day 32"	observed	customer_story.csv: purchase, promised, survey_sent, review_answered, delivered → 16 Jul 2018; 0; 7 Aug 2018; 22; 9 Aug 2018; 24; 11 Aug 2018; 26; 17 Aug 2018; 32	as on the slide
"Order #e1320973 · Health & beauty item (R$65 + R$28 freight)"	observed	customer_story.csv: order_id, category, price, freight → e1320973; health & beauty; 65.0; 27.7	R$64.99 + R$27.74
"2,267 km from São Luís (MA) to São Gonçalo (RJ)"	observed	customer_story.csv: distance_km (postcode-prefix centroids), seller and customer city → 2,266.9; sao luis (MA); sao goncalo (RJ)	as on the slide
"(∼21-day window)" … "(10.5 days late)" … "5.8 days old" … "took 31.8 days"	observed	customer_story.csv: promised_days, days_late, review_days_before_arrival, actual_days → 21.3; 10.50; 5.77; 31.79	as on the slide
"Zero proactive warning sent to the buyer."	no data source	none	the tables record no message and no contact other than the survey
"Triggered by the promised calendar date"	no data source	customer_story.csv; customer_survey_timing.csv	the trigger is not recorded; the survey went out 2 days after the promised date, 8.5 days before delivery (test window: 71.4% of late 1–2⋆ parcels exactly 2 days after)
"Frustrated and still empty-handed"	wording	customer_story.csv	rests on the timestamps (answered 11 Aug 17:33, delivered 17 Aug 11:55); the comment ('Bom' / 'Não veio a embalagem certa') is ambiguous and may mean the packaging
"the damage on the product page is already done"	no data source	none	the tables do not record what later buyers saw
Slide 3: broken promises and ratings *(rows: 2017–18, delivered and reviewed, unless bracketed)*			
"ACROSS 95,561 DELIVERED ORDERS"	observed	review_timing.csv (all): n → 95,561	delivered and reviewed orders
"DELIVERED ON TIME / 9.2%" … "87,902 orders (92.0%)"	observed	review_timing.csv (on time): pct_low_review, n, share → 9.20; 87,902; 91.99	as on the slide

Table 30, continued

On the slide	Kind	Source or formula	Reading or correction
"DELIVERED LATE / 54.0%" … "7,659 orders (8.0%)"	observed	review_timing.csv (late): pct_low_review, n, share → 54.00; 7,659; 8.01	as on the slide
"5.9×"	derived	54.0 / 9.2 → 5.87	as on the slide
"78.6%" … "78.0%" … "61.3%" … "19.1%" … "9.7%" … "8.9%"	observed	review_timing_lateness.csv: low_pct by days late or early → 78.63; 77.99; 61.31; 19.13; 9.68; 8.93	as on the slide
"1–3 days late doubles bad reviews (19.1%); 4+ days late jumps past 61%–79%"	derived	19.1 / 9.2; review_timing_lateness.csv → 2.1; 19.13; 61.3; 78.6	by day: 12.3% during the promised day, 19.8% one to two days late (customer_lateness_days.csv)
"Only 1 in 12 parcels (8.0%) arrives late" … "34% (one-third) of every 1–2★ review"	observed	review_timing.csv: late share; share_of_all_low_reviews_pct → 12.5; 8.01; 33.8	of the 1–2★ reviews on delivered orders
"On Olist's own data, bad logistics reviews directly depress a seller's future sales"	literature	refs.bib deshpande2023logistics	the study is of another marketplace, not Olist; seller sales were not measured here
Slide 4: when the review is written *(rows: 2017–18, delivered and reviewed, unless bracketed)*			
"ON-TIME PARCELS (11 IN 12)" … "LATE PARCELS (1 IN 12)"	observed	review_timing.csv: 12 × each share → 11.0; 1.0	as on the slide
"+2.1 days / after delivery scan"	observed	review_timing.csv (low review, on time): median days from delivery to the review → 2.12	on-time parcels rated 1–2★
"66% of all 1–2★ reviews / Wrong, damaged, or disappointing items" … "99.5% review AFTER delivery"	observed	review_timing.csv (low review, on time): share of 1–2★, written after arrival → 66.2; 99.51	not measured on 4 November; since: of on-time 1–2★ comments 34.0% mention a wrong or missing item, 15.0% damage or a defect, 16.9% quality (keyword topics, customer_complaint_topics.csv)
"1★ written 4.5d early!"	observed	review_timing.csv (low review, late): median days from the review to delivery → 4.50	all late 1–2★, not only 1★
"ALREADY WRITTEN / 87.3%"	observed	review_timing.csv (low review, late): pct_written_before_arrival → 87.31	77.4% on the test window
"34% of all 1–2★ reviews / Survey fires at the promised date"	observed	review_timing.csv (low review, late); customer_survey_timing.csv → 33.8	survey on or after the promised date for 99.5%, a median 2 days after; the rule is not recorded
Slide 5: why promises break *(rows: monthly, 2017–18, unless bracketed)*			
"sped up 2×" … "16.9 d → 7.7 d average delivery"	observed	lateness_monthly.csv mean_lead, Feb and Aug 2018 → 2.2; 16.95; 7.73	delivery time halved
"75% of total time lives with the carrier"	observed	orders table: mean (delivered − carrier handover) / mean lead time → 74.5 [2017–18, delivered]	74.5% (9.3 of 12.5 days), rounded up
"across 20 months (Jan 2017 – Aug 2018)"	observed	lateness_monthly.csv rows → 20	as on the slide

Table 30, continued

On the slide	Kind	Source or formula	Reading or correction
"Carrier jam peak 21.4%" … "Black Friday 14.3%" … "16.0%" … "10.4%"	observed	lateness_monthly.csv late_pct: Mar 2018, Nov 2017, Feb 2018, Aug 2018 → 21.36; 14.31; 16.00; 10.39	as on the slide
"∼11 extra buffer days on average"	observed	orders table: mean promised days − mean lead time → 11.1 [2017–18, delivered]	23.6 − 12.5; test window 9.1 (17.48 − 8.34)
"(9.8d error)"	observed	reg_p1a.csv: Olist's own delivery promise, MAE → 9.79 [test window]	a test-window figure; the ∼11 days are 2017–18
"missed 1 in 5 orders (21.4%) when regional carrier slowdowns struck"	observed	lateness_monthly.csv (Mar 2018); lateness_peak_by_region.csv → 4.7; 21.36	network-wide: every region 17.0–30.5% late in Feb–Mar 2018 (5.3–11.8% otherwise)
"inflates validation AUC from 0.59 to a fake 1.00"	observed	report/stats.txt: c1b_logreg_roc, c1b_leaked_roc → 0.589; 1.000 [test window]	a test-window score of the Phase 1 logistic regression, not a validation AUC
"unseen Jul–Aug 2018 orders"	wording	config.VAL_END, config.ANALYSIS_END	orders placed 30 June to 30 August 2018: 12,927 (12,855 reviewed, 12,630 delivered)
"(19.5 → 36.8 days), hiding June lateness (1.4%)"	observed	lateness_weekly.csv promise_days, weeks of 14 and 28 May 2018; lateness_monthly.csv, June 2018 → 19.49; 36.80; 1.36 [May–June 2018]	weekly means; 'hiding' is a reading
Slide 6: three moments, three tools *(rows: 2017–18, delivered and reviewed, unless bracketed)*			
"12.8% / (∼1 in 8 delivered orders)" … "14.6% across all reviewed orders including undelivered"	observed	review_timing.csv (all): pct_low_review; 14,365 / 98,329 reviewed (customer_complaint_topics.csv, train_val_test.csv) → 12.79; 7.8; 14.61 [2017–18, reviewed]	as on the slide
"a single decision tree captures 96% of the delivery-triage signal"	derived	clf_p2.csv PR-AUC: tree / deployed stack → 95.7 [test window]	review risk, not delivery: 0.396 against 0.414
"Replaces padded 24-day guesses"	observed	orders table: mean promised days → 23.6 [2017–18, delivered]	23.6 days (2017–18), 17.5 on test; the recommended date is the forecast + 8 days (18.6 on average)
"pauses the review email before 87.3% of late 1–2⋆ are written"	observed	as slide 4 → 87.31	recommended: ask for the review once the order is complete; every customer is still asked; no review is filtered, delayed in publication or rewarded
"Triggers automatically the instant a promised date passes"	wording	customer_outcomes_test.csv; customer_lateness_days.csv	the trigger waits for the end of the promised day: 335 of 936 late parcels arrived during it, rated like on-time ones (12.3% 1–2⋆, against 11.1% a day earlier)
"∼2.1-day post-delivery window"	observed	as slide 4 → 2.12	as on the slide

Table 30, continued

On the slide	Kind	Source or formula	Reading or correction
Slide 7: results at checkout			
"Tested on 12,630 unseen future deliveries"	observed	predictions_p1a_test.csv rows → 12,630	delivered test orders
"cuts date error by 60%" … "9.79 days" … "3.90 days (beats GBDT 4.14d & Linear 4.18d)"	derived	reg_p1a.csv MAE; 1 − 3.90 / 9.79 → 60.2; 9.788; 3.898; 4.141; 4.181	as on the slide
"Costs R$0 to quote"	assumed	none	the forecast on its own is exceeded by 31.9% of deliveries (the platform's date by 7.4%); forecast + 8 days: 2.5%, 1.1 days longer (customer_promise_ rules.csv)
"Pads ∼11 extra buffer days"	observed	as slide 5 → 11.1 [2017–18, delivered]	9.1 on the test window
"lifts PR-AUC 2.9× vs 1.2× (ROC-AUC 0.782 vs 0.584)"	observed	clf_p1b.csv: derived route vs HistGradientBoostingClassifier → 2.89; 1.24; 0.7820; 0.5838	as on the slide
"35% of late orders (3.5×)"	observed	capacity_p1b.csv (derived, top 10%): recall, lift → 34.8; 3.48	25.8% of that tenth were late
"#23 of 238 delivered orders on 16 July"	observed	customer_story.csv (p_late_ derived rank) → 23; 238 [16 July 2018 purchases]	as on the slide
"≥ 11% risk: Pre-warns 83% (5 in 6)"	observed	capacity_operating_points.csv (P1B, 0.11): recall → 11; 82.6; 5.0	reaches 44.7% of customers; 86.3% of them then on time (773 late, 4,874 on time)
"R$12 express upgrade vs. R$40 bad-review cost"	assumed	config.COSTS['expedite_cost'], ['late_delivery_cost'] → 12; 40	R$40 is charged for every late parcel
"27–28% precision < 30% break-even"	observed	capacity_p1b.csv precision, top 1–7.5%; 12 / 40 → 26.8; 27.4; 30	26.8–27.4% (25.8% at the top 10%)
"at least 30% (12 / 40)"	derived	12 / 40 → 30	conservative: with the observed 1–2⋆ gap (54.0% − 9.2%) a late parcel costs R$17.9, break-even 0.67
"75% of transit happens after checkout inside the carrier network"	observed	as slide 5 → 74.5 [2017–18, delivered]	74.5%, after the seller's handover
Slide 8: results at delivery			
"Across 62 unseen test days (12,855 reviewed orders)"	observed	predictions_p2_test.csv: purchase days, rows → 62; 12,855	as on the slide
"turns 11% random hits into 57%" … "(57.2%)" … "(11.0% base)" … "89 of 100 calls wasted on happy buyers"	observed	capacity_p2_daily.csv (k 10); review_timing_capacity.csv base_rate → 11.0; 57.2; 57.17; 11.03; 89.0	a historical outcome (no contact recorded besides the survey) of a purchase-day list scored with the realised delivery; deliveries not yet rated: 26.0 per 100; 3–5⋆ is not all 'happy'
"5.2× at 10 calls/day"	derived	0.572 / 0.110 → 5.18	as on the slide

Table 30, continued

On the slide	Kind	Source or formula	Reading or correction
"(68% at top 5%)"	observed	capacity_p2.csv (top 5%): precision → 67.8	43.1% of those 643 never arrived; 31.6 in 100 had rated before the parcel arrived; 4.8 in 100 could be reached at delivery
"Powered by delay vs. promise, multi-unit baskets, high freight share & category history."	observed	perm_p2.csv	delay vs promise 0.206, days 0.074, items 0.045; freight share and category history rank 32 and 35 of 36
"still hits 45.1% unhappy buyers (vs 9.6% at random)"	observed	review_timing_composition.csv (delivered only) → 45.08; 9.63	28.9 of the 45.1 had rated before their late parcel arrived; 16.2 per 100 not yet rated at delivery
"top 5% = 28% (3.5×), where 99.5% wait ∼2.1 days after delivery"	observed	review_timing_subsets.csv (on time); review_timing.csv → 28.0; 3.53; 99.51; 2.12	99.5% and 2.1 days are 2017–18 figures
"Only 13 hits / 100 needed"	derived	100 × 5 / 40 → 12.5	12.5 if every contact works; 12.5 / s if a share s does
"57 total hits (and 35 non-pre-empted) clear the 12.5% bar"	observed	review_timing_capacity.csv (10 a day) → 57.2; 35.2; 12.50	26.4 of the 35 never received the parcel; a call at delivery reaches 8.8, below 12.5
"35 not written before arrival" … "22 wrote waiting" … "43 satisfied (3–5⋆)" … "614 total calls over 62 days"	observed	review_timing_capacity.csv (10 a day) → 35.2; 22.0; 42.8; 614; 62	list entries, not calls; 3–5⋆ is not all 'satisfied'
Slide 9: the app and 16 July 2018 *(rows: 16 July 2018 purchases, unless bracketed)*			
"8-tab Streamlit app" … "replay any of 62 test days"	observed	predictions_p2_test.csv purchase days → 62 [test window]	eight tabs when the slides were locked on 4 November 2026
"16 JULY 2018 (244 REVIEWED ORDERS)" … "8 of 10 real 1–2⋆" … "(vs ∼1 by random)"	observed	customer_story.csv; 10 × the day's 1–2⋆ rate → 244; 8; 10; 1.3	a purchase-day list scored with the realised delivery: all 8 reachable only when the promised day ended (4 never delivered, 4 rated while waiting); 0 at delivery
"Checkout error: 4.3 d vs Olist's 10.7 d"	observed	predictions_p1a_test.csv, 16 July: MAE of forecast and promise → 4.35; 10.72	the day's 238 delivered orders
"4 buyers wrote 1⋆ while still waiting for a late parcel"	observed	customer_journeys_test.csv, 16 July top ten → 4	as on the slide
"4 buyers never received their parcel at all (shipped, invoiced or cancelled)"	observed	customer_journeys_test.csv, 16 July top ten: order_status → 4	2 shipped, 1 invoiced, 1 cancelled
"57 / 100 average precision"	observed	capacity_p2_daily.csv (k 10) → 57.2 [test window]	351 of 614, pooled
"(85% RISK)" … "Flagged #23 of 238 delivered at checkout, and #1 of 244 reviewed on the call list"	observed	customer_story.csv → 84.7; 23; 238; 1; 244	'that morning's list' was scored once the delivery was known (17 August); at checkout its late risk was 18.2%, above 11%

Table 30, continued

On the slide	Kind	Source or formula	Reading or correction
Slide 10: the recommendations			
"Replace padded promises with our checkout forecast (60% smaller error: 9.79 → 3.90 d)"	derived	reg_p1a.csv MAE → 60.2; 9.788; 3.898	the forecast on its own is exceeded by 31.9% of deliveries (the platform's date by 7.4%): recommended, forecast + 8 days (2.5%)
"banner above 11% risk — pre-warning 83% (5 in 6)"	observed	as slide 7 → 11; 82.6; 5.0	reaches 44.7%, 86.3% then on time; not needed with the forecast-plus-margin date
"closing the window behind 87.3% of late 1–2★"	observed	as slide 4 → 87.31 [2017–18, delivered and reviewed]	the effect on ratings is for the pilot's third arm to measure; every customer is still asked; no review is filtered, delayed in publication or rewarded
"Cost: ∼R$0 · Immediate trust gain"	assumed	none	an assumed cost; the trust gain is a hypothesis
"Stops pre-arrival 1★"	wording	none	by construction; the effect on the final rating is unmeasured
"top ∼10 orders each morning (57% precision = 5.2× random; 45% on delivered parcels alone)" … "still wait ∼2.1 days"	observed	as slides 4 and 8 → 10; 57.2; 5.18; 45.1; 2.12	as a team would work it: 26.0 per 100 (3.2× the 8.1% base)
"Checkout-only precision plateaus at 27–28% across top-K tiers — below the 30% break-even"	observed	as slide 7 → 26.8; 27.4; 30	26.8–27.4%; 30% is conservative
"Cost: ∼R$5/call"	assumed	config.COSTS['outreach_cost'] → 5	as on the slide
"Refit monthly on a rolling 12-month window (because transit speed halved in 6 months)"	wording	lateness_monthly.csv mean_lead	the 12-month window was not tested (expanding windows were); delivery time halved (16.9 → 7.7 days)
Slide 12: lessons			
"squeezing 0.001 AUC from a complex ensemble"	observed	clf_p2.csv PR-AUC: boosted trees − stack → 0.0011	PR-AUC; the deployed stack is 0.0011 below
"a fake 1.00 AUC"	observed	as slide 5 → 1.000	as on the slide
"87.3% of late-order bad reviews were written before the parcel even arrived"	observed	as slide 4 → 87.31 [2017–18, delivered and reviewed]	as on the slide
"27–28% (< 30% rush break-even)"	observed	as slide 7 → 26.8; 27.4; 30	as on the slide
"how to run the 6-week pilot"	planned	pilot_design.csv (k 10, 25%) → 6	per contact: 5 weeks (message, two arms; 7 with three), 5 (delivery-day call)
"75% of delivery time happens inside third-party carriers"	observed	as slide 5 → 74.5 [2017–18, delivered]	74.5%

Table 30, continued

On the slide	Kind	Source or formula	Reading or correction
"Northeast / North routes face structural distance disadvantages."	observed	segments_p1a_region.csv	mean delivery 15.6 (North), 13.4 (Northeast), 7.2 days (Southeast); forecast error 6.8, 6.3, 3.4
Slide 13: closing and demo			
"60% closer date" … "Top-10 calls (5.2×)"	derived	as slides 7 and 8 → 60.2; 5.18	as on the slide
"Saved before 1⋆!"	wording	none	means reaching the customer before they rate the order; every customer is still asked; no review is filtered, delayed in publication or rewarded
"8 of 10 real 1–2⋆ (#1 = São Gonçalo!)"	observed	as slide 9 → 8; 10; 1 [16 July 2018 purchases]	as on the slide
"What-if cost & pilot planner" … "Test R$5 call vs R$40 review & 6-wk RCT"	assumed	config.COSTS; pilot_design.csv weeks_80 → 5; 40; 6	both costs are assumptions (customer_value_sensitivity.csv varies them); pilot sizes per contact: customer_pilot_arms.csv

The deck's vocabulary. Some of the deck's words are not the report's, and some say more than the data show. The adjective the slides attach to the team's dates, test, tools, findings and caveats (slides 1, 5–7 and 10–13) is not used here. A date from the model means the forecast plus a margin chosen on validation (Section 9.2), which the slides did not yet have; the test is the out-of-time test window; a finding or a caveat is a stated result or limitation; and the platform's own date is described as what it is, a commercial choice, padded because a broken promise costs returns and retention [9]. The verbs the slides use for what a list or a flag does with an unhappy customer or a late parcel (slides 1, 6, 7, 9, 10 and 13) mean only that the list contains them: precision and recall count customers who went on to rate the order 1–2 stars and parcels that arrived late, not ratings or deliveries that were changed. "Pause the review email" and "hold the review survey until actual arrival" (slides 6, 10, 11 and 13) and "Saved before 1⋆!" (slide 13) mean reaching the customer before they rate the order and asking for the review once the order is complete; every customer is still asked, and no review is filtered, delayed in publication or rewarded. "Live validation" and "proves" (slide 11) mean that the held-out days show a pattern; only the pilot of Section 9.4 would measure what a contact changes.

Q The customer's view: the tables behind Sections 4 and 9

Everything here is computed by `src/customer_view.py` from the nine tables, the saved test predictions and the deployed lead-time model, scored on the validation and test windows exactly as `src/train_models.py` scored them. Nothing is refitted: the rescored test forecasts equal `predictions_p1a_test.csv` to 10^{-9}, an assertion in the script. The *test window* is the orders placed 30 June to 30 August 2018; the *analysis window*, the orders placed 1 January 2017 to 30 August 2018. Every figure is descriptive; the repeat-purchase comparisons are also exploratory.

A date the customer can rely on. Table 31 compares the platform's promised date with dates built from the forecast. The margin of Section 9.2 is chosen on the validation months alone: the smallest margin, in steps of 0.1 day, whose broken-promise rate there is no higher than the platform's own (4.97%), with the date rounded up to a calendar day — eight days. Two margins are shown for reference: the one that matches the platform's average quoted wait on the test window (6.9 days), chosen from quoted dates and forecasts only, never from outcomes, and the one that matches it on the validation months (12.4 days), which pads far more. *Broken* uses the report's definition of late (delivered after the promised date begins) for every date; the calendar-day column counts only deliveries after the promised day has ended.

Table 31: Promised dates on the validation and test windows, from `customer_promise_rules.csv`. Broken: share of delivered orders that arrived after the date. Let down: broken promises among the 12,630 delivered test orders. Calendar day: arrived after the promised day ended. Mean wait: the average quoted wait, in days. Days late: mean, among broken promises. The validation rule picks the margin; the two other margins match the platform's average quoted wait on the test and on the validation window. "Forecast alone" is the forecast itself, not rounded up to a calendar date; rounded up, as every margin is, it is broken for 25.1% of test orders (`customer_promise_frontier.csv`).

	validation	test				
Date shown to the customer	**broken**	**broken**	**let down**	**calendar day**	**mean wait**	**days late**
Platform's own promised date	4.97%	7.41%	936	4.76%	17.5	4.4
Forecast alone	26.28%	31.85%	4,023	22.37%	10.1	3.4
Forecast + 8 days (validation rule)	4.86%	**2.52%**	318	2.11%	18.6	6.8
Forecast + 6.9 days (platform's wait, test)	6.03%	3.20%	404	2.58%	17.5	6.4
Forecast + 12.4 days (platform's wait, val.)	2.55%	1.10%	139	0.92%	23.0	8.5

By region the margin moves the padding to where the risk is (`customer_promise_region.csv`). Southeast customers (8,951 test orders) are let down 8.59% of the time by the platform's date and 2.41% by the forecast plus eight days, for an average wait quoted 1.4 days longer; in the North and Northeast (1,312) the rate falls from 5.03% to 3.58% at the same average wait (26.6 days); in the South and Central-West (2,367) from 4.27% to 2.32%, for 0.6 days more. Overall 61.5% of customers would see a later date than today and 31.3% an earlier one, by at least a day, and the dates still missed are missed by more: 6.8 days on average, against 4.4. Three cautions. The validation months include the May 2018 strike and June's padded promises, so the margin over-delivers on the test window: 2.52% broken, against 7.41% for the platform's date there and the 4.97% bar it was chosen to meet on validation. The forecast over-predicts by 1.74 days on the test

window and the margin absorbs that bias, so it must be re-chosen at every refit. And the date assumes that delivery times and customers do not change because the quote does; the forecast also takes the platform's own estimate as an input, so the platform would keep computing that estimate and change only the date it shows.

What happened to every 1,000 orders. Table 32 is the full count behind Table 9, with 95% Wilson intervals (`customer_outcomes_test.csv`). Every row names its population: orders placed in the test window (12,927) or, for the checkout warning, delivered test orders (12,630).

Table 32: What happened to the customers of the test window, from `customer_outcomes_test.csv`. Per 1,000 orders of the population named in the third column, with 95% Wilson intervals. Indented rows are subsets of the row above them. "Reachable" names the contact that could still have come before the customer rated the order (Appendix M).

Customers whose order…	orders	of	per 1,000	95% interval
was placed (purchases 30 June to 30 August 2018)	12,927	placed	1,000.0	—
parcel arrived by the promised date	11,694	placed	904.6	[899.4, 909.6]
parcel arrived after the promised date began (the report's definition)	936	placed	72.4	[68.1, 77.0]
during the promised day itself	335	placed	25.9	[23.3, 28.8]
after the promised day ended	601	placed	46.5	[43.0, 50.3]
parcel never delivered within the extract	297	placed	23.0	[20.5, 25.7]
order cancelled or unavailable	149	placed	11.5	[9.8, 13.5]
no parcel when the promised day ended	898	placed	69.5	[65.2, 74.0]
of whom never received it	297	placed	23.0	[20.5, 25.7]
rated the order 1–2 stars	1,418	placed	109.7	[104.4, 115.2]
parcel on time	923	placed	71.4	[67.1, 76.0]
parcel late	287	placed	22.2	[19.8, 24.9]
parcel never delivered	208	placed	16.1	[14.1, 18.4]
late parcel, 1–2 stars written before it arrived	222	placed	17.2	[15.1, 19.6]
no parcel when the promised day ended, not yet rated then, went on to rate 1–2 stars	440	placed	34.0	[31.0, 37.3]
rated 1–2 stars; reachable only by the promised-date message	398	placed	30.8	[27.9, 33.9]
rated 1–2 stars; reachable only by a call at delivery	945	placed	73.1	[68.7, 77.7]
rated 1–2 stars; reachable by either contact	42	placed	3.2	[2.4, 4.4]
rated 1–2 stars before either moment	33	placed	2.6	[1.8, 3.6]
on the slides' daily top-ten list (grouped by purchase day)	614	placed	47.5	[44.0, 51.3]
and rated the order 1–2 stars	351	placed	27.2	[24.5, 30.1]
of whom not yet rated when the promised day ended (no parcel then)	271	placed	21.0	[18.6, 23.6]
of whom reachable only by a call at delivery	48	placed	3.7	[2.8, 4.9]
flagged at checkout (late risk at or above the 11% line)	5,647	delivered	447.1	[438.5, 455.8]
parcel then late	773	delivered	61.2	[57.2, 65.5]
late parcel, not flagged	163	delivered	12.9	[11.1, 15.0]
flagged, parcel then on time	4,874	delivered	385.9	[377.5, 394.4]

A parcel delivered during the promised day counts as late under the report's definition: 335 of the 936 late test parcels. Their customers rate them much like on-time ones (Table 33, analysis window): 12.3% gave 1–2 stars, against 11.1% for delivery during the day before and 19.8% for delivery a day later. That is why the promised-date message is triggered when the promised day ends, not when it begins.

Table 33: The 1–2 star rate by how early or late the parcel arrived against the promised date, on the delivered and reviewed orders of the analysis window, with 95% Wilson intervals (`customer_lateness_days.csv`). The promised date is a calendar date, so "during the promised day" is up to one day after it began.

Parcel arrived	orders	1–2 stars	95% interval
7 or more days early	70,673	8.9%	[8.7, 9.2]
3–7 days early	12,527	9.8%	[9.3, 10.4]
1–3 days early	3,253	11.3%	[10.2, 12.4]
up to 1 day early	1,449	11.1%	[9.6, 12.8]
during the promised day	1,280	12.3%	[10.7, 14.3]
1–2 days late	819	19.8%	[17.2, 22.6]
2–3 days late	536	34.3%	[30.4, 38.4]
3–7 days late	1,773	61.3%	[59.0, 63.5]
more than 7 days late	3,251	78.3%	[76.8, 79.7]

What unhappy customers wrote. The comments of the 1–2 star reviews of the analysis window (the review the repository keeps per order, title and message together, lower-cased, accents stripped and common abbreviations expanded) are matched against seven overlapping topics, each a regular expression in `src/customer_view.py`, listed in `customer_complaint_dictionary.csv`. The patterns were frozen before their precision was measured on a random sample of 25 matched comments per topic, each comment read and marked right or wrong by the AI assistant used for the revision (AI Tool Declaration); `customer_topic_labels.csv` records the 175 verdicts, with a note on each miss, so any of them can be re-read. Table 34 gives the shares. They are approximate: 18.9% of comments match no topic, often a paraphrase the patterns miss, and 4–20% of the sampled matches were wrong (the precision column). The comments separate two failures. Customers whose parcel was late or never came write that they do not have it (59.9% and 58.3%), and those whose order never came also write about refunds and cancellation (24.5%); customers whose parcel came on time write about what was in it: a wrong or missing item (34.0%), quality or a product unlike its description (16.9%), damage or a defect (15.0%). Comments are written with the rating, so they are read here and never used as a feature.

Table 34: Topics of 1–2 star comments on the analysis window, as % of the comments in each group (a comment can match several topics), and the held-out precision of each pattern with its 95% Wilson interval. From `customer_complaint_topics.csv` and `customer_complaint_precision.csv`.

	the parcel was				
Topic	**any**	**on time**	**late**	**never delivered**	**precision**
1–2 star reviews	14,365	8,084	4,136	2,145	
with a comment	10,826	6,385	2,889	1,552	
not received (yet)	34.5	17.3	59.9	58.3	88 [70, 96]
late, slow, missed date	12.4	5.4	24.6	18.2	96 [80, 99]
wrong or missing item	22.2	34.0	5.5	4.3	88 [70, 96]
defective or damaged	9.5	15.0	1.7	1.8	92 [75, 98]
quality, not as described	10.6	16.9	2.1	0.5	96 [80, 99]
seller service, communication	13.9	12.0	16.3	17.5	80 [61, 91]
refund, cancel, return	14.2	14.8	7.4	24.5	96 [80, 99]
no topic matched	18.9	20.2	17.0	17.5	—

Orders with several items or sellers. On the 87,902 on-time, delivered and reviewed orders of the analysis window (Table 35), a single item from one seller draws 1–2 stars

7.5% of the time, two or more items from one seller 21.0%, and two or more sellers 47.1%. Multi-item orders are 10.1% of on-time orders but 27.1% of their 1–2 star reviews, and 74.9% and 84.2% of those reviewers who comment say that part of the order was wrong, missing or never came. Two limits: `n_items` counts item rows, so two units of one product count as two; and an order carries one delivery timestamp, so "on time" does not mean that every package arrived. The finding is an association, not a mechanism.

Table 35: On-time, delivered and reviewed orders of the analysis window by basket, from `customer_basket.csv`: the 1–2 star rate with its 95% Wilson interval, the basket's share of all on-time 1–2 star reviews, and what the reviewers who commented wrote (the topics of Table 34; the first combines "wrong or missing item" and "not received").

	1 item, 1 seller	**2+ items, 1 seller**	**2+ sellers**	**all on time**
Orders	79,011	7,650	1,241	87,902
share of on-time orders	89.9%	8.7%	1.4%	100.0%
Rated 1–2 stars	7.5%	21.0%	47.1%	9.2%
95% interval	[7.3, 7.6]	[20.1, 21.9]	[44.4, 49.9]	[9.0, 9.4]
share of all on-time 1–2 star reviews	72.9%	19.9%	7.2%	100.0%
1–2 star reviews with a comment	4,482	1,403	500	6,385
say part of the order was wrong, missing or never came	33.0%	74.9%	84.2%	46.2%
say it was defective or damaged	19.3%	5.5%	2.4%	15.0%
say its quality was poor or not as described	21.7%	7.0%	2.4%	16.9%

Do let-down customers come back less? Each customer (`customer_unique_id`) is followed from their first order. A repeat is another order placed after the first was delivered and within the horizon of that delivery; the window must close by 22 August 2018, after which the extract thins out (`customer_repeat.csv`, Table 36). Of the 50,360 first-time buyers that leaves at 180 days, 1.84% came back. After a late first parcel 1.44% did, against 1.86% after an on-time one; after a 1–2 star first review 1.50%, against 1.90% after 4–5 stars. The direction is the same at every horizon, and some differences are nominally significant (after a 1–2 star review at 90 days, -0.36 points, $p = 0.003$), but they are tenths of a point on repeat rates of 0.56% to 3.21%, close to the smallest difference this sample can detect, and 28 comparisons were run over four horizons and two definitions of a repeat. They are exploratory. A bad experience therefore cannot be priced through the customer's own return, which is why the R$40 of Section 9.3 is an assumption standing for the rating's effect on later buyers.

The value of a contact under other assumptions. Table 37 restates each list in reais per 1,000 reviewed test orders (12,855) at the assumed R$5 a contact and R$40 a 1–2 star rating, for three shares of the reached customers a contact turns around: all, half and a quarter. Nothing in the data measures that share. `customer_value_sensitivity.csv` also varies the contact cost (R$2.5, 5, 10) and the rating's value (R$20, 40, 80); only the share times that value counts, so the "half" column is also every reached customer turned around at R$20 a rating. Read the other way, at R$5 a contact the promised-date message pays at full effect if a 1–2 star rating is worth at least R$9.1, the delivery-day call R$19.2 (R$36.5 and R$76.9 at one in four; `customer_value.csv`). The first panel counts every hit, as the slides did, including customers who had already rated the order or whose

Table 36: Repeat purchase by the first order's experience, from `customer_repeat.csv`: first-time buyers whose first order was delivered at least the horizon before 22 August 2018; a repeat is an order placed after that delivery and within the horizon. Rates in % ("all": every buyer of the cohort); the difference is in percentage points with a 95% Newcombe interval; the odds ratio is Mantel–Haenszel, stratified by customer region and purchase quarter, with its 95% interval. The other definition (any later order on a later day, counted from purchase) is in the CSV.

days	buyers	all	group vs reference	difference	odds ratio
First parcel late, against on time					
30	82,422	0.56	0.41 vs 0.57	−0.16 [−0.30, +0.03]	0.79 [0.53, 1.18]
90	72,007	1.16	0.86 vs 1.19	−0.33 [−0.55, −0.05]	0.79 [0.59, 1.06]
180	50,360	1.84	1.44 vs 1.86	−0.43 [−0.82, +0.08]	0.83 [0.61, 1.13]
365	18,546	3.21	2.06 vs 3.26	−1.20 [−2.07, +0.20]	0.64 [0.37, 1.09]
First review 1–2 stars, against 4–5 stars					
30	82,422	0.56	0.42 vs 0.59	−0.17 [−0.29, −0.02]	0.75 [0.55, 1.02]
90	72,007	1.16	0.88 vs 1.23	−0.36 [−0.55, −0.13]	0.74 [0.59, 0.93]
180	50,360	1.84	1.50 vs 1.90	−0.40 [−0.71, −0.03]	0.81 [0.65, 1.01]
365	18,546	3.21	2.87 vs 3.33	−0.46 [−1.18, +0.43]	0.86 [0.65, 1.14]

order never came; the second counts only the customers each contact could still reach at its own moment. The two contacts Section 9.2 recommends are the last two rows.

Table 37: Net value per 1,000 reviewed test orders at the assumed R$5 : R$40, by the share of reached 1–2 star customers a contact turns around, from `customer_value.csv`. Hits: listed customers who went on to rate the order 1–2 stars. "Must turn around": the break-even share; above 100% the contact cannot pay even if it turns every reached customer around.

				share turned around			must turn
List	listed	hits	precision	all	half	a quarter	around
Every hit counted, as on the slides							
Slides: threshold 0.11	1,927	712	0.369	1,466	358	−196	33.8%
Slides: ten a day by purchase day	614	351	0.572	853	307	34	21.9%
Deployed threshold 0.19	981	564	0.575	1,373	496	57	21.7%
Only customers the contact can still reach at its moment							
Threshold 0.11, call at delivery	1,927	282	0.146	128	−311	−530	85.4%
Slides' ten a day, call at delivery	614	54	0.088	−71	−155	−197	142.1%
Threshold 0.19, call at delivery	981	137	0.140	45	−168	−275	89.5%
Promised-date message	804	440	0.547	1,056	372	30	22.8%
Delivery-day call, ten a day	546	142	0.260	229	9	−102	48.1%

Table 38 splits the figures quoted on the slides by whom a contact would have reached. Of the slides' threshold-0.11 list, R$877 per 1,000 of gross value comes from delivered customers who had not yet written, R$647 from orders that never arrived and R$691 from late parcels already rated while the customer waited, against R$750 of contacts: 60.4% of the gross value is customers whose order never came or who rated a late parcel before it arrived. A call at delivery alone nets R$128 on that list and −R$71 on the ten-a-day list.

Table 38: Where the slides' values come from, per 1,000 reviewed test orders at the assumed R$5 : R$40 with every contact counted as turning its customer around, from `customer_value_decomposition.csv`. "Not written": the hits who had not rated the order before the parcel arrived, plus every hit whose order never arrived, whenever they rated (the slides' R$775 and R$433).

	threshold 0.11 (slides)	**threshold 0.19 (deployed)**	**ten a day (slides)**
Customers listed	1,927	981	614
Hits: delivered, not yet written	282	137	54
never delivered	208	208	162
late, written before arrival	222	219	135
Gross R$: delivered, not yet written	877	426	168
never delivered	647	647	504
late, written before arrival	691	681	420
Contacts at R$5	−750	−382	−239
Net, every hit counted	1,466	1,373	853
Net, not written before arrival	775	692	433
Net, a call at delivery alone	128	45	−71
Gross share: undelivered or written first	60.4%	75.7%	84.6%

AI Tool Declaration

We used two AI assistants during this project, under the course's academic-integrity rules, and declare their use here.

Anthropic's Claude (Claude Code, an agentic coding assistant) was used as a programming and writing assistant working under our direction: it drafted and refactored Python code in `src/`, `deployment/` and `tests/` from our specifications; helped locate and fix bugs in the data pipeline, the training scripts and the Streamlit application; ran the regeneration scripts whose outputs (`report/results/`, `report/stats.txt`, `report/stats_phase3.txt`) are the source of the numbers quoted here; typeset this report in LaTeX and helped build the slide deck from our outlines, results and the evaluators' feedback; and cross-checked the manuscript and the slides against the result tables before submission.

OpenAI's ChatGPT was used to generate and validate ideas, to improve expression and to refine our writing.

The revision after the presentation of 5 November 2026 used Claude Code in the same role, at the team's direction: it proposed and drafted the changes made in response to the feedback — the revised sections and appendices of this report, the application's new Customer journey tab and the changes to its other tabs, and two new analysis scripts, `src/customer_view.py` and `src/slide_provenance.py` — and it labelled the 175 held-out comments behind the complaint-topic precision check (`customer_topic_labels.csv`, Appendix Q). Every number the revision adds is read from a script output in `report/results/`, `report/stats_phase3.txt` or `report/stats.txt`, or recomputed from the Olist tables; assertions in the new scripts require the earlier results (the deployed models' test predictions, and the lists and values shown on the slides) to be reproduced exactly, and `src/slide_provenance.py` recomputes the 88 rows of the slide-provenance table that carry a checkable number and stops if one no longer matches its slide (Appendix P). The revision adds no reference.

The choice of problems, the analysis design (the prediction moments, the leakage regimes, the chronological split, the pre-registered success criteria and selection rule), the interpretation of the results and the business recommendations of the report as first submitted are the team's own, and for that report we reviewed every AI-assisted piece of code and text, verified every number against the underlying data by re-running the scripts, and checked every reference against the publisher's record or DOI. We are responsible for the content and quality of the submitted work.